Parallel Computing

Mihir Rao

Contents

1 Why Parallelism? Why Efficiency? 4
1.1 Course themes . . . 4
1.2 Programs and processors . . . 4
1.3 Instruction-level parallelism . . . 6
1.4 The end of free performance . . . 7
1.5 Efficiency and specialization . . . 8
1.6 Memory, latency, and caches . . . 8

2 A Modern Multi-Core Processor 10
2.1 Idea 1: multiple cores . . . 10
2.2 Idea 2: SIMD processing . . . 11
2.3 Accessing memory . . . 14
2.4 Idea 3: hardware multithreading . . . 15
2.5 How the ideas combine . . . 17

3 Multi-Core Architecture Part II and ISPC Programming Abstractions 18
3.1 Latency and bandwidth . . . 18
3.2 Abstraction versus implementation . . . 20
3.3 ISPC . . . 20
3.4 ISPC: abstraction and implementation . . . 24

4 Parallel Programming Basics 26
4.1 Creating a parallel program . . . 26
4.2 Decomposition and Amdahl's law . . . 26
4.3 Assignment, orchestration, and mapping . . . 28
4.4 Case study: a grid solver . . . 29
4.5 Data-parallel expression . . . 31
4.6 Shared address space expression . . . 31

5 Performance Optimization I: Work Distribution and Scheduling 35
5.1 Balancing the workload . . . 35
5.2 Task granularity . . . 36
5.3 Smarter task scheduling . . . 37
5.4 Fork-join parallelism . . . 38
5.5 Implementing fork-join: work stealing . . . 39
5.6 Implementing sync . . . 41

6 Performance Optimization II: Locality, Communication, and Contention 43
6.1 Shared address space hardware . . . 43
6.2 Message passing . . . 43
6.3 Communication and arithmetic intensity . . . 46
6.4 Inherent and artifactual communication . . . 47
6.5 Techniques for reducing communication . . . 48

6.6 Contention . . . 49
6.7 Understanding parallel performance . . . 50
6.8 Problem size and scaling . . . 51

7 GPU Architecture and CUDA Programming **53**
7.1 Graphics and the origins of GPU computing . . . 53
7.2 CUDA abstractions . . . 54
7.3 Implementing CUDA . . . 58
7.4 CUDA semantics in summary . . . 60

8 Data-Parallel Thinking **61**
8.1 Sequences and map, fold, scan . . . 61
8.2 Parallel scan . . . 62
8.3 Segmented scan . . . 64
8.4 Gather, scatter, and other operations . . . 66
8.5 Example: building a grid of particles . . . 66

9 Distributed Data-Parallel Computing Using Spark **69**
9.1 Clusters and distributed storage . . . 69
9.2 MapReduce . . . 69
9.3 Spark and resilient distributed datasets . . . 71
9.4 Implementing RDDs . . . 73

10 Efficiently Evaluating DNNs on GPUs **77**
10.1 Convolutional neural networks . . . 77
10.2 Approach 1: more efficient topologies . . . 78
10.3 Approach 2: implementing layers efficiently . . . 78
10.4 Fusion . . . 82
10.5 Hardware for DNNs . . . 83

11 Cache Coherence **85**
11.1 Cache design and write policies . . . 85
11.2 The coherence problem . . . 85
11.3 Snooping coherence . . . 87
11.4 Directory-based coherence . . . 90
11.5 Implications for the programmer . . . 90

12 Memory Consistency **92**
12.1 Coherence versus consistency . . . 92
12.2 Relaxed consistency . . . 93
12.3 Synchronization . . . 94
12.4 Language memory models . . . 95

13 Fine-Grained Synchronization and Lock-Free Programming **96**
13.1 Deadlock, livelock, and starvation . . . 96
13.2 Implementing locks . . . 97
13.3 Fine-grained locking . . . 100
13.4 Lock-free data structures . . . 103

14 Domain-Specific Programming Languages **109**
14.1 Optimizing image processing by hand . . . 109
14.2 Halide . . . 112
14.3 Liszt . . . 115
14.4 Designing domain-specific systems . . . 117

15 Transactional Memory 1 **118**
15.1 Programming with transactions . . . 118
15.2 Motivation . . . 119
15.3 Implementing transactional memory . . . 121
15.4 Software transactional memory . . . 123

16 Transactional Memory 2 **125**
16.1 Challenges and optimization of STM . . . 125
16.2 Hardware transactional memory . . . 127
16.3 Transactional coherence and consistency . . . 128
16.4 HTM in Intel Haswell . . . 129

17 Hardware Specialization **130**
17.1 Heterogeneous processing . . . 130
17.2 Energy-constrained computing . . . 130
17.3 Specialized processors . . . 131
17.4 Programming accelerators: Spatial . . . 133
17.5 Streaming execution and attention . . . 134
17.6 Moving less data . . . 136

18 Accessing Memory and Course Wrap Up **137**
18.1 How DRAM works . . . 137
18.2 Bringing memory closer . . . 139
18.3 Course wrap up . . . 139

Lecture 1: Why Parallelism? Why Efficiency?

Definition: A parallel computer is a collection of processing elements that cooperate to solve problems quickly. We care about performance and about efficiency, and we use multiple processors to get them.

The main motivation for parallel processing is speedup. For a given problem,

$$\text{speedup}(P \text{ processors}) = \frac{\text{execution time using 1 processor}}{\text{execution time using } P \text{ processors}}.$$

Three in-class experiments, in which students summed numbers in parallel, show what limits speedup. Communication (telling each other partial sums) limited the maximum speedup, and reducing its cost (moving the "processors" closer together) improved it. An imbalanced assignment of work left some processors idle while others still worked, and a better distribution of work improved speedup. When a problem has much communication relative to computation, communication costs can dominate and severely limit speedup.

1.1 Course themes

1. *Designing and writing parallel programs that scale.* Parallel thinking means decomposing work into pieces that can safely run in parallel, assigning the work to processors, and managing communication and synchronization so they do not limit speedup; we also need abstractions and mechanisms for these tasks, and code in popular parallel programming languages.
2. *Parallel hardware implementation.* The mechanisms that implement the abstractions efficiently, their performance characteristics, and the design trade-offs among performance, convenience, and cost. The characteristics of the machine matter (as the communication speed in the experiments showed), and we care about efficiency and performance.
3. *Efficiency.* Fast is not the same as efficient: a program that runs faster in parallel may still use the hardware poorly (is a 2× speedup on 10 processors good?). The programmer's job is to use the machine's capabilities; the hardware designer's job is to choose the right capabilities given performance and cost (silicon area, power, and so on).

1.2 Programs and processors

For decades single-threaded CPU performance doubled about every 18 months, so parallelizing code was rarely worth the effort: the same code simply ran faster next year. Until about fifteen

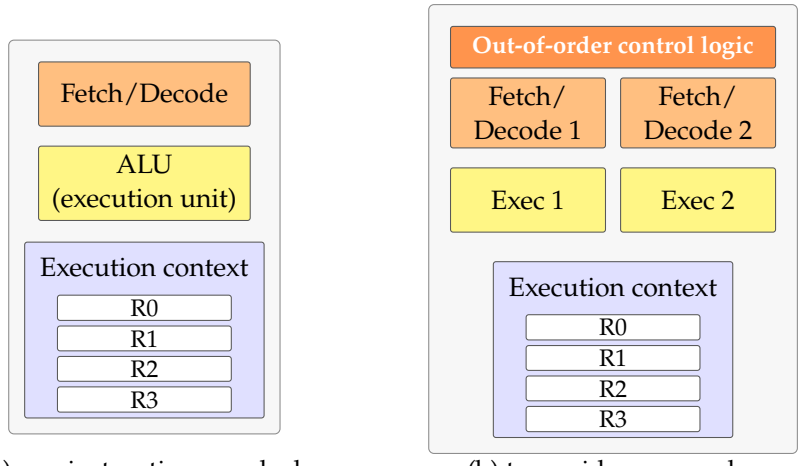
years ago the two main sources of improvement were exploiting INSTRUCTION-LEVEL PARALLELISM (superscalar execution) and increasing the clock frequency.

From a processor's point of view a program is just a list of instructions. The C program

```c
int main(int argc, char** argv) {
  int x = 1;
  for (int i=0; i<10; i++) {
    x = x + x;
  }
  printf("%d\n", x);
  return 0;
}
```

compiles to about thirty x86-64 instructions (`movl`, `cmpl`, `jge`, `addl`, `jmp`, `callq`, ...). An INSTRUCTION describes an operation for the processor to perform; executing it typically modifies the computer's STATE, the values of program data stored in the processor's registers or in memory.

A simple processor (Figure 1a) has three parts. The FETCH/DECODE unit determines which instruction to run next. The ALU (execution unit) performs the operation an instruction describes, which may modify the registers or memory. The EXECUTION CONTEXT (registers) holds the program state: the variables used as inputs and outputs of operations. To execute `add R0 <- R0, R1` with `R0` = 32 and `R1` = 64, the processor fetches the instruction, reads both registers into the execution unit, adds them, and stores 96 back into `R0`. This processor executes one instruction per clock.



(a) one instruction per clock (b) two-wide superscalar

Figure 1: (a) A simple processor: fetch/decode, one ALU, and an execution context of registers; it executes one instruction per clock. (b) A superscalar processor that decodes and executes up to two independent instructions per clock from one instruction stream.

1.3 Instruction-level parallelism

Consider $a = x^2 + y^2 + z^2$ with `R0` $= x$, `R1` $= y$, `R2` $= z$:

```
1 mul R0, R0, R0   4 add R0, R0, R1
2 mul R1, R1, R1   5 add R3, R0, R2
3 mul R2, R2, R2   (R3 now holds a)
```

Executed one per clock it takes five clocks. With two execution units we may run instructions simultaneously provided the schedule respects program order, meaning every instruction runs after the instructions it depends on: one valid schedule runs $\{1, 2\}$, then $\{3, 4\}$, then 5, in three clocks. With three units, $\{1, 2, 3\}$, 4, 5 also takes three clocks; the dependences of the adds prevent anything better (Figure 2).

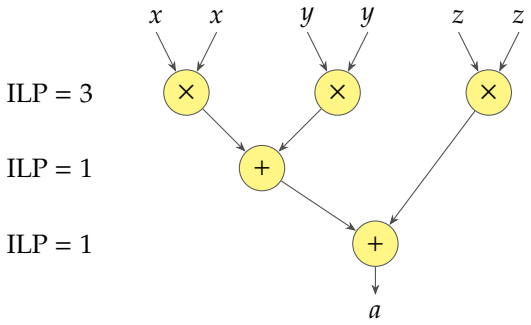


Figure 2: Dataflow of $a = x \cdot x + y \cdot y + z \cdot z$. The three multiplies are independent (ILP = 3); each add depends on the previous step (ILP = 1).

Definition: In SUPERSCALAR EXECUTION the processor automatically finds independent instructions in an instruction sequence and executes them in parallel on multiple execution units (alternatively, the compiler finds them at compile time and encodes the dependencies in the binary). Here instructions 1–3 can run in parallel without affecting correctness, but 4 must follow 1 and 2, and 5 must follow 4.

A two-wide superscalar core (Figure 1b) has two fetch/decode units, two execution units, out-of-order control logic, and still one execution context: it runs one instruction stream faster. Real cores such as the Pentium 4 contain much more of this machinery: several instruction decoders, a register alias table, a reorder buffer, reservation stations, and a branch target buffer. Figure 3 shows the dependency graph of a longer program.

Superscalar execution has diminishing returns. Most of the available ILP is exploited by a processor that can issue four instructions per clock; in the data of Johnson (1991), reported by Culler and Singh, the speedup levels off below 3× and building processors that issue 8 or 16 instructions per clock brings little further benefit.

PC	Instruction	Value
00	`a = 2`	
01	`b = 4`	
02	`tmp2 = a + b`	6
03	`tmp3 = tmp2 + a`	8
04	`tmp4 = b + b`	8
05	`tmp5 = b * b`	16
06	`tmp6 = tmp2 + tmp4`	14
07	`tmp7 = tmp5 + tmp6`	30
08	`if (tmp3 > 7)`	
09	`  print tmp3`	
	`else`	
10	`  print tmp7`	

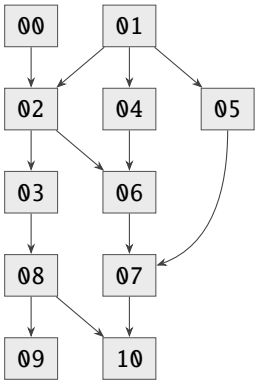


Figure 3: A program (with each value computed during execution) and its instruction dependency graph. Instructions in the same row of the graph can execute in parallel once their inputs are ready.

1.4 The end of free performance

Transistor density kept growing, but around 2005 clock rates stopped increasing and ILP stopped providing benefit. The cause of the frequency limit is power.

Definition: A transistor consumes DYNAMIC POWER

$$P_{\text{dynamic}} \propto \text{capacitive load} \times \text{voltage}^2 \times \text{frequency},$$

and STATIC POWER, since transistors leak and burn power even when inactive. The maximum allowed frequency is determined by the core voltage, so raising the frequency also requires raising the voltage, and power grows much faster than linearly with clock speed.

High power means high heat, so power is a critical design constraint (the POWER WALL). Typical thermal design power (TDP) figures:

Device	TDP
Mobile phone processor	1/2–2 W
Apple M1 laptop	13 W
Intel Core i9 10900K (desktop CPU)	95 W
NVIDIA RTX 4090 GPU	450 W
Standard microwave oven	900 W
World's fastest supercomputer	megawatts

Single-instruction-stream performance scaling has therefore slowed almost to zero: frequency scaling is limited by power and ILP scaling is tapped out. Architects now build faster processors by adding more execution units that run in parallel, or units specialized for a task such as graphics or

audio/video playback. Software must be written to be parallel to see performance gains; there is no more free lunch for software developers.

Parallel hardware is everywhere:

- multi-core CPUs, such as the 10-core Intel "Comet Lake" Core i9 (2020) and the 64-core, 4.3 GHz AMD Ryzen Threadripper 3990X built from 8-core chiplets;
- GPUs, such as the NVIDIA AD102 (GeForce RTX 4090, 2022) with 76 billion transistors and 18,432 fp32 multipliers organized in 144 processing blocks called SMs;
- supercomputers, such as Frontier at Oak Ridge (world's fastest in fall 2022): 9,472 64-core AMD CPUs (606,208 cores), 37,888 Radeon GPUs, and 21 MW;
- mobile systems, whose design is also shaped by power, such as the Apple A15 Bionic (15 billion transistors, a 6-core CPU with 2 "big" and 4 "small" cores, and a multi-core GPU) and the quad-core ARM A53 in a Raspberry Pi 3.

On a quad-core Intel CPU with AVX SIMD instructions and hyper-threading, a program that uses all the parallel execution resources can run about 32–40× faster than a single-threaded C baseline compiled with `-O3`.

1.5 Efficiency and specialization

Modern software must be not only parallel but also efficient. Systems achieve high power efficiency by also using specialized processing units: the A15 includes a neural engine (NPU) for DNN acceleration, an image/video encode/decode processor, and a motion (sensor) processor; datacenters use Google's TPU (Tensor Processing Unit) pods for machine learning; and many accelerators target DNN inference and training (Huawei Kirin NPU, Apple Neural Engine, GraphCore IPU, SambaNova SN10, Cerebras Wafer Scale Engine, GPUs with tensor cores). Achieving efficient processing almost always comes down to accessing data efficiently.

1.6 Memory, latency, and caches

A program's memory ADDRESS SPACE is an array of bytes, each identified by its ADDRESS, its position in the array (memory is byte-addressable). For a 32-byte address space the valid addresses are `0x0` to `0x1F`. A LOAD instruction moves data from memory into a register: `ld R0 <- mem[R2]` loads the four-byte value starting at the address held in `R2` into `R0`; a STORE moves data from a register to memory.

Definition: MEMORY ACCESS LATENCY is the time the memory system takes to provide data to the processor, for example 100 clock cycles or 100 ns. A processor STALLS when it cannot run the next instruction in the stream because it depends on a previous instruction that has not completed.

Memory access is a major source of stalls. In

```
ld r0 mem[r2];  ld r1 mem[r3];  add r0, r0, r1
```

the add cannot execute until both loads return, and memory access times are hundreds of cycles.

Definition: A CACHE is on-chip storage that keeps a copy of a subset of the values in memory. If an address is in the cache, the processor can load or store it more quickly than if it resides only in DRAM. A cache is a hardware implementation detail: it affects only performance, never the output of a program. Caches operate at the granularity of CACHE LINES.

Example: *Two access sequences.* Take an array of 16 bytes, a cache of 8 bytes with 4-byte lines (so two lines fit), and least recently used (LRU) replacement. The sequence `0x0, 0x1, 0x2, 0x3, 0x2, 0x1, 0x4, 0x1` incurs a COLD MISS at `0x0` (loading line `0x0`) and at `0x4` (loading line `0x4`); every other access hits. Reading all addresses `0x0`–`0xF` in order and then `0x0` again misses at `0x0`, `0x4`, `0x8` (evicting line `0x0`), and `0xC` (evicting line `0x4`), and hits on the other 12 addresses; the final access to `0x0` is a CAPACITY MISS (evicting line `0x8`), because the data was evicted only for lack of space.

These sequences show two forms of DATA LOCALITY. SPATIAL LOCALITY: loading one cache line preloads the data needed for later accesses to different addresses in the same line. TEMPORAL LOCALITY: repeated accesses to the same address hit. Processors run efficiently when they access data resident in caches, because caches reduce the latency of accessing recently used data (they also provide high-bandwidth data transfer), and so reduce the length of stalls.

On a modern computer the abstraction of a linear address space is implemented by a hierarchy of caches, so loading address X into `R0` may involve several caches and DRAM. Smaller caches near the processor have lower latency; larger caches farther away have larger latency. Typical sizes and the access latencies of an Intel Kaby Lake CPU at 4 GHz:

Level	Typical size	Latency (cycles)
L1 cache	32 KB	4
L2 cache	256 KB	12
L3 cache	20 MB	38
DRAM (best case)	64 GB	~248

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 1 (Fall 2023).

Lecture 2: A Modern Multi-Core Processor

Modern parallel processors reach high throughput through three ideas: two concern parallel execution (multiple cores and SIMD execution) and one addresses the latency of memory (hardware multithreading). The running example computes $\sin x$ for each element of an array of N floats by the Taylor expansion $\sin x = x - x^3/3! + x^5/5! - x^7/7! + \cdots$:

```
void sinx(int N, int terms, float* x, float* y)
{
  for (int i=0; i<N; i++)
  {
    float value = x[i];
    float numer = x[i] * x[i] * x[i];
    int denom = 6;  // 3!
    int sign = -1;

    for (int j=1; j<=terms; j++)
    {
      value += sign * numer / denom;
      numer *= x[i] * x[i];
      denom *= (2*j+2) * (2*j+3);
      sign *= -1;
    }
    y[i] = value;
  }
}
```

Compiled, the loop body is a stream of scalar instructions (`ld`, `mul`, . . . , `st`) that a simple core runs one per clock. A superscalar core does not help here, since this region of the program has no ILP.

2.1 Idea 1: multiple cores

Before the multi-core era, most of a chip's transistors made a single instruction stream run fast: a big data cache, out-of-order control logic, a fancy branch predictor, and a memory prefetcher. More transistors meant a larger cache and smarter control logic.

Definition: The multi-core idea: rather than using transistors to increase the sophistication of logic that accelerates a single instruction stream (out-of-order and speculative execution), use the growing transistor count to add more cores to the processor.

Simpler cores may each run a single instruction stream more slowly than the original fancy core, say 25% slower, but two of them offer a potential speedup of $2 \times 0.75 = 1.5$. However, the C program

above expresses no parallelism: it compiles to one instruction stream that runs as one thread on one core, and therefore now runs 25% slower. The programmer must expose parallelism, for example with C++ threads:

```
typedef struct {
  int N;
  int terms;
  float* x;
  float* y;
} my_args;

void my_thread_func(my_args* args)
{
  sinx(args->N, args->terms, args->x, args->y); // do work
}

void parallel_sinx(int N, int terms, float* x, float* y)
{
  std::thread my_thread;
  my_args args;
  args.N = N/2;
  args.terms = terms;
  args.x = x;
  args.y = y;

  my_thread = std::thread(my_thread_func, &args); // launch thread
  sinx(N - args.N, terms, x + args.N, y + args.N); // do work on main thread
  my_thread.join(); // wait for thread to complete
}
```

Alternatively, a data-parallel language construct can declare that loop iterations are independent. In a hypothetical language with a `forall` construct, replacing `for (int i=0; i<N; i++)` by `forall (int i from 0 to N)` declares the iterations independent, and with this information a compiler could generate threaded code automatically. Four cores compute four elements in parallel; sixteen cores run sixteen simultaneous instruction streams. Examples are the 10-core Intel "Comet Lake" Core i9, the GeForce RTX 4090 GPU with 144 processing blocks (SMs), and the Apple A15 with two "big" and four "small" CPU cores.

2.2 Idea 2: SIMD processing

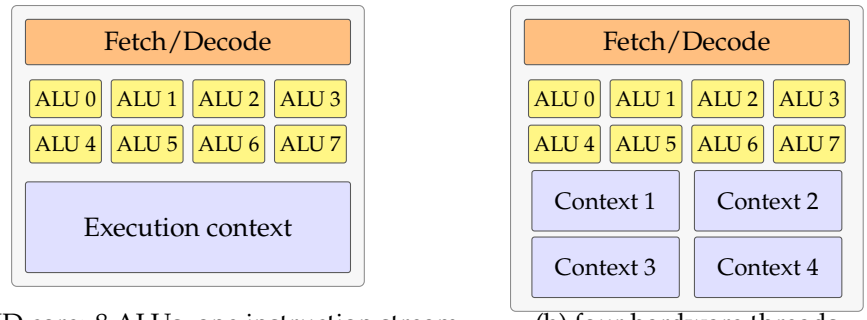
The `forall` loop has another property: every iteration executes the same sequence of instructions (the loop body computing $\sin x[i]$) on different input data.

Definition: SIMD PROCESSING (single instruction, multiple data) amortizes the cost and complexity of managing an instruction stream across many ALUs: one fetch/decode unit broadcasts the same instruction to all ALUs, which execute it in parallel on different data (Figure 4a).

With AVX intrinsics, which give C programmers vector datatypes and functions operating on eight 32-bit values, the program becomes:

```
#include <immintrin.h>
void sinx(int N, int terms, float* x, float* y)
{
  float three_fact = 6; // 3!
  for (int i=0; i<N; i+=8)
  {
    __m256 origx = _mm256_load_ps(&x[i]);
    __m256 value = origx;
    __m256 numer = _mm256_mul_ps(origx, _mm256_mul_ps(origx, origx));
    __m256 denom = _mm256_broadcast_ss(&three_fact);
    int sign = -1;
    for (int j=1; j<=terms; j++)
    {
      // value += sign * numer / denom
      __m256 tmp = _mm256_div_ps(_mm256_mul_ps(_mm256_set1ps(sign), numer), denom);
      value = _mm256_add_ps(value, tmp);
      numer = _mm256_mul_ps(numer, _mm256_mul_ps(origx, origx));
      denom = _mm256_mul_ps(denom, _mm256_broadcast_ss((2*j+2) * (2*j+3)));
      sign *= -1;
    }
    _mm256_store_ps(&y[i], value);
  }
}
```

It compiles to vector instructions (`vloadps`, `vmulps`, ..., `vstoreps`) on 256-bit vector registers, processing eight array elements simultaneously. Sixteen such SIMD cores process 128 elements in parallel (16 cores, 128 ALUs, 16 simultaneous instruction streams). The `forall` abstraction lets a compiler generate both multi-core parallel code and vector instructions within a core.



(a) SIMD core: 8 ALUs, one instruction stream (b) four hardware threads

Figure 4: (a) A SIMD core: one instruction stream drives eight ALUs. (b) The same core with storage for four hardware threads (execution contexts), used for latency hiding in Section 2.4.

Conditional execution. When the loop body branches, different lanes may need different instructions. All lanes evaluate the condition; lanes where it holds execute the `if` branch while the outputs of the other ALUs are masked (discarded), and then the roles swap for the `else` branch (Figure 5). Not all ALUs do useful work. In the worst case, when one lane runs a long branch while the other seven are masked, an 8-wide unit runs at 1/8 of peak performance. After the branch, execution continues at full performance.



Figure 5: Executing `if (t > 0.0) {...} else {...}` on an 8-wide SIMD unit with conditions T T F T F F F F. Crossed cells are masked ALUs doing no useful work.

Definition: Instruction stream coherence (coherent execution) is the property that the same instruction sequence applies to many data elements. It is necessary for SIMD resources to be used efficiently, but not for efficient parallelization across cores, since each core can fetch and decode a different instruction stream. Divergent execution is a lack of instruction stream coherence.

On CPUs, Intel AVX2 instructions operate on 256 bits (8×32-bit or 4×64-bit values, so 8-wide float vectors), AVX-512 on 512 bits (16×32 bits), and ARM Neon on 128 bits (4×32 bits). The compiler generates these instructions, with parallelism explicitly requested through intrinsics, conveyed by parallel language semantics (`forall`), or inferred by dependency analysis of loops in an auto-vectorizing compiler. This is explicit SIMD: SIMD parallelization happens at compile time,

and the binary contains SIMD instructions such as `vstoreps` and `vmulps`.

Many GPUs use IMPLICIT SIMD: the compiler generates a binary of scalar instructions, but N instances of the program always run together on the processor, and the hardware (not the compiler) executes the same instruction from multiple instances on different data on SIMD ALUs. GPU SIMD widths range from 8 to 32, so divergence can be a big issue: poorly written code might run at 1/32 of the machine's peak.

Form	What runs in parallel
Superscalar	Different instructions from the same instruction stream, within a core (ILP); parallelism discovered automatically by the hardware during execution.
SIMD	Multiple ALUs controlled by the same instruction, within a core; efficient for data-parallel workloads since control costs are amortized; vectorized by the compiler (explicit) or at run time by hardware (implicit).
Multi-core	Completely different instruction streams on different cores (thread-level parallelism); software creates threads to expose parallelism to the hardware, for example through a threading API.

Example: *Peak throughput.* A four-core Intel i7-7700K (Kaby Lake) has three 8-wide AVX2 SIMD ALUs per core: 4 cores $\times$ 8 $\times$ 3 $\times$ 4.2 GHz $\approx$ 400 GFLOPs. An NVIDIA V100 GPU has 80 SM cores with 128 SIMD ALUs each at 1.6 GHz, about 16 TFLOPs at roughly 250 W.

2.3 Accessing memory

Recall the long latency of data access: about 4 cycles in L1, 12 in L2, 38 in L3, and about 248 in DRAM on a 4 GHz Kaby Lake. In a multi-core CPU each core has private L1 (32 KB) and L2 (256 KB) caches, all cores share an L3 cache (for example 8 MB), and DRAM (gigabytes of DDR4) is reached at about 38 GB/s. Caches reduce stalls when a processor reuses recently accessed data, and also provide high bandwidth.

In the second access sequence of Lecture 1 (reading a 16-byte array twice through an 8-byte LRU cache), the second read of `0x0` misses because sweeping through four lines evicts each line before it is reused; the second read of `0x4` misses for the same reason. With a capacity of four lines the whole array fits, and every access of the second pass would hit.

Definition: PREFETCHING hides latency: many CPUs dynamically analyze a program's access pattern, predict which data will be accessed, and initiate loads early, so that the data is already in cache when the load instruction executes. A wrong guess can reduce performance by consuming bandwidth and polluting the caches.

Neither caching nor prefetching helps when data has not been read recently and the next address is not predictable, as in `int x = some_function(); int y = A[x];`.

2.4 Idea 3: hardware multithreading

Definition: MULTITHREADING interleaves the processing of multiple threads on the same core to hide stalls: if the core cannot make progress on the current thread, it works on another one. The core stores one EXECUTION CONTEXT per HARDWARE THREAD (Figure 4b).

Consider one core with four hardware threads, each processing eight elements on the 8-wide SIMD unit (thread 1 elements 0–7, thread 2 elements 8–15, and so on). When thread 1 stalls on a memory access, the core switches to thread 2; when thread 2 stalls it runs thread 3, and so on. By the time thread 4 stalls, thread 1's data has arrived and it is runnable again. This is the key trade-off of THROUGHPUT-ORIENTED systems: they potentially increase the time to complete any one thread's work in order to increase overall throughput when running multiple threads. A thread may be runnable yet not executing, because the core is running another thread.

On-chip storage for execution contexts is a finite resource (the same storage could serve as L1 cache). It can hold many small contexts, such as 16 hardware threads with a small working set each (high latency-hiding ability), or a few large ones, such as 4 threads with a large working set each (low latency-hiding ability).

Example: *How many threads hide the latency?* A core issues one scalar instruction per clock from one of its hardware threads. Each thread performs three arithmetic instructions and then a memory load with a 12-cycle latency. One thread keeps the core busy 3 of every 15 cycles (20% utilization); two threads give 6/15 = 40%; five threads give 100%, and more threads add nothing (Figure 6a). If each thread instead performs six arithmetic instructions per load, three threads reach 100% (Figure 6b). In general, with c cycles of arithmetic per load of latency L, full utilization needs about $(c + L)/c$ threads.

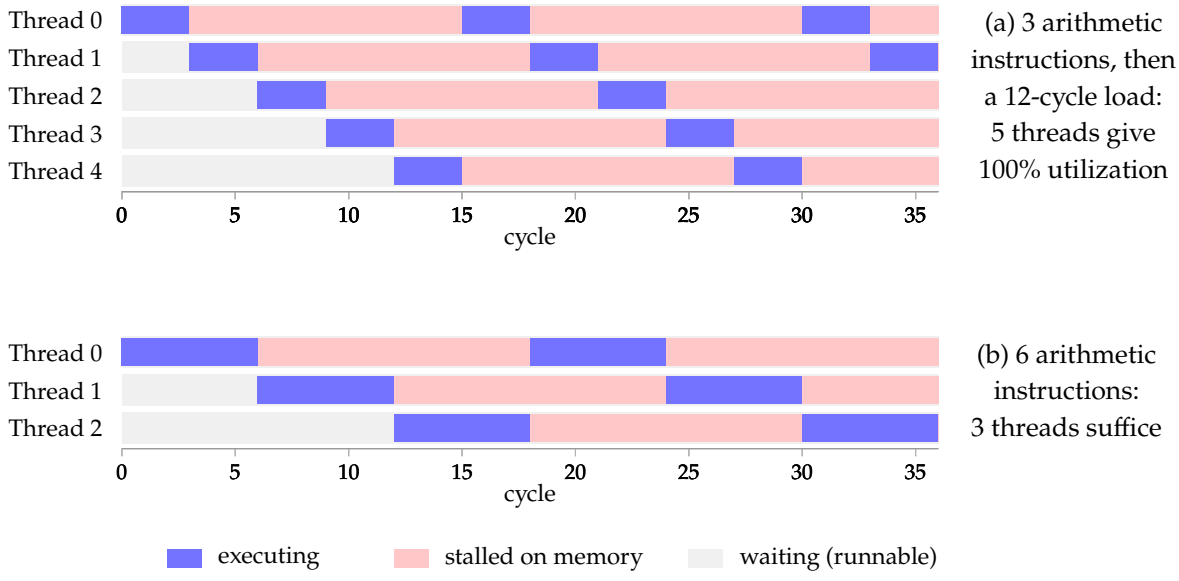


Figure 6: Interleaved multithreading on a core that issues one instruction per clock, with a 12-cycle memory latency. Each thread computes, stalls on its load, and resumes when the core is free.

Two takeaways. A processor with multiple hardware threads can avoid stalls by executing instructions from other threads while one thread waits for a long-latency operation; the latency of the memory operation is unchanged, it just no longer reduces processor utilization. And a multithreaded processor hides memory latency by performing arithmetic from other threads, so programs with more arithmetic per memory access need fewer threads to hide stalls.

Definition: In hardware-supported multithreading the core manages execution contexts for multiple threads and decides each clock which thread to run; it has the same ALUs, and multithreading only helps use them efficiently in the face of long-latency operations. In INTERLEAVED (TEMPORAL) MULTITHREADING the core chooses one thread each clock and runs an instruction from it. In SIMULTANEOUS MULTITHREADING (SMT) the core chooses instructions from multiple threads each clock to run on its ALUs; Intel Hyper-threading runs two threads per core.

Example: *Putting the ideas together.* A hypothetical chip with 16 cores, 8 SIMD ALUs per core (128 total), and 4 threads per core runs 16 simultaneous instruction streams and holds 64 concurrent instruction streams; it needs $16 \cdot 4 \cdot 8 = 512$ independent pieces of work to run with maximal latency-hiding ability.

Real cores. An Intel Skylake/Kaby Lake core is two-way multithreaded (two execution contexts). Each clock it can run up to four independent scalar instructions and up to three 8-wide vector instructions (up to two vector multiplies or three vector adds); load and store units come in addition.

An NVIDIA V100 STREAMING MULTIPROCESSOR (SM) is an extreme throughput-oriented core. It has four sub-cores, each with a warp selector and fetch/decode unit, and 64 "warp" execution contexts in total. Its SIMD fp32 and int units are 16 wide, carrying out one 32-wide SIMD operation every two clocks; fp64 units are 8 wide (one 32-wide operation every four clocks); there are also tensor cores and load/store units. Registers total 256 KB per SM (64 KB per sub-core), divided among up to 64 warps, and 128 KB of storage serves as "shared" memory and L1 cache. So up to $64 \times 32 = 2048$ data items are processed concurrently per SM, and with 80 SMs the V100 processes 163,840 pieces of data concurrently for maximal latency hiding. The chip has a 6 MB L2 cache and 16 GB of HBM GPU memory delivering 900 GB/s over a 4096-bit interface.

To use modern parallel processors efficiently, an application must:

1. have sufficient parallel work to use all available execution units, across many cores and many execution units per core;
2. group parallel work items that require the same sequences of instructions, to use SIMD execution;
3. expose more parallel work than there are ALUs, so that work can be interleaved to hide memory stalls.

2.5 How the ideas combine

The ideas compose into progressively richer cores. Each line of the table describes what one core can start per clock.

Core	Per clock
Simple, single-threaded	one scalar instruction
Two-way superscalar	up to two independent scalar instructions from one instruction stream (one hardware thread)
SIMD	one 8-wide vector instruction
Heterogeneous superscalar (scalar + SIMD)	up to two independent instructions from one stream, one scalar and one vector
Multithreaded (2 threads)	one scalar instruction from one of the threads; threads may run completely different instruction streams and are interleaved in time
Multithreaded superscalar (4 threads)	up to two independent instructions (one scalar, one vector) from any of the threads, so instructions from different threads may run simultaneously
Dual-core, 4 threads per core	each core as above, with a shared data cache: 8 resident instruction streams

Many GPUs execute hardware threads whose instruction streams contain only scalar instructions (SIMT, single instruction multiple thread). A GPU core detects when different hardware threads execute the same instruction and runs up to SIMD-width of them simultaneously on its SIMD ALUs; a thread executing a different instruction has its ALU masked off (divergent execution).

Mapping an application's threads onto the processor's hardware execution contexts is the job of the operating system. On a processor with two cores and two execution contexts per core, two application threads are best placed on different cores so that they do not compete for one core's execution units; with more threads than contexts, the operating system time-multiplexes them.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 2 (Fall 2023).

Lecture 3: Multi-Core Architecture Part II and ISPC Programming Abstractions

3.1 Latency and bandwidth

Consider element-wise multiplication of two vectors with millions of elements: load $A[i]$, load $B[i]$, compute $A[i] \times B[i]$, store the result into $C[i]$. Is this a good application for a throughput-oriented processor? The NVIDIA V100 has 80 SMs with 64 fp32 ALUs each, 5120 ALUs in total, and every clock all of them need data. Answering requires distinguishing latency from bandwidth.

Example: *A highway.* Driving the 50 km from San Francisco to Stanford at 100 km/h has a latency of 0.5 hours. If only one car may be on the road at a time, throughput is 2 cars per hour. Driving at 200 km/h doubles throughput to 4 cars per hour; building four lanes gives 8 cars per hour (2 per lane). Using the road more efficiently, with cars spaced 1 km apart, gives 100 cars per hour in one lane and 400 in four lanes, without changing anyone's latency.

Definition: Memory bandwidth is the rate at which the memory system can provide data to a processor, for example 20 GB/s. It is distinct from the latency of transferring any one item: a memory system can deliver more items per second while each item still takes the same time to arrive.

Pipelining raises throughput without duplicating resources. Laundry takes 45 minutes to wash, 60 to dry, and 15 to fold, so one load has a latency of 2 hours. Two washers, two dryers, and a friend double throughput to 1 load per hour by doubling resources. Starting the next wash while the previous load dries also achieves 1 load per hour, with one washer and one dryer and the same 2-hour latency per load. A pipeline runs at the rate of its slowest stage: connecting a pipe with maximum flow 100 L/s to one with 50 L/s gives at most 50 L/s.

Bandwidth-bound execution. Many threads each repeat three dependent instructions: `X = load 64 bytes`, `Y = add x + x`, `Z = add x + y`. They run on a multithreaded core with plenty of hardware threads to hide latency, which executes one math operation per clock, can issue loads in parallel with math, receives 8 bytes per clock from memory (so each load needs 8 clocks of transfer), and allows up to three outstanding loads. Initially the core issues an add every clock while loads accumulate; once three loads are outstanding, issuing a fourth stalls. In steady state memory transfers data 100% of the time and cannot go faster, so the core completes only the 2 adds enabled by each 64-byte load per 8 clocks (Figure 7). The rate of instructions is determined by the rate at which memory provides data: execution is memory bandwidth bound. In steady state, core utilization depends only on instruction throughput and memory throughput, not on memory latency or the number of outstanding requests.

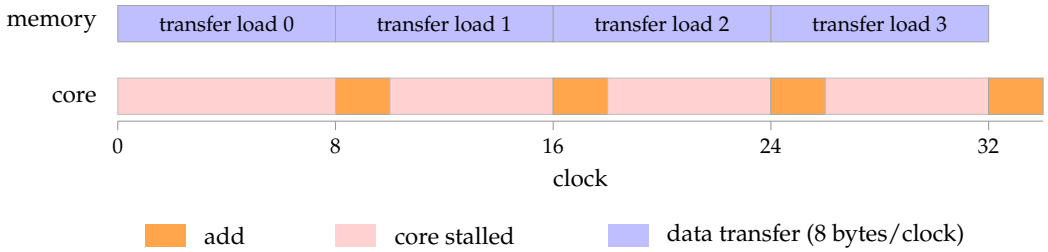


Figure 7: Steady state of the bandwidth-bound example. Memory is busy all the time, delivering one 64-byte load per 8 clocks; each load enables two adds, so the core is busy 2 of every 8 clocks.

Modern GPUs use high-bandwidth memories located near the processor; the V100 uses HBM2 at 900 GB/s.

Example: *The vector product, revisited.* Each multiply needs three memory operations (12 bytes). The V100 performs 5120 fp32 multiplies per clock at 1.6 GHz, so keeping its ALUs busy would need about 98 TB/s ($5120 \times 1.6 \times 10^9 \times 12$ bytes per second), against 900 GB/s available: under 1% efficiency. That is still much faster than an eight-core 3.2 GHz Xeon E5v4 on a 76 GB/s memory bus, which reaches about 3% efficiency on this computation.

The computation is bandwidth limited: if processors request data at too high a rate, the memory system cannot keep up. Overcoming bandwidth limits is often the most important challenge for software on throughput-optimized systems. Bandwidth is the critical resource, so performant parallel programs

- organize computation to fetch data from memory less often, by reusing data previously loaded by the same thread (temporal locality optimizations) and by sharing data across threads (inter-thread cooperation);
- favor performing additional arithmetic over storing and reloading values (the math is "free").

Programs must access memory infrequently to use modern processors efficiently.

Instruction pipelines. When a core "does one operation per clock" this refers to instruction throughput, not latency. A four-stage pipeline (instruction fetch IF; decode and register read D; execute EX; write back results WB) has a latency of 4 cycles per instruction but completes one instruction per cycle (Figure 8). Care is needed for correctness when back-to-back instructions are dependent. Real pipelines have variable length depending on the instruction and can be about 20 stages deep in modern CPUs.

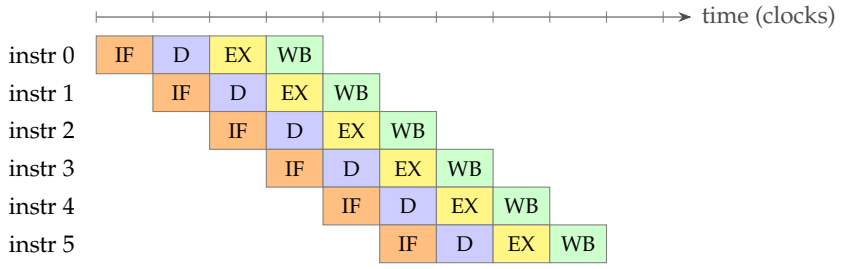


Figure 8: A four-stage instruction pipeline: latency 4 cycles, throughput 1 instruction per cycle.

3.2 Abstraction versus implementation

Conflating the semantics of an abstraction with the details of its implementation is a common source of confusion. The SEMANTICS of a programming model say what its operations mean: given a program, what answer will it compute? The IMPLEMENTATION (or SCHEDULING) says how the answer is computed on a parallel machine: in what (potentially parallel) order operations execute, and which operations each thread, execution unit, or vector lane performs. The goal is to be able, given a program and knowledge of how its programming model is implemented, to trace what each part of the machine does at each step.

3.3 ISPC

ISPC, the Intel SPMD Program Compiler, implements an SPMD (single program, multiple data) model. Calling the `sinx` example from C++ is an ordinary function call; in ISPC the function becomes:

```
export void ispc_sinx(
   uniform int N,
   uniform int terms,
   uniform float* x,
   uniform float* result)
{
   // assume N % programCount = 0
   for (uniform int i=0; i<N; i+=programCount)
   {
      int idx = i + programIndex;
      float value = x[idx];
      float numer = x[idx] * x[idx] * x[idx];
      uniform int denom = 6;  // 3!
      uniform int sign = -1;

      for (uniform int j=1; j<=terms; j++)
      {
         value += sign * numer / denom;
         numer *= x[idx] * x[idx];
         denom *= (2*j+2) * (2*j+3);
         sign *= -1;
      }
      result[idx] = value;
   }
}
```

and `main.cpp` calls `ispc_sinx(N, terms, x, result)` with $N = 1024$ and `terms` $= 5$.

Definition: In the SPMD abstraction, a call to an ISPC function spawns a GANG of PROGRAM INSTANCES. All instances run the ISPC code concurrently, each with its own copy of local variables, and when the call returns all instances have completed; sequential C execution then resumes. The language provides

- `programCount`: the number of simultaneously executing instances in the gang (a uniform value);
- `programIndex`: the id of the current instance in the gang (a non-uniform, "varying" value);
- `uniform`: a type modifier stating that all instances have the same value for the variable. Its use is purely an optimization and is not needed for correctness.

The code above assigns array elements to instances in an INTERLEAVED manner: in step i, instance k processes element $i + k$ (Figure 9a). A second version uses a BLOCKED assignment, with `uniform int count = N / programCount; int start = programIndex * count;` and loop `for (uniform int i=0; i<count; i++)` over `idx = start + i` (Figure 9b).

(a) interleaved

instance	0	1	2	3	4	5	6	7
$i = 0$	0	1	2	3	4	5	6	7
$i = 1$	8	9	10	11	12	13	14	15
$i = 2$	16	17	18	19	20	21	22	23
$i = 3$	24	25	26	27	28	29	30	31

(b) blocked

	0	1	2	3	4	5	6	7
$i = 0$	0	8	16	24	32	40	48	56
$i = 1$	1	9	17	25	33	41	49	57
$i = 2$	2	10	18	26	34	42	50	58
$i = 3$	3	11	19	27	35	43	51	59

Figure 9: Elements processed by each of the eight program instances in the first four loop steps. The highlighted row is one vector load: contiguous under interleaving, strided under blocking.

ISPC implements the gang with SIMD instructions. The number of instances is the SIMD width of the hardware (or a small multiple of it), and the compiler emits a C++-callable object file whose body contains SIMD instructions; the C++ code links against it as usual. The choice of assignment therefore matters for performance. Under interleaving, `float value = x[idx];` touches eight contiguous values in each step and is a single packed vector load (`vmovaps`, compare `_mm256_load_ps`). Under blocking, the eight instances touch eight non-contiguous values, which needs a GATHER instruction (`vgatherdps`, compare `_mm256_i32gather_ps`), a more complex and more costly SIMD instruction.

`foreach`. The key ISPC construct `foreach (i = 0 ... N)` declares parallel loop iterations: the programmer states the iterations that the entire gang (not each instance) must perform, and ISPC takes responsibility for assigning iterations to instances. Possible implementations include: instance 0 executes all iterations; interleave iterations onto instances; block them; or assign them dynamically, each instance repeatedly claiming the next iteration with an atomic increment of a shared counter. With `foreach`, many simple programs read almost as if they were sequential: "independently, for each element of the input, do this."

Example: *What do these programs do?*

```
export void absolute_repeat(
   uniform int N,
   uniform float* x,
   uniform float* y)
{
   foreach (i = 0 ... N)
   {
      if (x[i] < 0)
         y[2*i] = -x[i];
      else
         y[2*i] = x[i];
      y[2*i+1] = y[2*i];
   }
}
```

```
export void shift_negative(
   uniform int N,
   uniform float* x,
   uniform float* y)
{
   foreach (i = 0 ... N)
   {
      if (i >= 1 && x[i] < 0)
         y[i-1] = x[i];
      else
         y[i] = x[i];
   }
}
```

The left program computes the absolute value of each element of `x` (called with $N/2$ elements) and writes it twice into `y`. The right program has undefined output, since multiple iterations may write the same location `y[i-1]`.

Reductions. Summing an array with `float sum = 0.0f;` accumulated inside a `foreach` and then returned is a compile-time type error: `sum` is varying, a separate variable per instance, and the C caller expects one `float`. Declaring `uniform float sum` and executing `sum += x[i]` is also a type error, because `x[i]` differs across instances. The correct version accumulates private partial sums without communication and combines them with a cross-instance primitive:

```
export uniform float sum_array(
   uniform int N,
   uniform float* x)
{
   uniform float sum;
   float partial = 0.0f;
   foreach (i = 0 ... N)
   {
      partial += x[i];
   }

   // reduce_add() is part of ISPC's cross
   // program instance standard library
   sum = reduce_add(partial);

   return sum;
}
```

`reduce_add()` returns a uniform float, the same total for all instances. ISPC's cross-instance operations include

Operation	Meaning
`uniform int64 reduce_add(int32 x)`	sum of `x` over all instances
`uniform int32 reduce_min(int32 a)`	minimum over all instances
`int32 broadcast(int32 value, uniform int index)`	value of instance `index` sent to all
`int32 rotate(int32 value, uniform int offset)`	for all i, value of instance i passed to instance $(i + \mathit{offset})$ mod `programCount`

The ISPC `sum_array` executes much like the following C code with AVX intrinsics, which keeps eight partial sums in one vector register and adds them at the end:

```
float sum_summary_AVX(int N, float* x) {

  float tmp[8];  // assume 16-byte alignment
  __mm256 partial = _mm256_broadcast_ss(0.0f);

  for (int i=0; i<N; i+=8)
    partial = _mm256_add_ps(partial, _mm256_load_ps(&x[i]));

  _mm256_store_ps(tmp, partial);

  float sum = 0.f;
  for (int i=0; i<8; i++)
    sum += tmp[i];

  return sum;
}
```

3.4 ISPC: abstraction and implementation

The programming abstraction is SPMD: the programmer thinks of running a gang as spawning `programCount` logical instruction streams, each with a different `programIndex`, and writes the program in those terms. The implementation is SIMD: the compiler emits vector instructions (AVX2, ARM Neon) that carry out the gang's logic, mapping conditional control flow onto vector instructions by masking lanes. Uniform values let implementation details peek through the abstraction, which makes ISPC's semantics tricky. More generally, SPMD means defining one function and running multiple instances of it in parallel on different inputs: a single thread of control calls the SPMD function, multiple logical threads of control run, and the single thread resumes when it returns.

The gang executes as SIMD instructions within one thread on one x86 core, so the code above uses only one core of a multi-core CPU. ISPC's TASK abstraction provides multi-core execution.

ISPC is a low-level language: by exposing `programIndex` and `programCount`, it lets the programmer define what work and data each instance handles, so one can write programs with undefined output or programs correct only for a particular `programCount`. It can also express advanced cooperation, such as the product of eight elements in $\lg 8 = 3$ steps, where `shift` passes values between instances:

```
// compute the product of all eight elements in the
// input array. Assumes the gang size is 8.
export void vec8product(
   uniform float* x,
   uniform float* result)
{
   float val1 = x[programIndex];
   float val2 = shift(val1, -1);

   if (programIndex % 2 == 0)
     val1 = val1 * val2;

   val2 = shift(val1, -2);
   if (programIndex % 4 == 0)
      val1 = val1 * val2;

   val2 = shift(val1, -4);
   if (programIndex % 8 == 0) {
      *result = val1 * val2;
   }
}
```

A higher-level language could remove `programIndex` and `programCount` and expect the programmer to use only `foreach`; then there is little need to think about instances at all, and everything outside a `foreach` must be uniform values and uniform logic. Going further, a language could forbid array indexing and instead invoke a function once per element of a "collection" data structure (`map(doWork, x)` applies `doWork` to every element of collection `x`), so the programmer writes no loops and performs no indexing. NumPy programmers know this style: `Z = X + Y` on arrays, or `np.vectorize(addOne)` applied to an array. Programming models provide a way to think about the organization of parallel programs through abstractions that permit multiple valid implementations.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 3 (Fall 2023).

Lecture 4: Parallel Programming Basics

This unit is a case study in writing and optimizing a parallel program, expressed in two programming models: data parallel and shared address space.

4.1 Creating a parallel program

The thought process is: identify work that can be performed in parallel; partition the work and the data associated with it; and manage data access, communication, and synchronization. A common goal is maximizing speedup for a fixed computation,

$$\text{speedup}(P) = \frac{\text{time(1 processor)}}{\text{time}(P \text{ processors})};$$

other goals include high efficiency (cost, area, power) and working on problems bigger than fit on one machine. Creating the program has four steps (Figure 10), whose responsibilities may fall on the programmer, on the system (compiler, runtime, hardware), or on both.

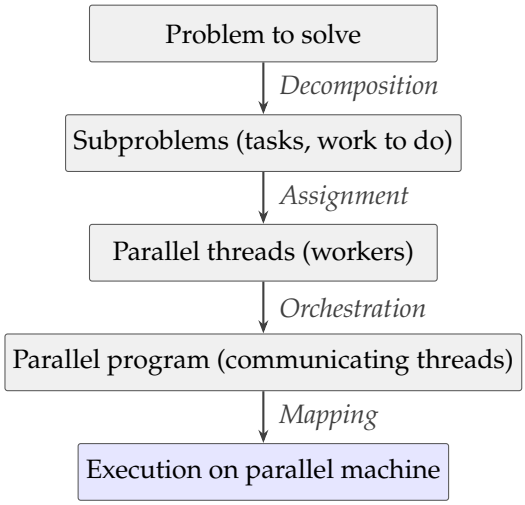


Figure 10: Creating a parallel program: decomposition, assignment, orchestration, and mapping (after Culler, Singh, and Gupta).

4.2 Decomposition and Amdahl's law

Decomposition breaks a problem into tasks that can be carried out in parallel; in general it should create at least enough tasks to keep all execution units busy. Its key challenge is identifying dependencies, or their absence.

Theorem 4.1 [Amdahl's law]. *Let S be the fraction of sequential execution that is inherently sequential, because dependencies prevent parallel execution. Then the maximum speedup due to parallel execution is at most $1/S$, and on P processors*

$$\text{speedup} \le \frac{1}{S + \frac{1-S}{P}}.$$

Proof. With normalized sequential time 1, the sequential fraction takes time S regardless of P and the rest takes at least $(1-S)/P$; letting $P \to \infty$ gives the bound $1/S$. □

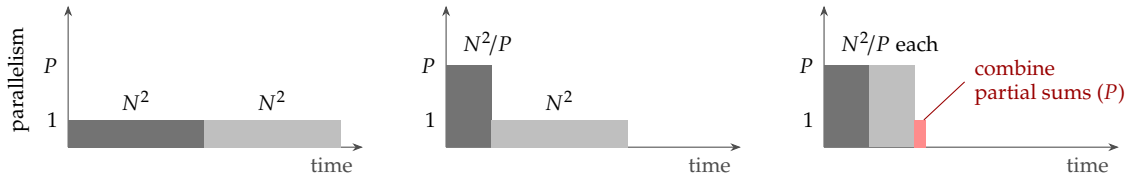
Example: *Brightening and averaging an image.* A two-step computation on an $N \times N$ image first multiplies the brightness of every pixel by two (independent per pixel) and then computes the average of all pixels. Sequentially both steps take about N^2 time, $2N^2$ in total. Running step 1 in parallel on P processors (time N^2/P) and step 2 serially (time N^2) gives

$$\text{speedup} \le \frac{2N^2}{N^2/P + N^2} \le 2.$$

Parallelizing step 2 as well, by computing P partial sums in parallel and combining them serially (time $N^2/P + P$, where P is the overhead of combining), gives

$$\text{speedup} \le \frac{2N^2}{2N^2/P + P},$$

which approaches P when $N \gg P$ (Figure 11).



(a) sequential (b) step 1 parallel (c) partial sums in parallel

Figure 11: Parallelism over time for the image example: (a) sequential, (b) only step 1 parallel, (c) both steps parallel, with a short serial combination of the P partial sums. Schematic.

A small serial region limits speedup on a large machine (Figure 12). The Summit supercomputer has 27,648 GPUs with 5,376 ALUs each, 148,635,648 ALUs that can perform about 148 million single-precision operations in parallel; if 0.1% of an application is serial, its maximum speedup is $1/(0.001 + 0.999/148{,}635{,}648) \approx 1000$.

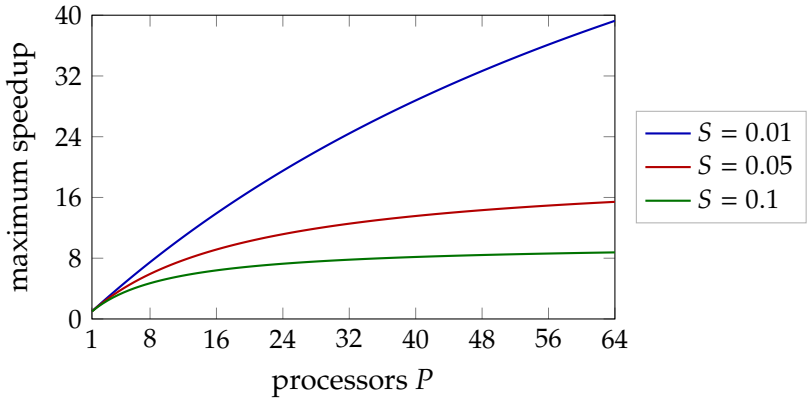


Figure 12: Amdahl's law: maximum speedup on P processors for serial fractions $S = 0.01$, 0.05, and 0.1.

In most cases the programmer is responsible for decomposition. Automatic decomposition of sequential programs remains a hard research problem: the compiler must analyze the program and identify dependencies, which may be data dependent and unknown at compile time. Researchers have had modest success with simple loop nests, but the "magic parallelizing compiler" for complex, general-purpose code has not been achieved.

4.3 Assignment, orchestration, and mapping

Assignment gives tasks (things to do) to workers, which might be threads, program instances, vector lanes, and so on. Its goals are good workload balance and low communication cost. It can be static (before the program runs) or dynamic (as it executes). Although the programmer often decomposes the problem, many languages and runtimes take responsibility for assignment.

- In ISPC, the programmer can assign loop iterations to program instances statically, for example interleaved through `programIndex` and `programCount`, or expose independent iterations with `foreach` and let the system assign them (the abstraction leaves room for dynamic assignment, but the current ISPC implementation is static).
- With C++ threads, the programmer can assign iterations statically in blocks, for example the first half of the array to a spawned thread and the second half to the main thread.
- With ISPC tasks, `launch[100] my_ispc_task(input, output, N);` creates 100 tasks, and the ISPC runtime, invisibly to the programmer, assigns them dynamically to worker threads in a thread pool: after completing its current task, a worker inspects the task list and assigns itself the next uncompleted task.

Orchestration involves structuring communication, adding synchronization to preserve dependencies, organizing data structures in memory, and scheduling tasks. Its goals are to reduce the costs of communication and synchronization, preserve locality of data reference, and reduce overhead.

Machine details affect these decisions: if synchronization is expensive, the programmer might use it sparingly.

Mapping assigns workers to hardware execution units: the operating system maps threads to hardware execution contexts on CPU cores, the compiler maps ISPC program instances to vector instruction lanes, and the hardware maps CUDA thread blocks to GPU cores. Mapping decisions include placing related (cooperating) threads on the same core to maximize locality and data sharing and minimize communication and synchronization costs, or placing unrelated threads on the same core (one bandwidth limited, the other compute limited) to use the machine more efficiently.

4.4 Case study: a grid solver

The example solves a partial differential equation on an $(N+2) \times (N+2)$ grid by Gauss-Seidel sweeps until convergence. Each sweep updates every interior cell from itself and its four neighbors,

$$A[i,j] \leftarrow 0.2\left(A[i,j] + A[i,j-1] + A[i-1,j] + A[i,j+1] + A[i+1,j]\right).$$

The sequential algorithm, in C-like pseudocode:

```
const int n;
float* A;  // assume allocated for grid of N+2 x N+2 elements

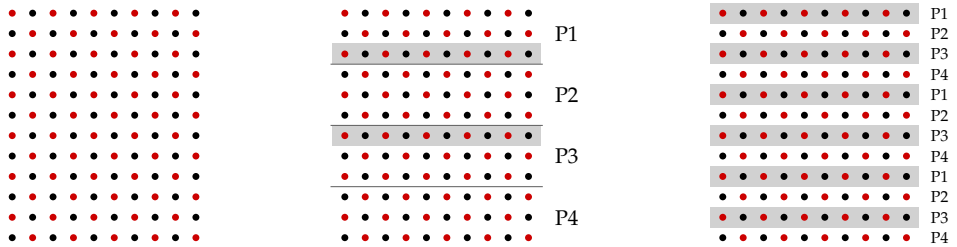
void solve(float* A) {
  float diff, prev;
  bool done = false;

  while (!done) {                    // outermost loop: iterations
    diff = 0.f;
    for (int i=1; i<n; i++) {        // iterate over non-border points of grid
      for (int j=1; j<n; j++) {
        prev = A[i,j];
        A[i,j] = 0.2f * (A[i,j] + A[i,j-1] + A[i-1,j] +
                         A[i,j+1] + A[i+1,j]);
        diff += fabs(A[i,j] - prev); // compute amount of change
      }
    }
    if (diff/(n*n) < TOLERANCE)      // quit if converged
      done = true;
  }
}
```

Step 1: identify dependencies. Within one iteration of the `while` loop, each element depends on the element to its left, and each row depends on the previous row. Independent work exists along the anti-diagonals: one could partition the cells of a diagonal into tasks, update them in parallel, and then move to the next diagonal. But this parallelism is hard to exploit: there is little parallelism at the beginning and end of the computation, and synchronization is frequent (after each diagonal).

A better approach changes the algorithm to one more amenable to parallelism by changing the order in which cells are updated. The new algorithm iterates to approximately the same solution, converging differently; the floating-point values differ but still converge to within the error threshold. Realizing this is permissible needs domain knowledge of the Gauss-Seidel method, but it is a common technique in parallel programming.

Definition: In RED-BLACK ORDERING, the grid is colored like a checkerboard. All red cells are updated in parallel; when they are done, all black cells are updated in parallel (they depend on the red cells); repeat until convergence.



(a) red-black coloring (b) blocked (c) interleaved

Figure 13: (a) Red-black coloring. (b) Blocked and (c) interleaved assignment of rows to four processors; shaded rows hold the data that P2 must receive each iteration.

Each iteration then performs the red update in parallel, waits until all processors are done, communicates updated red cells to other processors, performs the black update in parallel, waits, communicates updated black cells, and repeats. Rows can be assigned to processors in blocks or interleaved (Figure 13). Which is better depends on the system the program runs on; blocked assignment requires less data to be communicated between processors, since only the rows at block boundaries are needed by a neighbor.

4.5 Data-parallel expression

In the data-parallel model (showing only the red-cell update):

```
const int n;
float* A = allocate(n+2, n+2);  // allocate grid

void solve(float* A) {
  bool done = false;
  float diff = 0.f;
  while (!done) {
    for_all (red cells (i,j)) {
      float prev = A[i,j];
      A[i,j] = 0.2f * (A[i-1,j] + A[i,j-1] + A[i,j] +
                       A[i+1,j] + A[i,j+1]);
      reduceAdd(diff, abs(A[i,j] - prev));
    }
    if (diff/(n*n) < TOLERANCE)
      done = true;
  }
}
```

Decomposition is by individual grid elements, which constitute independent work. Assignment is left to the system. Orchestration is also handled by the system: the built-in communication primitive `reduceAdd` combines the changes, and the end of the `for_all` block is an implicit wait for all workers before returning to sequential control.

4.6 Shared address space expression

Definition: In the SHARED ADDRESS SPACE model, threads communicate by reading and writing shared variables, so communication is implicit in memory loads and stores, and by manipulating synchronization primitives. It is a natural extension of sequential programming (and has been the implicit model so far), like a bulletin board that everyone can read and write. For example, with `x = 0` initially, thread 1 may store `x = 1` while thread 2 spins in `while (x == 0) {}` and then prints `x`.

Shared variables need coordinated access. If two threads each execute `x++` on a shared `x`, each thread loads `x` into a register `r1`, adds `r2` = 1, and stores `r1` back. Starting from $x = 0$, the interleaving in which T1 and T2 both load 0, both compute 1, and both store 1 leaves $x = 1$ instead of 2. The three instructions must execute ATOMICALLY. Mechanisms for preserving atomicity include locking and unlocking a mutex around a critical section (`mylock.lock(); ...; mylock.unlock();`), first-class language support for atomic code blocks (`atomic { ...}`), and intrinsics for hardware-supported atomic read-modify-write operations (`atomicAdd(x, 10)`).

In the SPMD shared address space version, every thread runs `solve()`, and the programmer is

responsible for synchronization with LOCKS, which provide mutual exclusion (only one thread in the critical region at a time), and BARRIERS, which wait for threads to reach a point:

```
int     n;               // grid size
bool    done = false;
float   diff = 0.0;
LOCK    myLock;
BARRIER myBarrier;

// allocate grid
float* A = allocate(n+2, n+2);

void solve(float* A) {
  float myDiff;
  int threadId = getThreadId();
  int myMin = 1 + (threadId * n / NUM_PROCESSORS);
  int myMax = myMin + (n / NUM_PROCESSORS);

  while (!done) {
    float myDiff = 0.f;
    diff = 0.f;
    barrier(myBarrier, NUM_PROCESSORS);
    for (j=myMin to myMax) {
      for (i = red cells in this row) {
        float prev = A[i,j];
        A[i,j] = 0.2f * (A[i-1,j] + A[i,j-1] + A[i,j] + A[i+1,j] + A[i,j+1]);
        myDiff += abs(A[i,j] - prev);
      }
    }
    lock(myLock);
    diff += myDiff;
    unlock(myLock);
    barrier(myBarrier, NUM_PROCESSORS);
    if (diff/(n*n) < TOLERANCE)  // check convergence, all threads get same answer
      done = true;
    barrier(myBarrier, NUM_PROCESSORS);
  }
}
```

The value of `threadId` differs per SPMD instance and determines the block of rows each thread updates. The lock provides mutual exclusion on the shared `diff`. Accumulating into the private partial sum `myDiff` and completing the global reduction at the end of the iteration means each thread locks once per iteration rather than once per (i, j) update, which would serialize the threads on the lock.

Definition: A BARRIER, `barrier(num_threads)`, divides computation into phases: all computation

by all threads before the barrier completes before any computation after the barrier begins. Barriers are a conservative way to express dependencies, since all computation after the barrier is assumed to depend on all computation before it.

The three barriers per iteration each protect a dependency on the shared `diff`: the first ensures every thread has reset `diff` to 0 before any thread adds its contribution; the second ensures all contributions are added before any thread tests convergence; the third ensures every thread has read `diff` for the test before any thread resets it in the next iteration.

Example: *One barrier.* Using a different `diff` variable in successive iterations removes the dependencies, trading memory footprint for fewer dependencies (a common technique):

```
1  int      n;               // grid size
2  bool     done = false;
3  LOCK     myLock;
4  BARRIER myBarrier;
5  float diff[3];  // global diff, but now 3 copies

6  float *A = allocate(n+2, n+2);

7  void solve(float* A) {
8    float myDiff;   // thread local variable
9    int index = 0;  // thread local variable

10   diff[0] = 0.0f;
11   barrier(myBarrier, NUM_PROCESSORS);  // one-time only: just for init

12   while (!done) {
13     myDiff = 0.0f;
14     //
15     // perform computation (accumulate locally into myDiff)
16     //
17     lock(myLock);
18     diff[index] += myDiff;  // atomically update global diff
19     unlock(myLock);
20     diff[(index+1) % 3] = 0.0f;
21     barrier(myBarrier, NUM_PROCESSORS);
22     if (diff[index]/(n*n) < TOLERANCE)
23       break;
24     index = (index + 1) % 3;
25   }
26 }
```

Iteration k accumulates into `diff[k%3]` and clears `diff[(k+1)%3]` for the next iteration, so a slow thread still reading `diff[k%3]` after the barrier is never disturbed by faster threads that have

moved on.

	Data parallel	Shared address space
Synchronization	single logical thread of control; the system may parallelize `forall` iterations, with an implicit barrier at the end of the loop body	mutual exclusion for shared variables (locks); barriers express dependencies between phases
Communication	implicit in loads and stores, plus built-in primitives for complex patterns such as reduce	implicit in loads and stores to shared variables

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 4 (Fall 2023).

Lecture 5: Performance Optimization I: Work Distribution and Scheduling

Optimizing a parallel program is an iterative process of refining the choices of decomposition, assignment, and orchestration. Its key goals are at odds with each other: balance the workload onto the available execution resources, reduce communication (to avoid stalls), and reduce the extra work (overhead) performed to increase parallelism, manage assignment, or reduce communication. A first rule: always implement the simplest solution first, then measure performance to see whether you need to do better.

5.1 Balancing the workload

Ideally all processors compute all the time and finish their portions of the work at the same time. A small imbalance can significantly bound the maximum speedup: if one of four processors does twice as much work as each of the others, it takes twice as long, so 50% of the parallel runtime is serial. The serialized part, the extra unit of work done alone by the slow processor, is 1/5 of the total 5 units, so $S = 0.2$ in Amdahl's law.

Definition: In static assignment, the assignment of work to threads does not depend on dynamic behavior. It need not be fixed at compile time: it is static if it is determined once the amount of work and the number of workers are known, so it may depend on run-time parameters such as input size or thread count.

For example, the grid cells of a Mandelbrot image can be divided among threads in contiguous blocks of rows or in interleaved rows. Static assignment is simple and has essentially zero run-time overhead (a little indexing math). It applies when the cost and amount of work are predictable, so the programmer can work out a good assignment in advance: most simply, when all work has the same cost (twelve equal tasks on four processors, three each); when jobs have unequal but known costs; or when statistics of the execution time are predictable, such as the same cost on average, so that assigning an equal number of tasks to each processor balances the load on average. A row-interleaved assignment works well for the Mandelbrot image because costly regions (rows crossing the set) are spread evenly across threads.

Definition: In semi-static assignment, the cost of work is predictable for the near-term future, since the recent past predicts the near future. The application periodically profiles its execution and re-adjusts the assignment, which is static between re-adjustments. Examples: a particle simulation that redistributes particles to workers as they move (rarely, if motion is slow), and an adaptive mesh that changes slowly as an object moves or the flow over it changes.

Definition: In dynamic assignment, the program determines the assignment at run time to keep

the load well distributed, because the execution time of tasks, or the number of tasks, is unknown or unpredictable.

Example: *Testing primality.* Sequentially, a loop with independent iterations of unknown cost computes `is_prime[i] = test_primality(x[i])` for $N = 1024$ elements. In SPMD form, every thread repeatedly claims the next index from a shared counter:

```
1  LOCK counter_lock;
2  int counter = 0;    // shared variable

3  while (1) {
4    int i;
5    lock(counter_lock);
6    i = counter++;
7    unlock(counter_lock);
8    if (i >= N)
9      break;
10   is_prime[i] = test_primality(x[i]);
11 }
```

The three locked lines could instead be a single `atomic_incr(counter)`.

Conceptually this is a SHARED WORK QUEUE: a list of work to do (for now, independent pieces of work), from which worker threads pull data and to which they push new work as it is created. Breaking a problem into many ISPC tasks improves performance for the same reason: dynamic assignment of many tasks balances uneven costs.

5.2 Task granularity

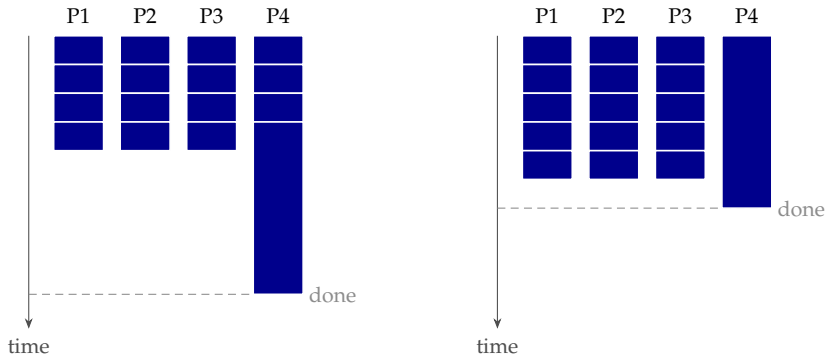
In the code above one task is one element. This fine-grained partitioning likely gives good workload balance (many small tasks) but may have high synchronization cost: time in the critical section is overhead that does not exist in the serial program, and it executes serially (recall Amdahl's law). Whether it is a problem depends on how long `test_primality` takes compared with the critical section. Coarser granularity, with one task covering `GRANULARITY` = 10 elements, enters the critical section ten times less often:

```
while (1) {
  int i;
  lock(counter_lock);
  i = counter;
  counter += GRANULARITY;
  unlock(counter_lock);
  if (i >= N)
    break;
  int end = min(i + GRANULARITY, N);
  for (int j=i; j<end; j++)
    is_prime[j] = test_primality(x[j]);
}
```

Choosing the task size is a trade-off. Having many more tasks than processors (many small tasks) enables good workload balance through dynamic assignment; having as few tasks as possible minimizes the overhead of managing the assignment. The ideal granularity depends on many factors: you must know your workload and your machine.

5.3 Smarter task scheduling

Suppose 16 tasks wait in a shared queue and the last one is much longer than the others. If the scheduler assigns tasks in left-to-right order, the long task runs last and leaves the other processors idle at the end (Figure 14a). One remedy is to divide the work into more, smaller tasks, hoping to make the "long pole" shorter relative to the total; this may increase synchronization overhead, and it may be impossible if the long task is fundamentally sequential. Another remedy is smarter scheduling: schedule long tasks first to reduce the "slop" at the end (Figure 14b). The thread that runs the long task performs fewer tasks but about the same amount of work as the others. This requires some knowledge of the workload (some predictability of cost).



(a) long task last (b) long task first

Figure 14: Dynamic scheduling of 15 short tasks and one long task on four processors: (a) the long task is dequeued last; (b) it is scheduled first. Schematic.

Distributed work queues. To avoid making all workers synchronize on a single queue, use a set of work queues, in general one per worker thread. Workers pull work from and push new work to their own queue, and when the local queue is empty they STEAL work from another queue.

Task dependencies. Work in task queues need not be independent. In a task management system, workers submit new tasks with optional explicit dependencies, and the scheduler assigns a task to a worker only when all its dependencies are satisfied:

```
// enqueue task foo (independent of all prior tasks)
foo_handle = enqueue_task(foo);
// enqueue task bar, cannot run until foo is complete
bar_handle = enqueue_task(bar, foo_handle);
```

The challenge is to keep all processors working all the time at low cost, minimizing both the computational overhead of scheduling and assignment logic and the synchronization costs. Static and dynamic assignment are not an either/or decision but a continuum: use up-front knowledge about the workload as much as possible to reduce both load imbalance and task-management costs (in the limit, if the system knows everything, use fully static assignment).

5.4 Fork-join parallelism

A common pattern is DATA PARALLELISM: performing the same sequence of operations on many data elements, for example through ISPC `foreach` or bulk task launch, a higher-order `map(foo, A, B)`, an OpenMP loop marked `#pragma omp parallel for`, or a bulk CUDA launch `foo<<<numBlocks, threadsPerBlock>>>(A, B)`. Another is explicit management of parallelism, creating one thread per execution unit (or per unit of desired concurrency) with C++ threads and joining them.

Divide-and-conquer algorithms have a different structure. In quicksort, after partitioning, the two recursive calls are independent work, giving a tree of dependencies.

Definition: The FORK-JOIN pattern expresses the independent work inherent in divide-and-conquer algorithms. In Cilk Plus (a C++ extension originally developed at MIT and adopted as an open standard in GCC and Intel ICC):

- `cilk_spawn foo(args);` "forks" a new logical thread of control: it invokes `foo`, but unlike a standard function call the caller may continue executing asynchronously with `foo`;
- `cilk_sync;` "joins": it returns when all calls spawned by the current function have completed.

There is an implicit `cilk_sync` at the end of every function that contains a `cilk_spawn`, so when a Cilk function returns, all work associated with it is complete.

In an ordinary call, control moves to the called function and returns to the caller when it fin-

ishes. With `cilk_spawn foo(); bar(); cilk_sync;`, `foo` and `bar` may run in parallel. Writing `cilk_spawn foo(); cilk_spawn bar(); cilk_sync;` exposes the same amount of independent work at potentially higher run-time overhead (two spawns instead of one), and `cilk_spawn` of `foo`, `bar`, and `fizz` followed by a call to `buzz` lets all four run in parallel.

The `cilk_spawn` abstraction does not specify how or when spawned calls are scheduled, only that they *may* run concurrently with the caller and with the caller's other spawns. Hence implementing `cilk_spawn foo()` as an ordinary call to `foo()` is correct. `cilk_sync` does constrain scheduling: all spawned calls must complete before it returns.

Example: *Parallel quicksort.*

```
1 void quick_sort(int* begin, int* end) {
2   if (begin >= end - PARALLEL_CUTOFF)
3     std::sort(begin, end);
4   else {
5     int* middle = partition(begin, end);
6     cilk_spawn quick_sort(begin, middle);
7     quick_sort(middle+1, end);
8   }
9 }
```

Small subproblems are sorted sequentially, since the overhead of a spawn outweighs the benefit of potential parallelism.

The main idea of fork-join programming is to expose independent work (potential parallelism) to the system with `cilk_spawn`. Programs should spawn at least enough work to fill the machine, and more independent work than execution capability so the load can be balanced. PARALLEL SLACK is the ratio of independent work to the machine's parallel execution capability; about 8 is a good ratio in practice. Too much slack makes the work too fine-grained and incurs management overhead.

5.5 Implementing fork-join: work stealing

A very simple scheduler would launch a pthread for each `cilk_spawn` (`pthread_create`) and translate `cilk_sync` into `pthread_join` calls. Its problems: a heavyweight spawn operation, many more concurrently running threads than cores, context-switching overhead, and a larger working set than necessary, with less cache locality.

Instead, the Cilk Plus runtime maintains a POOL OF WORKER THREADS, conceptually all created at application launch, with exactly as many workers as execution contexts in the machine (for example eight on a quad-core laptop with Hyper-Threading). Each worker loops: while work exists, get new work and run it. (Runtimes tend to create the workers lazily at the first spawn; ISPC does the same for its task workers.)

At `cilk_spawn foo(); bar();`, `foo` is the SPAWNED CHILD and `bar` the CONTINUATION (the rest of the

calling function). A serial implementation runs the child first via a regular call, and the continuation is implicit in the thread's stack; but then an idle thread cannot run `bar` while `foo` executes. So each worker keeps a WORK QUEUE of "work to do": on reaching `cilk_spawn foo()`, a thread places the continuation in its queue and executes `foo`. An idle thread (with an empty queue) looks in a busy thread's queue, moves the work to its own queue, and resumes execution of it: it STEALS the work.

Child first or continuation first? Running the continuation first queues the child for later (CHILD STEALING); running the child first queues the continuation (CONTINUATION STEALING). Consider `for (i=0; i<N; i++) cilk_spawn foo(i); cilk_sync;`.

- With child stealing, the caller spawns work for all iterations before executing any of it: a breadth-first traversal of the call graph, using $O(N)$ space for spawned work. Without stealing, the execution order differs greatly from that of the program with the spawns removed.
- With continuation stealing, the caller creates only one item to steal, the continuation representing all remaining iterations. Without stealing, the thread repeatedly pops the continuation and enqueues a new one with i advanced by one, so the execution order equals that of the program without spawns: a depth-first traversal. If the continuation is stolen, the thief spawns and executes the next iteration. One can prove that the work-queue storage for a system with T threads is at most T times the stack storage of single-threaded execution.

Deques and choice of victim. Each worker's queue is a DEQUE (double-ended queue). The local thread pushes and pops at the tail (bottom); remote threads steal from the head (top). In parallel quicksort of 200 elements, thread 0 working on elements 0–25 holds the continuations for 101–200, 51–100, and 26–50, from head to tail. Thieves take the largest pieces, 101–200 and 51–100, and recursion then fills every deque (Figure 15). Idle threads choose a victim at random and steal from the top of its deque. This steals the largest amount of work (reducing the number of steals), gives maximum locality in the work each thread performs when combined with child-first execution, and keeps the thief and the owner from contending for the same deque elements (efficient lock-free deque implementations exist).

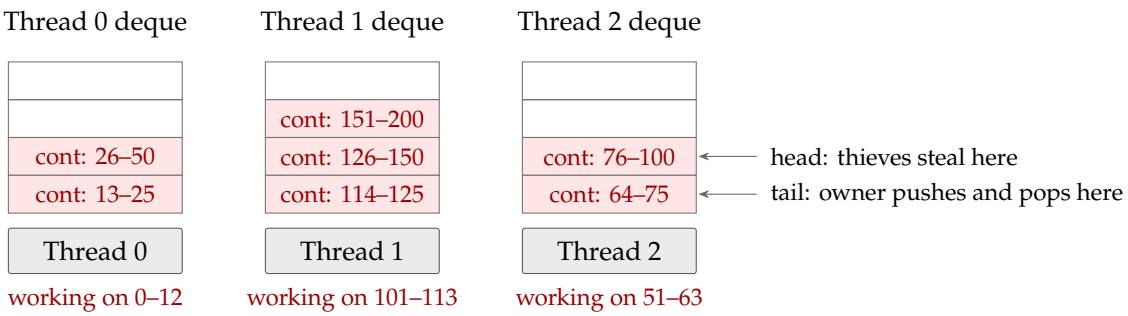


Figure 15: Work-stealing deques during parallel quicksort of 200 elements, after threads 1 and 2 stole the continuations 101–200 and 51–100 from thread 0.

A child-first work-stealing scheduler anticipates divide-and-conquer parallelism. The loop that spawns `foo(i)` for each i generates work serially, one continuation at a time. A recursive version generates work in parallel and so fills the machine more quickly:

```
void recursive_for(int start, int end) {
  while (start <= end - GRANULARITY) {
    int mid = start + (end - start) / 2;
    cilk_spawn recursive_for(start, mid);
    start = mid;
  }
  for (int i=start; i<end; i++)
    foo(i);
}

recursive_for(0, N);
```

Here a thief takes a continuation such as $(N/2, N)$ and itself splits it into $(N/2, 3N/4)$ and further halves.

5.6 Implementing sync

Consider a block A that spawns `foo(0)`, ..., `foo(9)` and then executes `cilk_sync; bar();`. If no work has been stolen, the sync is a no-op: the thread that ran the loop finds nothing outstanding. If work is stolen, the runtime creates a DESCRIPTOR for block A that tracks the number of outstanding spawns and how many have completed:

1. Thread 1 steals the continuation from thread 0, which is running `foo(0)`; the descriptor records 1 spawn, 0 done (the spawn is `foo(0)`, since thread 1 now owns the continuation).
2. Thread 1 runs `foo(1)` and the count becomes 2 spawns; thread 2 steals and runs `foo(2)`, giving 3 spawns.
3. When thread 0 completes `foo(0)` the descriptor shows 1 done, and thread 0 steals more work (`foo(3)`).
4. Near the end the descriptor reads 10 spawns, 9 done; when the last spawn completes (10 done), the thread that finished it resumes the continuation after the sync and executes `bar()`, and the descriptor is freed.

Definition: Cilk uses GREEDY JOIN SCHEDULING: threads always attempt to steal if they have nothing to do, go idle only if there is no work to steal in the system, and do not wait at a join, so the thread that initiated a spawn may not be the one that executes the code after `cilk_sync`.

The bookkeeping of steals and sync points occurs only when steals occur. If large pieces of work are stolen, steals are infrequent, and most of the time threads push and pop local work at the tail

of their own deque. The Cilk Plus runtime thus implements the spawn/sync abstraction with a locality-aware work-stealing scheduler that always runs the spawned child (continuation stealing) and is greedy at joins. Many other systems, such as OpenMP, also provide fork-join primitives.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 5 (Fall 2023).

Lecture 6: Performance Optimization II: Locality, Communication, and Contention

This unit covers techniques for reducing the cost of communication, both between processors and between processors and memory, and general tips for understanding performance.

6.1 Shared address space hardware

So far all processors have been assumed to share a memory system that provides the abstraction of a single address space, but its implementation can be complex. A load may involve several caches and DRAM; an eight-core CPU might have private 32 KB L1 and 256 KB L2 caches per core, a shared 20 MB L3, and 32 GB of DRAM. In a shared address space architecture any processor can directly reference any memory location. Cores are linked by an interconnect:

- Intel's ring interconnect (introduced with Sandy Bridge; a quad-core Core i7 such as Kaby Lake uses one) has four rings for different message types (request, snoop, acknowledgment, and 32-byte data) and six nodes: four 2 MB L3 slices with their cores, the system agent, and graphics. Each L3 bank connects to the ring twice, and the theoretical peak bandwidth from cores to L3 at 3.4 GHz is about 435 GB/s when each core accesses its local slice.
- The Sun Niagara 2 (UltraSPARC T2) uses a crossbar connecting all eight cores directly to all L2 banks and memory; the crossbar occupies about the area of one core.

Definition: In a non-uniform memory access (NUMA) system, the latency of accessing a memory location, and possibly the bandwidth, differs across processing cores. A multi-socket machine, where each socket has its own memory controller and memory, is the typical example; NUMA behavior also appears within one socket, since different cache slices are at different distances from each core.

The shared address space model communicates by reading and writing shared variables and manipulating synchronization primitives (locks, atomic operations), a logical extension of uniprocessor programming. It needs hardware support so that any processor can load and store any address, which is costly to scale to many processors (one reason high-core-count processors are expensive), and NUMA implementations require reasoning about locality for performance.

6.2 Message passing

Definition: In the message passing model, threads operate within their own private address spaces and communicate by sending and receiving messages, which is the only way to exchange data. A

`send` specifies the recipient, the buffer to transmit, and an optional message identifier ("tag"); a `receive` specifies the sender, the buffer to store the data, and an optional identifier. For example, `send(X, 2, my_msg_id)` sends the contents of thread 1's local variable `X` to thread 2 with tag `my_msg_id`, and thread 2's `recv(Y, 1, my_msg_id)` stores it in its local variable `Y`.

The hardware need not implement a single shared address space; it only needs a mechanism for communicating messages between nodes. Commodity systems can therefore be connected into a large parallel machine, and message passing is the programming model of clusters and supercomputers (for example a cluster of workstations on an InfiniBand network).

The grid solver with messages. Consider the red-black solver of Lecture 4 on a cluster where each computer has its own processor, cache, and memory. The grid is partitioned into four per-thread private arrays. Data replication is now required: after the red cells are processed, threads 1 and 3 each send a row to thread 2, which needs up-to-date red values to update its black cells. Ghost cells are grid cells replicated from a remote address space; their information is "owned" by other threads. Each thread allocates `rows_per_thread+2` rows and receives its ghost rows into the first and last. The solver has the same structure as the shared address space version, but communication is explicit:

```
int N;
int tid = get_thread_id();
int rows_per_thread = N / get_num_threads();

float* localA = allocate(rows_per_thread+2, N+2);

// assume localA is initialized with starting values
// assume MSG_ID_ROW, MSG_ID_DONE, MSG_ID_DIFF are constants used as msg ids

void solve() {
  bool done = false;
  while (!done) {
    float my_diff = 0.0f;

    // send and receive ghost rows to "neighbor threads"
    if (tid != 0)
      send(&localA[1,0], sizeof(float)*(N+2), tid-1, MSG_ID_ROW);
    if (tid != get_num_threads()-1)
      send(&localA[rows_per_thread,0], sizeof(float)*(N+2), tid+1, MSG_ID_ROW);
    if (tid != 0)
      recv(&localA[0,0], sizeof(float)*(N+2), tid-1, MSG_ID_ROW);
    if (tid != get_num_threads()-1)
      recv(&localA[rows_per_thread+1,0], sizeof(float)*(N+2), tid+1, MSG_ID_ROW);

    // perform computation (just like the shared address space version)
```

```
    for (int i=1; i<rows_per_thread+1; i++) {
      for (int j=1; j<N+1; j++) {
        float prev = localA[i,j];
        localA[i,j] = 0.2 * (localA[i-1,j] + localA[i,j] + localA[i+1,j] +
                             localA[i,j-1] + localA[i,j+1]);
        my_diff += fabs(localA[i,j] - prev);
      }
    }

    // all threads send local my_diff to thread 0, which evaluates the
    // termination predicate and sends the result back to all other threads
    if (tid != 0) {
      send(&my_diff, sizeof(float), 0, MSG_ID_DIFF);
      recv(&done, sizeof(bool), 0, MSG_ID_DONE);
    } else {
      float remote_diff;
      for (int i=1; i<get_num_threads(); i++) {
        recv(&remote_diff, sizeof(float), i, MSG_ID_DIFF);
        my_diff += remote_diff;
      }
      if (my_diff/(N*N) < TOLERANCE)
        done = true;
      for (int i=1; i<get_num_threads(); i++)
        send(&done, sizeof(bool), i, MSG_ID_DONE);
    }
  }
}
```

Array indexing is relative to the local address space. Communication is performed by messages, as bulk transfers of entire rows. Synchronization is also performed by sending and receiving messages; mutual exclusion, barriers, and flags must likewise be built from messages.

Definition: With SYNCHRONOUS (BLOCKING) send and receive, `send()` returns when the sender receives an acknowledgment that the message data resides in the receiver's address space, and `recv()` returns when the received data has been copied into the receiver's address space and an acknowledgment sent back. The sender copies data from its buffer into a network buffer and sends the message; the receiver copies the message into its buffer and sends the acknowledgment.

With blocking send and receive, the solver above deadlocks: every thread first sends to a neighbor, and no send can complete because no thread has reached its receive. Keeping synchronous operations, the fix is to order them so that each send meets a waiting receive: even-numbered threads send then receive, odd-numbered threads receive then send:

```
if (tid % 2 == 0) {
  sendDown(); recvDown();
  sendUp();   recvUp();
} else {
  recvUp();   sendUp();
  recvDown(); sendDown();
}
```

Definition: With NON-BLOCKING ASYNCHRONOUS operations, `send()` returns immediately with a handle, and the buffer passed to it must not be modified while message processing proceeds concurrently with the thread, which can do other work meanwhile. `recv()` posts an intent to receive in the future and returns immediately. `checksend(h)` and `checkrecv(h)` report the actual status: once the send has completed it is safe to modify the send buffer, and once the message is received (the messaging library having copied it into the buffer) it is safe to read the receive buffer.

6.3 Communication and arithmetic intensity

Communication is not only messages between machines in a datacenter. It also occurs between cores on a chip, between a core and its cache, and between a core and memory. A parallel system can be viewed as an extended memory hierarchy (Figure 16): accesses not satisfied at one level communicate with the next, so managing locality to reduce communication is important at all levels. For example, a load that misses in L1 and L2 sends a request to memory, which transfers the cache line over the memory bus, and the total latency includes the time to send the line over the bus. Recall the bandwidth-bound example of Lecture 3, where the processor executed two instructions per cache line loaded and memory was busy 100% of the time.

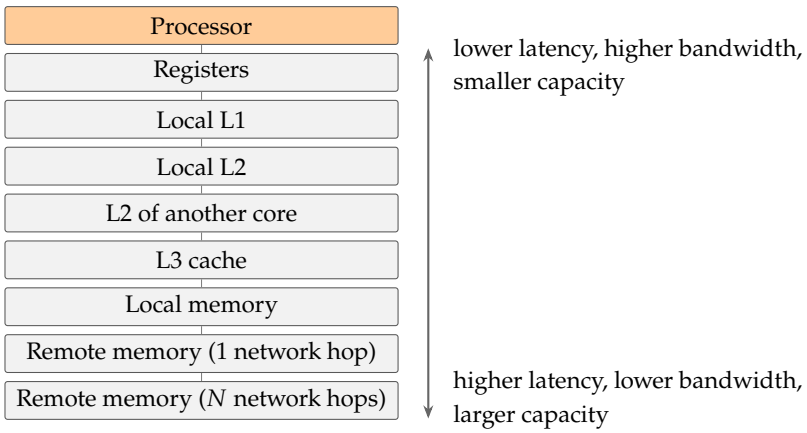


Figure 16: A parallel system as an extended memory hierarchy, viewed from one processor.

Definition: The ARITHMETIC INTENSITY of a program is

$$\frac{\text{amount of computation (e.g., instructions)}}{\text{amount of communication (e.g., bytes)}}.$$

Its reciprocal is the COMMUNICATION-TO-COMPUTATION RATIO. If the numerator is the execution time of the computation, the ratio gives the average bandwidth requirement of the code.

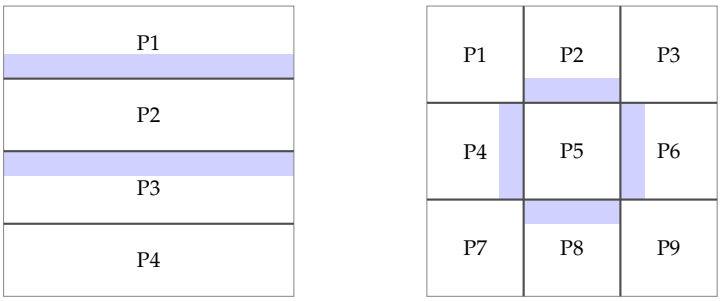
High arithmetic intensity (higher is better) is required to use modern parallel processors efficiently, because their ratio of compute capability to available bandwidth is high (recall the element-wise vector product).

6.4 Inherent and artifactual communication

Definition: INHERENT COMMUNICATION is information that must fundamentally be moved between processors to carry out the algorithm given the specified assignment (assuming unlimited-capacity caches, minimum-granularity transfers, and so on); sending ghost rows in the message-passing solver is inherent. ARTIFACTUAL COMMUNICATION is all other communication, arising from practical details of the system implementation.

Good assignment decisions reduce inherent communication, increasing arithmetic intensity. For an $N \times N$ grid on P processors (Figure 17):

- *1D blocked*: each processor computes about N^2/P elements and communicates about $2N$, so the ratio is proportional to N/P;
- *1D interleaved*: each row computed requires communicating its two neighbors, so the ratio is $1/2$;
- *2D blocked*: each processor computes N^2/P elements and communicates elements proportional to $N/\sqrt{P}$, so the arithmetic intensity is proportional to $N/\sqrt{P}$. This scales asymptotically better than 1D blocking: communication costs grow sublinearly with P, because the assignment captures the 2D locality of the algorithm.



(a) 1D blocked (b) 2D blocked

Figure 17: (a) 1D blocked assignment to four processors and (b) 2D blocked assignment to nine. Shaded cells are the neighboring data P2 (a) and P5 (b) must receive each iteration.

Artifactual communication from caches. Consider the solver's row-major traversal of a row-major grid with 4-element cache lines and a cache of 24 elements (6 lines). By the start of the second output row, elements of the first and second rows that were already used have been evicted, so the program loads three cache lines for every four output elements. Other sources of artifactual communication:

- a minimum granularity of transfer: loading one 4-byte float transfers a whole 64-byte cache line (16× more communication than needed);
- unnecessary communication by system operation: storing 16 consecutive 4-byte floats loads the entire 64-byte line from memory, overwrites it entirely, and later stores it back (2× overhead, since the load was unnecessary);
- finite replication capacity: the same data is communicated to the processor repeatedly because the cache is too small to retain it between accesses (capacity misses).

6.5 Techniques for reducing communication

Blocking. Changing the traversal order to a BLOCKED iteration order, which processes the grid in narrow column blocks, reduces capacity misses: the program now loads two cache lines for every six output elements, improving temporal locality.

Loop fusion. Composing modular array operations, in the style of NumPy, gives low arithmetic intensity:

```
void add(int n, float* A, float* B, float* C) {
  for (int i=0; i<n; i++)
    C[i] = A[i] + B[i];
}

void mul(int n, float* A, float* B, float* C) {
  for (int i=0; i<n; i++)
    C[i] = A[i] * B[i];
}

float* A, *B, *C, *D, *E, *tmp1, *tmp2;

// assume arrays are allocated here

// compute E = D + ((A + B) * C)
add(n, A, B, tmp1);
mul(n, tmp1, C, tmp2);
add(n, tmp2, D, E);
```

Each function performs two loads and one store per math operation (arithmetic intensity 1/3), and so does the whole computation. The fused version

```
void fused(int n, float* A, float* B, float* C, float* D, float* E) {
  for (int i=0; i<n; i++)
    E[i] = D[i] + (A[i] + B[i]) * C[i];
}

// compute E = D + (A + B) * C
fused(n, A, B, C, D, E);
```

performs four loads and one store per three math operations (arithmetic intensity 3/5), and performs much better because the intermediate arrays never travel to and from memory.

Sharing. Co-locating tasks that operate on the same data, by scheduling threads that work on the same data structure at the same time on the same processor, exploits sharing and reduces inherent communication.

6.6 Contention

Example: *Office hours.* Answering a question takes 5 minutes, and walking to the office takes 5 minutes. With drop-in office hours from 3:00 to 3:20, five students arriving within a few minutes of each other wait in line: the first spends 10 minutes in total, while the fourth spends 23, and the fifth is not served by 3:20. With appointments at 3:00 and 4:30, each student spends only 10 minutes.

Contention for a shared resource lengthens the overall operation time.

Definition: A resource (memory, a communication link, a server, a TA) performs operations at a given throughput. CONTENTION occurs when many requests are made to a resource within a small window of time, making it a HOT SPOT.

For example, when many processors update a shared variable, flat communication (all send to one) has potential for high contention but low latency without contention, while tree-structured communication reduces contention at the cost of higher latency without contention. Distributed work queues reduce contention for a single shared queue: when all processors have work there is no contention, and synchronization happens only when stealing, when the thread would otherwise sit idle.

To reduce communication costs:

- reduce the overhead to sender and receiver: send fewer, larger messages, coalescing many small messages into large ones;
- reduce latency: restructure code to exploit locality (application writer) or improve the communication architecture (hardware);
- reduce contention: replicate contended resources (local copies, fine-grained locks) and stagger accesses to them;
- increase overlap of communication and computation: asynchronous communication (application writer), or pipelining, multithreading, prefetching, and out-of-order execution (hardware), all of which require more concurrency in the application than there are execution units.

6.7 Understanding parallel performance

Always try the simplest parallel solution first, then measure performance. Determine whether performance is limited by computation, memory bandwidth (or latency), or synchronization, and establish "high watermarks": the best achievable in practice, and how close the implementation is to it.

Definition: The ROOFLINE MODEL plots, for programs of different arithmetic intensity (horizontal axis), the maximum obtainable instruction throughput (vertical axis). In the diagonal region execution is limited by memory bandwidth; in the horizontal region it is compute limited. Benchmarks with different levels of optimization (for example with and without SIMD, or with and without ILP) give lower ceilings that delimit optimization regions (Figure 18).

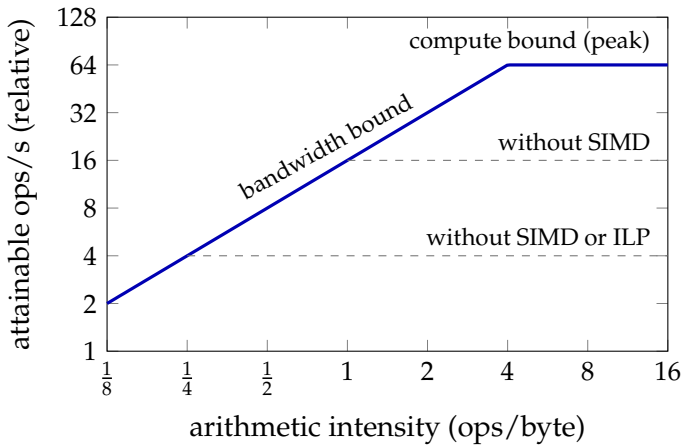


Figure 18: The roofline model: attainable throughput is the lesser of peak compute and peak bandwidth times arithmetic intensity; dashed lines are lower ceilings for less optimized code. Schematic.

Simple experiments establish high watermarks:

- add math (non-memory instructions): if execution time grows linearly with the operation count, the code is instruction-rate limited;
- remove almost all math but load the same data: if execution time barely decreases, suspect a memory bottleneck;
- change all array accesses to `A[0]`: the speedup bounds the benefit of improving the locality of data access;
- remove all atomic operations or locks: the speedup (provided the program does about the same work) bounds the benefit of reducing synchronization overhead.

Computation, memory access, and synchronization are almost never perfectly overlapped, so performance is rarely dictated by just one of them; still, the sensitivity of performance to these modifications indicates the dominant costs. Operating-system CPU-usage graphs only show the fraction of time threads are scheduled and are not very helpful. Modern processors have PERFORMANCE COUNTERS, registers counting instructions completed, clock ticks, L2/L3 hits and misses, bytes read from the memory controller, and so on, accessible for example through Intel's Performance Counter Monitor C++ API (`getIPC`, `getL3CacheHitRatio`, `getBytesReadFromMC` between two counter snapshots), VTune, PAPI, or oprofile.

6.8 Problem size and scaling

A design can be judged by absolute performance (wall-clock time, operations per second), by speedup (sequential time over time on P processors), or by efficiency (performance per unit resource, such as operations per second per chip area, per dollar, or per watt). Speedup should be measured

against the best sequential program, not the parallel algorithm run on one core; the latter is a common pitfall that makes results look better, since a parallelized algorithm (such as the red-black solver) may converge more slowly.

Fixed-problem-size analysis has pitfalls. On a 32-processor SGI Origin 2000, the solver on a 258×258 grid (about 310 cells per processor at 32 processors) stops benefiting from more processors and even slows down, because the problem is too small for the machine: with 2D blocking the arithmetic intensity is proportional to $N/\sqrt{P}$, so small N or large P gives a high communication-to-computation ratio. A $1\text{K} \times 1\text{K}$ grid (about 32K cells per processor) scales well. Conversely, a $12\text{K} \times 12\text{K}$ grid achieves SUPER-LINEAR SPEEDUP: with enough processors, each processor's chunk begins to fit in its cache. Similarly, a problem too large for one machine may thrash to disk, making speedup on a bigger machine with more memory look amazing.

In general, problem size and machine size interact in complex, application-dependent ways, affecting load balance, overhead, arithmetic intensity, and locality. Too small a problem lets parallel overheads dominate (possibly causing slowdowns) and does not reflect realistic use of a large machine; too large a problem may not fit a small machine (thrashing, working sets exceeding cache, or not running at all), producing super-linear speedups on the large one. It can be desirable to scale the problem size with the machine: a bigger machine is bought to compute more, not just to compute the same problem faster.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 6 (Fall 2023).

Lecture 7: GPU Architecture and CUDA Programming

Graphics processors, originally designed to accelerate 3D games, evolved into highly parallel compute engines for deep learning, computer vision, and scientific computing. A GPU is a multi-core chip with SIMD execution within each core (many execution units performing the same instruction) and multithreaded execution on each core, attached to high-bandwidth memory: about 150–300 GB/s and a few GB on earlier designs, about 1 TB/s and tens of GB on current high-end GPUs.

7.1 Graphics and the origins of GPU computing

Rendering takes a description of a scene (3D surface geometry such as a triangle mesh, surface materials, lights, and a camera) and computes how each triangle contributes to the appearance of each pixel. Surfaces are meshes of vertices (points in space) assembled into primitives (triangles, points, lines). The workload: given a triangle, determine where it lies on screen from the camera position; then, for every output pixel covered by the triangle, compute the color of the surface at that pixel. Materials and lights are diverse, so this color computation is a programmable SHADER program, for example in GLSL:

```
uniform sampler2D myTexture;  // read-only global variables
uniform float3 lightDir;
varying vec3 norm;            // inputs whose value changes per pixel
varying vec2 uv;

void myShader()
{
  vec3 kd = texture2D(myTexture, uv);
  kd *= clamp(dot(lightDir, norm), 0.0, 1.0);
  return vec4(kd, 1.0);       // per-pixel output: RGBA surface color
}
```

It runs once per fragment (per pixel covered by a triangle); what matters is that a shader is a pure function invoked on a stream of inputs. Many SIMD, multithreaded cores execute shaders efficiently.

Around 2001–2003 it was observed that GPUs are very fast processors for performing the same computation in parallel on large collections of data: data parallelism, getting faster every year as transistors become parallelism. Early GPU-based scientific computing was a hack: to run a function on every element of a 512×512 array, set the output image size to 512×512 and render two triangles exactly covering the screen, so the fragment shader is mapped over the 512×512 collection. This

"GPGPU" (general-purpose GPU) work included coupled map lattice simulation, sparse matrix solvers, and ray tracing. The Brook stream language (Stanford, 2004) abstracted the GPU as a data-parallel processor: a kernel such as `kernel void scale(float amount, float a<>, out float b<>) { b = amount * a; }` is mapped onto streams by `scale(scale_amount, input_stream, output_stream)`, and the compiler translated the program into graphics commands (such as drawing triangles) and shaders.

On a CPU, running a program means the OS loads the program, selects an execution context, sets its registers and program counter, and the processor executes instructions in that context. Before 2007, the only interface to GPU hardware was the graphics pipeline: the application provided shader binaries, set pipeline parameters (such as output image size), provided a vertex buffer, and issued `drawPrimitives` through stages of vertex generation and processing, primitive generation, fragment generation (rasterization), fragment processing, and pixel operations. The NVIDIA Tesla architecture (2007) added the first non-graphics COMPUTE MODE: the application allocates buffers in GPU memory and copies data to and from them, provides a single kernel binary, and tells the GPU to run the kernel in SPMD fashion, `launch(myKernel, N)`, a far simpler operation than `drawPrimitives`.

7.2 CUDA abstractions

CUDA, introduced with Tesla in 2007, is a "C-like" language for programs that run on GPUs through the compute-mode interface. It is relatively low level: its abstractions closely match the capabilities and performance characteristics of GPUs (low abstraction distance). A CUDA THREAD is a logical thread of control, like a pthread, but its implementation is very different.

Definition: A CUDA program launches a GRID of THREAD BLOCKS, each a group of CUDA threads; thread and block IDs can be up to three-dimensional, convenient for naturally N-dimensional problems. The kernel launch `kernel<<<numBlocks, threadsPerBlock>>>(args)` from the HOST (the CPU) is a bulk launch of many CUDA threads that runs the DEVICE kernel function (marked `__global__`) in SPMD fashion on the GPU; the call returns when all threads have terminated. Each thread computes its position from `threadIdx` (its position in its block), `blockIdx` (its block's position in the grid), and `blockDim`.

```cpp
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/threadsPerBlock.y);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will launch 72 CUDA threads:
// 6 thread blocks of 12 threads each
matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);

// kernel definition (runs on GPU)
__global__ void matrixAdd(float A[Ny][Nx],
                          float B[Ny][Nx],
                          float C[Ny][Nx])
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  int j = blockIdx.y * blockDim.y + threadIdx.y;

  C[j][i] = A[j][i] + B[j][i];
}
```

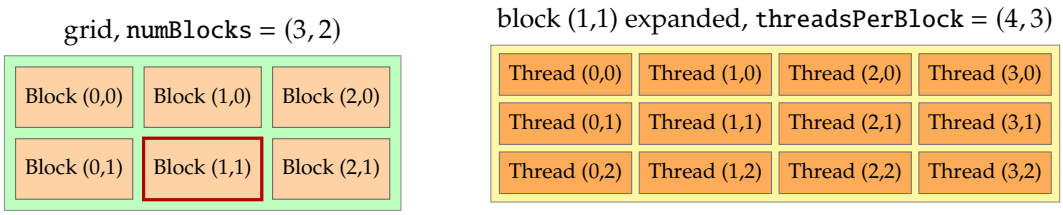


Figure 19: The `matrixAdd` launch: a 3×2 grid of thread blocks, each a 4×3 block of CUDA threads.

The programmer separates host and device code statically: host code runs serially on the CPU, and device code, kernels and `__device__` helper functions such as `__device__ float doubleValue(float x) { return 2 * x; }`, runs SPMD on the GPU. The number of CUDA threads is explicit in the program, not determined by the size of a data collection as with `map(kernel, collection)` in shader programming. If $N_x = 11$ and $N_y = 5$ are not multiples of the block size, the launch rounds up, `numBlocks((Nx+threadsPerBlock.x-1)/threadsPerBlock.x, (Ny+threadsPerBlock.y-1)/threadsPerBlock.y)`, still launching 72 threads, and the kernel guards against out-of-bounds accesses with `if (i < Nx && j < Ny)`.

Memory model. Host and device have distinct address spaces (Figure 20). Data moves between them with `cudaMemcpy`, much like message passing:

```
float* A = new float[N];  // allocate buffer in host mem

// populate host address space pointer A
for (int i=0; i<N; i++)
  A[i] = (float)i;

int bytes = sizeof(float) * N;
float* deviceA;               // allocate buffer in
cudaMalloc(&deviceA, bytes);  // device address space

// populate deviceA
cudaMemcpy(deviceA, A, bytes, cudaMemcpyHostToDevice);

// note: directly accessing deviceA[i] is an invalid
// operation here (cannot manipulate contents of deviceA
// directly from host, since deviceA is not a pointer
// into the host's address space)
```

Kernels see three types of device address space: per-thread private memory, per-block SHARED MEMORY readable and writable by all threads in the block, and device GLOBAL MEMORY readable and writable by all threads. These reflect different regions of locality in the program, which matters for efficient implementation (for instance, threads known to share variables can be scheduled together).

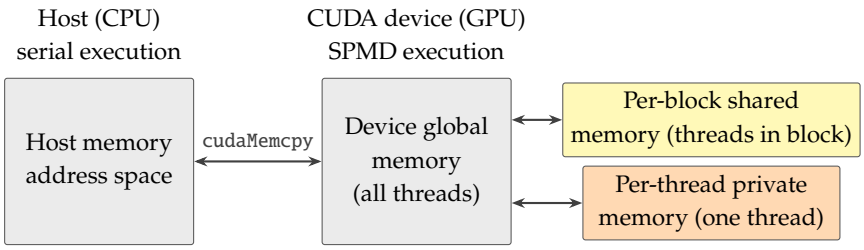


Figure 20: CUDA memory model: host and device address spaces, and the global, per-block shared, and per-thread private address spaces visible to kernels.

Example: *1D convolution.* Compute `output[i] = (input[i] + input[i+1] + input[i+2]) / 3.f` for $N = 1024 \cdot 1024$ outputs, with one thread per output element and 128 threads per block. In version 1 each thread reads its three inputs from global memory:

```
#define THREADS_PER_BLK 128

__global__ void convolve(int N, float* input, float* output) {
  int index = blockIdx.x * blockDim.x + threadIdx.x;  // thread local variable

  float result = 0.0f;  // thread-local variable
  for (int i=0; i<3; i++)
    result += input[index + i];

  output[index] = result / 3.f;
}

// host code
int N = 1024 * 1024;
cudaMalloc(&devInput, sizeof(float) * (N+2));  // allocate input array in device memory
cudaMalloc(&devOutput, sizeof(float) * N);     // allocate output array in device memory

// properly initialize contents of devInput here ...

convolve<<<N/THREADS_PER_BLK, THREADS_PER_BLK>>>(N, devInput, devOutput);
```

In version 2, all threads of a block cooperatively load the block's support region into shared memory, 130 loads instead of $3 \cdot 128$, and synchronize before computing:

```
__global__ void convolve(int N, float* input, float* output) {

  __shared__ float support[THREADS_PER_BLK+2];        // per-block allocation
  int index = blockIdx.x * blockDim.x + threadIdx.x;  // thread local variable

  support[threadIdx.x] = input[index];
  if (threadIdx.x < 2) {
    support[THREADS_PER_BLK + threadIdx.x] = input[index+THREADS_PER_BLK];
  }

  __syncthreads();

  float result = 0.0f;  // thread-local variable
  for (int i=0; i<3; i++)
    result += support[threadIdx.x + i];

  output[index] = result / 3.f;
}
```

CUDA's synchronization constructs are `__syncthreads()`, a barrier for all threads in a block; atomic operations such as `float atomicAdd(float* addr, float amount)` on both global and shared memory addresses; and host/device synchronization, an implicit barrier across all threads when a

kernel returns. Thus CUDA provides a two-level thread hierarchy launched in bulk, a distributed address space with memcpy between host and device and three device address spaces (per thread, per block, global), a barrier for threads in a block, and atomics.

7.3 Implementing CUDA

Creating a pthread allocates a stack and a control block so the OS can schedule it. Launching the convolution creates over a million CUDA threads in over 8K blocks; must the implementation create a million stacks and 8K copies of `support`? CUDA code contains no notion of the number of cores, and the same program should run unmodified on a GPU with 6 cores or 16; a kernel launch is in the spirit of a `forall` loop.

A compiled CUDA binary contains the program text and the resources each block requires: here 128 threads per block, B bytes of local data per thread, and 130 floats (520 bytes) of shared memory per block. The THREAD BLOCK SCHEDULER, special hardware in the GPU, maps blocks to cores dynamically, respecting the resource requirements. The major CUDA assumption is that blocks can execute in any order, with no dependencies between them. This is another instance of the worker-pool design pattern, like ISPC tasks running on one pthread per hyper-thread, or a web server whose thread count depends on the number of cores rather than outstanding requests.

Warps and the V100 SM. A V100 SM has four sub-cores, each with a warp selector, a fetch/decode unit, 16-wide SIMD fp32 and int units (one 32-wide operation every two clocks), 8-wide fp64 units, tensor cores, load/store units, and 64 KB of registers.

Definition: A WARP is a group of 32 consecutive CUDA threads of a thread block (threads 0–31, 32–63, and so on), so a block of 256 threads maps to 8 warps. Threads in a warp execute in SIMD fashion when they share the same instruction, which NVIDIA calls SIMT (single instruction, multiple CUDA thread); otherwise divergence reduces performance. Unlike an ISPC gang, the program is not compiled to SIMD instructions: the hardware dynamically checks whether the 32 threads share an instruction. Warps are an implementation detail, not part of the CUDA programming model.

The registers of a sub-core hold the scalar registers of each thread of its warps; each sub-core can schedule and interleave up to 16 warps. Each instruction of the warp's stream runs for all 32 threads, so on 16 ALUs it takes two clocks. Each clock, each sub-core selects one runnable warp from its 16 and runs that warp's next instruction, which may apply to all or some of its threads depending on divergence. A convolve block of 128 threads runs as 4 warps.

The full SM has 64 warp contexts, 256 KB of registers, and 128 KB of combined shared memory and L1 storage. The V100 has 80 SMs at 1.245 GHz, $80 \times 4 \times 16 = 5120$ fp32 mul-add ALUs giving 12.7 TFLOPs (a mul-add counts as two flops), up to $80 \times 64 = 5120$ interleaved warps (163,840 CUDA threads) per chip, a 6 MB L2 cache, and 16 GB of HBM at 900 GB/s.

Running a kernel. Consider the convolution with very large N (here 1000 blocks) on a fictitious two-core GPU whose cores each hold execution contexts for 384 threads (12 warps) and 1.5 KB of shared storage. The host sends the GPU a command (execute `convolve` with these arguments, 1000 blocks). The scheduler maps block 0 to core 0, reserving contexts for 128 threads and 520 bytes of shared storage; it continues mapping blocks to available contexts (interleaved across cores), so blocks 0 and 2 occupy core 0 and blocks 1 and 3 core 1 (Figure 21). A third block does not fit on a core, since 3×520 bytes exceeds 1.5 KB, even though contexts for 128 more threads remain. When block 0 completes, the scheduler maps block 4 to the freed contexts 0–127 of core 0; when block 2 completes, block 5 takes contexts 128–255, and so on.

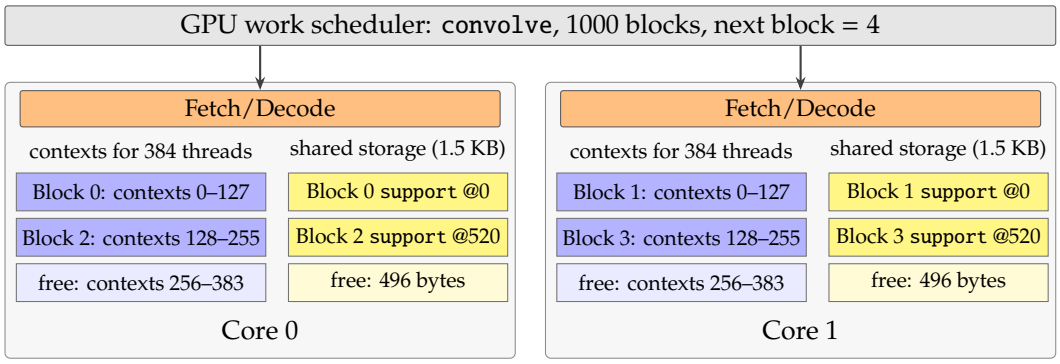


Figure 21: Thread-block scheduling of `convolve` on a fictitious two-core GPU after blocks 0–3 are mapped. Shared storage, not execution contexts, limits each core to two blocks.

Why allocate contexts for all threads of a block? Consider a block of 256 threads on a core with only 128 threads (4 warps) of hardware contexts. The implementation cannot run threads 0–127 to completion and then 128–255, because threads in a block may depend on each other, most simply through `__syncthreads()`: the first half would wait at the barrier forever. CUDA semantics say the threads in a block run concurrently, and a runnable thread will eventually run (no deadlock), so all contexts of a block are reserved when it starts.

- Thread blocks can be scheduled in any order; the system assumes no dependencies between blocks, which are logically concurrent, much like ISPC tasks.
- CUDA threads in a block run concurrently (live at the same time): when the block starts, all its threads exist with allocated register state. A block is itself an SPMD program, like an ISPC gang, whose threads are concurrent, cooperating workers.
- A warp has performance characteristics like an ISPC gang, but unlike a gang it is not part of the programming model (exceptions are intra-warp built-ins such as swizzle and vote).
- All warps of a block are scheduled on the same SM, enabling high-bandwidth, low-latency communication through shared memory; when a block completes, its shared memory and warp contexts become available for the next block.

Example: *Dependencies between blocks.* Blocks are not guaranteed to be independent; CUDA only reserves the right to schedule them in any order. Building a histogram in which every thread executes `atomicAdd(&counts[A[i]], 1)` on a global array is valid code: the atomics provide mutual exclusion only and do not constrain the order of blocks. By contrast, if block 0 executes `atomicAdd(&myFlag, 1)` after doing some work while block 1 spins in `while (atomicAdd(&myFlag, 0) == 0) {}`, then on a GPU with resources for only one block, running block 0 first works, but running block 1 first deadlocks, since block 0 never gets to run.

Persistent threads. One style of CUDA programming assumes knowledge of the GPU: it launches exactly as many blocks as fill the chip (for the V100, $80 \cdot (32 \cdot 64/128)$ blocks of 128 threads) and has each block repeatedly claim the next chunk of work from a global counter with `atomicInc`, so work assignment is implemented entirely by the application, circumventing the thread block scheduler. The programmer's mental model is that all CUDA threads run concurrently, which assumes the GPU will in fact run all the blocks at once.

7.4 CUDA semantics in summary

Partitioning a problem into thread blocks follows the data-parallel model: it is machine independent, since the system schedules blocks onto any number of cores. Threads within a block do run concurrently, because they cooperate: inside a block, CUDA is SPMD shared address space programming. Memory is a distributed address space (host and device memories), and within the device, thread-local, block-shared, and global variables are distinct address spaces between which loads and stores move data. The key implementation details are that threads of a block are scheduled on the same SM for fast communication through shared memory, and are grouped into warps for SIMT execution. CUDA programs use the GPU's programmable cores; the fixed-function hardware that accelerates the graphics pipeline is largely idle when running CUDA, and the tensor cores provide hundreds of teraflops for deep learning.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 7 (Fall 2023).

Lecture 8: Data-Parallel Thinking

Instead of thinking about parallel programs in terms of what workers do, this unit describes algorithms as operations on sequences of data: map, filter, fold (reduce), scan and segmented scan, sort, groupBy, join, and partition/flatten. High-performance parallel implementations of these operations exist, so programs written in terms of them can often run efficiently on a parallel machine, provided they avoid being bandwidth bound.

Applications must expose large amounts of parallelism to use many cores, many machines (such as a cloud cluster), SIMD and multithreaded cores, and above all GPUs: the V100 can run up to 163,840 CUDA threads concurrently, and programs without much parallelism and high arithmetic intensity will not run efficiently on it. The key is understanding dependencies: in `x = a + b; y = b * 7; z = (x-y) * (x+y);` the computations of `x` and `y` are independent, as are `x-y` and `x+y`, and a lack of dependencies means potential parallel execution.

8.1 Sequences and map, fold, scan

The data-parallel model organizes computation as operations on sequences of elements, for example performing the same function on all elements, as in NumPy's `C = A + B`.

Definition: A sequence is an ordered collection of elements (`Sequence<T>` in C++-like notation, `List[T]` in Scala, a Pandas DataFrame, `seq T` in Haskell). Unlike arrays, programs can access the elements of a sequence only through specific operations.

Definition: Map is a higher-order function (a function taking a function as argument) that applies a side-effect-free unary function $f :: a \to b$ to every element of the input sequence, producing an output sequence of the same length: map $:: (a \to b) \to \text{seq}\, a \to \text{seq}\, b$. In C++ it is `std::transform`; with `f(x) = x + 10`, $[3, 8, 4, 6, 3, 9, 2, 8] \mapsto [13, 18, 14, 16, 13, 19, 12, 18]$.

Since f has no side effects, applying it to the elements in any order gives the same output, so an implementation may reorder or parallelize freely: partition the sequence into P subsequences, map each in parallel, and concatenate the results.

Definition: Fold (fold left) applies a binary operation $f :: (b, a) \to b$ to each element and an accumulated value, seeded by an initial value of type b: fold $:: b \to ((b, a) \to b) \to \text{seq}\, a \to b$ (Scala's `foldLeft`). For example, `fold 10 +` over $[3, 8, 4, 6, 3, 9, 2, 8]$ accumulates $10, 13, 21, 25, 31, 34, 43, 45, 53$ and returns 53. A parallel fold additionally needs a combiner comb $:: (b, b) \to b$, with the initial value an identity for both f and comb: each partition is folded from the identity, and the partial results are combined in a tree. No combiner is needed if $f :: (b, b) \to b$ is an associative binary operator.

Definition: Let $A = [a_0, a_1, \ldots, a_{n-1}]$ and $\oplus$ an associative binary operator with identity I. The INCLUSIVE SCAN is $[a_0,\ a_0 \oplus a_1,\ a_0 \oplus a_1 \oplus a_2,\ \ldots]$ and the EXCLUSIVE SCAN is $[I,\ a_0,\ a_0 \oplus a_1,\ \ldots]$, where entry i combines all elements before, but excluding, a_i. With $\oplus = +$ the inclusive scan is the PREFIX SUM; for example the inclusive $+$-scan of $[3, 8, 4, 6, 3, 9, 2, 8]$ is $[3, 11, 15, 21, 24, 33, 35, 43]$. Sequentially, `out[0] = in[0]` and `out[i] = op(out[i-1], in[i])`.

8.2 Parallel scan

Write $a_{i:j} = a_i \oplus \cdots \oplus a_j$. The simplest data-parallel inclusive scan runs $\lg N$ steps; in step d every element $i \geq 2^d$ adds the element 2^d positions to its left (Figure 22). Subtracting the original vector gives the exclusive scan. Its SPAN, the longest chain of sequential steps, is $O(\lg N)$, but its WORK, the total number of operations, is $O(N \lg N)$: inefficient compared with the $O(N)$ sequential algorithm.

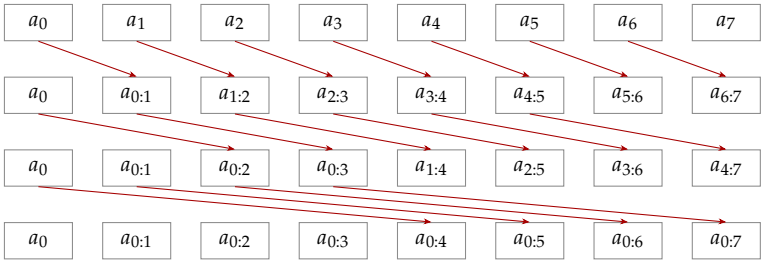


Figure 22: Inclusive scan of eight elements by repeated doubling: $\lg N$ steps, $O(N \lg N)$ work.

The work-efficient exclusive scan uses a balanced-tree up-sweep followed by a down-sweep (Figure 23):

Algorithm 8.1: Work-efficient exclusive scan (with $\oplus = +$)

Input: array $a[0..n-1]$, n a power of two
// up-sweep
for $d = 0$ **to** $\log_2 n - 1$ **do**
  for each $k = 0$ to $n-1$ by 2^{d+1} (in parallel) **do**
    $a[k + 2^{d+1} - 1] \leftarrow a[k + 2^d - 1] + a[k + 2^{d+1} - 1]$
// down-sweep
$a[n-1] \leftarrow 0$
for $d = \log_2 n - 1$ **to** 0 **do**
  for each $k = 0$ to $n-1$ by 2^{d+1} (in parallel) **do**
    $t \leftarrow a[k + 2^d - 1]$
    $a[k + 2^d - 1] \leftarrow a[k + 2^{d+1} - 1]$
    $a[k + 2^{d+1} - 1] \leftarrow t + a[k + 2^{d+1} - 1]$

Its work is $O(N)$ and span $O(\lg N)$, though both hide constants (each sweep does about N operations, and there are $2 \lg N$ steps), and its locality is poor, since the strides grow with d.

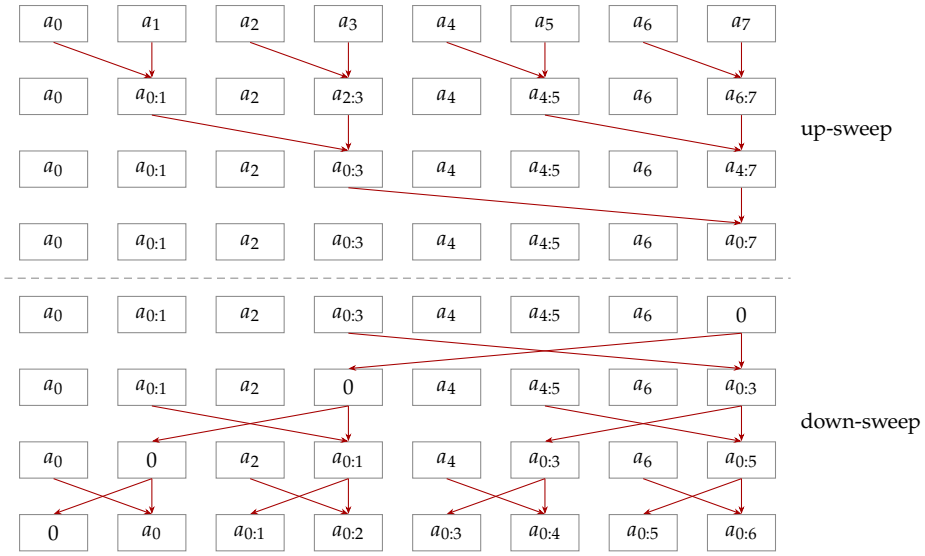


Figure 23: Work-efficient exclusive scan of eight elements. The up-sweep builds partial sums in a tree; the down-sweep sets the root to 0 and, at each node, passes the right value to the left child and the sum to the right child.

Scan on two cores. Running the tree algorithm on two processors wastes effort. A better shared-memory implementation has P_1 sequentially scan elements 0–7 while P_2 scans elements 8–15; then, with base = $a_{0:7}$, P_1 adds the base to P_2's first four results and P_2 to its last four. The work is $O(N)$ with constant only 1.5, and data access has very high spatial locality (contiguous). P_1's accesses to elements 8–11 may cost more on a large NUMA machine, but on a small multi-core they likely cost the same as from P_2.

Scan on a GPU warp. When run by a warp of 32 CUDA threads, each thread returns the exclusive scan result for its element (and `ptr[]` ends up holding the inclusive scan):

```
__device__ int scan_warp(int *ptr, const unsigned int idx)
{
  const unsigned int lane = idx % 32; // index of thread in warp (0..31)

  if (lane >= 1)  ptr[idx] = ptr[idx - 1]  + ptr[idx];
  if (lane >= 2)  ptr[idx] = ptr[idx - 2]  + ptr[idx];
  if (lane >= 4)  ptr[idx] = ptr[idx - 4]  + ptr[idx];
  if (lane >= 8)  ptr[idx] = ptr[idx - 8]  + ptr[idx];
  if (lane >= 16) ptr[idx] = ptr[idx - 16] + ptr[idx];

  return (lane > 0) ? ptr[idx-1] : 0;
}
```

This does $N \lg N$ work, yet it is the right choice: the work-efficient formulation would give low SIMD utilization and need more than twice as many instructions.

Larger scans are built hierarchically. A 128-element scan with a four-warp thread block scans each 32-element chunk with one warp, scans the four chunk totals with a length-4 SIMD scan in warp 0, and adds the resulting bases to warps 1–3:

```
__device__ void scan_block(int* ptr, const unsigned int idx)
{
  const unsigned int lane = idx % 32;    // index of thread in warp (0..31)
  const unsigned int warp_id = idx >> 5; // warp index in block

  int val = scan_warp(ptr, idx);         // Step 1. per-warp partial scan
                                         // (performed by all threads in block,
                                         // with threads in same warp communicating
                                         // through shared memory buffer 'ptr')

  if (lane == 31) ptr[warp_id] = ptr[idx]; // Step 2. thread 31 in each warp copies
  __syncthreads();                         // partial-scan result into per-block
                                           // shared mem

  if (warp_id == 0) scan_warp(ptr, idx);   // Step 3. scan to accumulate bases
  __syncthreads();                         // (only performed by warp 0)

  if (warp_id > 0)                         // Step 4. apply bases to all elements
    val = val + ptr[warp_id-1];            // (performed by all threads in block)
  __syncthreads();

  ptr[idx] = val;
}
```

The code assumes the array length equals the number of threads per block. A one-million-element scan with 1024 elements per block uses three kernel launches: per-block scans, a scan of the block totals in one block, and a pass adding each block's base; beyond about a million elements the second phase must itself be partitioned into multiple blocks.

Although scan has $O(N)$ parallel work, efficient implementations use only as much parallelism as needed to fill the machine, reducing work and communication or synchronization. They match the memory hierarchy with a multi-level implementation (per-block scans in shared memory), and they are heterogeneous, using different algorithms at different levels: a different intra-warp and inter-thread scan in CUDA, and an approach based largely on sequential scan on a CPU with few cores.

8.3 Segmented scan

A common problem is operating on a sequence of sequences: for each vertex in a graph, each incident edge; for each particle, each particle within distance D; for each document, each word.

There are two levels of parallelism, but they are irregular, since edge lists, neighbor lists, and documents vary greatly in size.

Definition: A SEGMENTED SCAN simultaneously performs scans on contiguous partitions of the input sequence. For $A = [[1,2],[6],[1,2,3,4]]$ and $\oplus = +$, the exclusive segmented scan is $[[0,1],[0],[0,1,3,6]]$. A nested sequence is represented by a flat data array and a START-FLAG array; $[[1,2,3],[4,5,6,7,8]]$ becomes flags 1 0 0 1 0 0 0 0 and data 1 2 3 4 5 6 7 8.

The work-efficient algorithm extends to segmented scan by propagating flags during the up-sweep and resetting at segment starts during the down-sweep, which needs a copy of the original flags:

Algorithm 8.2: Work-efficient segmented exclusive scan (with $\oplus = +$)

```
   // up-sweep
1  for d = 0 to log2 n − 1 do
2      for each k = 0 to n − 1 by 2^(d+1) (in parallel) do
3          if flag[k + 2^(d+1) − 1] = 0 then
4              data[k + 2^(d+1) − 1] ← data[k + 2^d − 1] + data[k + 2^(d+1) − 1]
5          flag[k + 2^(d+1) − 1] ← flag[k + 2^d − 1] ∨ flag[k + 2^(d+1) − 1]
   // down-sweep
6  data[n − 1] ← 0
7  for d = log2 n − 1 to 0 do
8      for each k = 0 to n − 1 by 2^(d+1) (in parallel) do
9          t ← data[k + 2^d − 1]
10         data[k + 2^d − 1] ← data[k + 2^(d+1) − 1]
11         if flag_original[k + 2^d] = 1 then
12             data[k + 2^(d+1) − 1] ← 0                    // start of segment
13         else if flag[k + 2^d − 1] = 1 then
14             data[k + 2^(d+1) − 1] ← t
15         else
16             data[k + 2^(d+1) − 1] ← t + data[k + 2^(d+1) − 1]
17         flag[k + 2^d − 1] ← 0
```

Example: *Sparse matrix-vector multiplication.* Most entries of a sparse matrix are zero. Parallelizing over the per-row dot products is easy, but rows hold different amounts of work, which complicates wide SIMD execution. In COMPRESSED SPARSE ROW format, the matrix with rows $(3,0,1,0)$, $(0,2,0,0)$, $(0,0,4,0)$, $(0,2,6,8)$ is stored as `values = [[3,1],[2],[4],[2,6,8]]`, `cols = [[0,2],[1],[2],[1,2,3]]`, and `row_starts = [0,2,3,4]`. To compute $y = Ax$ with scan:

1. map over all nonzeros (a gather from x),

$$products[i] = values[i] \cdot x[cols[i]],$$

   giving $[3x_0, x_2, 2x_1, 4x_2, 2x_1, 6x_2, 8x_3]$;

2. create flags from `row_starts`: $[1,0,1,1,1,0,0]$;

3. inclusive segmented scan with +: $[3x_0,\ 3x_0 + x_2,\ 2x_1,\ 4x_2,\ 2x_1,\ 2x_1 + 6x_2,\ 2x_1 + 6x_2 + 8x_3]$;

4. take the last element of each segment: $y = [3x_0 + x_2,\ 2x_1,\ 4x_2,\ 2x_1 + 6x_2 + 8x_3]$.

In theory scan's parallelism is linear in the number of elements; in practice implementations exploit locality and use only as much parallelism as fills the machine, applying different strategies at different levels. Segmented scan expresses computations on irregular data structures (such as lists of lists) in a regular, data-parallel way.

8.4 Gather, scatter, and other operations

Definition: The operation `gather(index, input, output)` computes $output[i] = input[index[i]]$, and `scatter(index, input, output)` computes $output[index[i]] = input[i]$.

For example, gathering a data sequence $0, 1, \ldots, 15$ with index sequence $3, 12, 4, 9, 9, 15, 13, 0$ yields $3, 12, 4, 9, 9, 15, 13, 0$. A gather machine instruction `gather(R1, R0, mem_base)` loads into vector register `R1` the elements of the buffer at `mem_base` indexed by `R0`. AVX2 (2013) supports gather but not SIMD scatter, which must be a scalar loop; AVX-512 has scatter, and GPUs support both in hardware, though they remain expensive compared with loading or storing a contiguous vector.

If the indices of a scatter are unique and cover all outputs, the scatter is a permutation and can be implemented by sorting the input by index: with index $[0, 2, 1, 4, 3, 6, 7, 5]$ and input $[3, 8, 4, 6, 3, 9, 2, 8]$, the input sorted by index is $[3, 4, 8, 3, 6, 8, 9, 2]$. If indices repeat, the update

$$output[index[i]] \leftarrow \mathrm{op}\big(output[index[i]],\ input[i]\big)$$

needs atomicity, which can be obtained with sort, map, and segmented scan. For index $[1, 1, 0, 2, 0, 0]$: sort the input by index (sorted index $[0, 0, 0, 1, 1, 2]$, input $[in_2, in_4, in_5, in_0, in_1, in_3]$); compute the starts of each run of equal indices, $[1, 0, 0, 1, 0, 1]$; and segmented-scan each run with op, whose last elements are $\mathrm{op}(\mathrm{op}(in_2, in_4), in_5)$, $\mathrm{op}(in_0, in_1)$, and in_3.

Other sequence operations include GROUP BY KEY, which turns a sequence of (key, value) pairs into a sequence of (key, sequence of values), so $(1,3), (2,8), (2,4), (1,6), (3,3), (1,9), (1,2), (2,8)$ becomes $(1, [3, 6, 9, 2]), (2, [8, 4, 8]), (3, [3])$; FILTER, which removes elements not matching a predicate (filtering out odd values of $[3, 8, 4, 6, 3, 9, 2, 8]$ leaves $[8, 4, 6, 2, 8]$); and sort.

8.5 Example: building a grid of particles

Place a million particles into a 16-cell uniform 2D grid, building a 2D array of lists, on a GPU with up to 2048 CUDA threads per SM and 80 SMs. Such grids serve N-body problems: to find all particles within radius R of a particle, use cells of size R and inspect only the surrounding cells.

1. *Parallelize over particles*: each thread computes its particle's cell and appends it to the cell's list under one global lock. Thousands of threads contend for one data structure.

2. *Finer-grained locks*: one lock per cell gives about 16× less contention for uniformly distributed particles.
3. *Parallelize over cells*: each cell independently scans all particles, eliminating synchronization, but there are only 16 parallel tasks where thousands are needed, and the work is 16 times that of the sequential algorithm.
4. *Partial results and merge*: build N partial grids in parallel, for example one per thread block (at least as many blocks as SMs), with all threads of a block updating its grid. Contention drops by a factor of N and synchronization is cheaper, since it uses block-local variables in shared memory, at the cost of extra work to merge the grids and extra memory for N grids.
5. *Data-parallel approach*: map each particle to its cell; sort the (cell, particle) pairs by cell, which permutes the particle indices; then find the start and end of each cell in parallel over the sorted array.

Example: *The data-parallel grid.* Six particles with cells $9, 6, 6, 4, 6, 4$ sort to particle indices $3, 5, 1, 2, 4, 0$ with cells $4, 4, 6, 6, 6, 9$. One invocation per element of the sorted array runs:

```
this_cell = grid_cell[thread_index];
if (thread_index == 0)     // special case for first cell
  cell_starts[this_cell] = 0;
else if (this_cell != grid_cell[thread_index-1]) {
  cell_starts[this_cell] = thread_index;
  cell_ends[grid_cell[thread_index-1]] = thread_index;
}
if (thread_index == numParticles-1)  // special case for last cell
  cell_ends[this_cell] = thread_index+1;
```

giving starts $0, 2, 5$ and (exclusive) ends $2, 5, 6$ for cells $4, 6, 9$. This keeps a large amount of parallelism and removes fine-grained synchronization, at the cost of a sort and extra passes over the data (extra bandwidth).

Example: *Parallel histogram.* The sequential loop `histogram_bins[f(input[i])]++` can be made massively parallel using only map and sort:

```
void compute_bin(float* input, int* bin_ids) {
  bin_ids[thread_index] = f(input[thread_index]);
}

void find_starts(int* bin_ids, int* bin_starts) {
  if (thread_index == 0 || bin_ids[thread_index] != bin_ids[thread_index-1])
    bin_starts[bin_ids[thread_index]] = thread_index;
}

float input[N];
```

```
int   histogram_bins[NUM_BINS];

// temporary buffers
int   bin_ids[N];          // bin_ids[i] = id of bin that element i goes in
int   sorted_bin_ids[N];
int   bin_starts[NUM_BINS]; // initialized to -1

// map f onto input sequence to get bin ids of all elements
launch<<<N>>>compute_bin(input, bin_ids);

// find starting point of each bin in sorted list
sort(N, bin_ids, sorted_bin_ids);
launch<<<N>>>find_starts(sorted_bin_ids, bin_starts);

// compute bin sizes
launch<<<NUM_BINS>>>bin_sizes(bin_starts, histogram_bins, N, NUM_BINS);
```

where `thread_index` identifies each invocation, and `bin_sizes`, launched with one thread per bin, sets an empty bin (start -1) to 0 and otherwise subtracts its start from the start of the next non-empty bin, searching forward past empty bins (or from `num_items` for the last one).

Data-parallel thinking implements algorithms in terms of simple, widely parallelizable, efficiently implemented operations on large collections. It turns irregular parallelism into regular parallelism and fine-grained synchronization into coarse synchronization, but most solutions make multiple passes over the data and are bandwidth hungry. These primitives underlie many parallel and distributed systems, such as CUDA's Thrust library, Apache Spark and Hadoop, and Pandas DataFrame operations.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 8 (Fall 2023).

Lecture 9: Distributed Data-Parallel Computing Using Spark

How do we program 10,000 to 100,000 cores, and avoid losing data when components fail? The programming model is data-parallel operations such as map and reduce, made scalable (to 100,000 cores), fault tolerant, and efficient in their use of memory.

9.1 Clusters and distributed storage

Processing 100 TB of log data (one day at Facebook) at 50 MB/s takes 23 days on one node but 33 minutes on 1000 nodes. Using so many nodes is very hard: thousands of cores are hard to program, and something breaks every hour, so an efficient, reliable, usable framework is needed.

A warehouse-scale computer is a cluster of commodity Linux nodes (multi-core x86), each with private memory, hence separate address spaces and separate operating systems, connected by Ethernet (10–40 Gb/s and more). Thousands of nodes cost under $10M, although the datacenter network is customized and expensive, using supercomputer networking ideas to provide high bandwidth across the datacenter. A node has 16–32 cores, 128 GB to 1 TB of DRAM at about 100 GB/s, and a 10–30 TB SSD at 1–4 GB/s; 20–40 servers share a rack (12–20 kW) and a top-of-rack switch, with 1–2 GB/s to nodes in the same rack and 0.1–2 GB/s to nodes in other racks. Memory is far faster than storage, which is faster than the network, especially across racks. Communication between nodes is message passing.

If nodes can fail, data must be stored persistently in a distributed file system with a global file namespace, such as Google's GFS or Hadoop's HDFS. Typical files are huge (hundreds of GB to TB), rarely updated in place, and mostly read or appended (like logs). Files are split into contiguous chunks (blocks) of 64–256 MB, each replicated two or three times, preferably in different racks, on chunk servers (HDFS DataNodes). A master node (HDFS NameNode) stores the metadata, usually replicated. A client library asks the master where chunks are and then accesses the chunk servers directly.

9.2 MapReduce

Suppose a large log of page views `cs149log.txt` is stored in HDFS, its blocks spread evenly over the nodes, and we want to know which mobile phones students use. Writing this with message passing (MPI) is laborious. Recall map and fold (Lecture 8); reduce applies a binary operation to each element and an accumulated value, for example `reduce` + of $[3, 8, 4, 6, 3, 9, 2, 8]$ is 43. The MapReduce programming model:

```
// called once per line in input file by runtime
// input: string (line of input file)
// output: adds (user_agent, 1) entry to list
void mapper(string line, multimap<string,string>& results) {
  string user_agent = parse_requester_user_agent(line);
  if (is_mobile_client(user_agent))
    results.add(user_agent, 1);
}

// called once per unique key (user_agent) in results
// values is a list of values associated with the given key
void reducer(string key, list<string> values, int& result) {
  int sum = 0;
  for (v in values)
    sum += v;
  result = sum;
}

// iterator over lines of text file
LineByLineReader input("hdfs://cs149log.txt");

// stores output
Writer output("hdfs://...");

// do stuff
runMapReduceJob(mapper, reducer, input, output);
```

This counts page views by each type of mobile phone. The dataflow is map, then group by key, then reduce (it "should be called MapGroupByKeyReduce"): in word count, the mappers emit pairs such as `(the, 1)` for each word of their lines, the pairs are grouped by word, and the reducers sum each group.

Implementing `runMapReduceJob`. Step 1 runs the mapper on all lines. Work can be assigned dynamically from a work queue of input blocks, or by data distribution: each node processes the lines of blocks stored locally. Step 2 prepares the intermediate data for the reducers and step 3 runs the reducer on every key. Each node writes its mapper output for each key to a local file ("Safari iOS values 0" on node 0, and so on); the reduce task for a key, assigned to some node (for example Safari iOS to node 0), gathers that key's files from all nodes and calls the reducer.

At scale, nodes may fail during execution, and some nodes run slower than others (different amounts of work, heterogeneity in the cluster). The job scheduler therefore

- exploits data locality, "moving computation to the data": it runs mapper jobs on nodes that

hold the input blocks and reducer jobs on nodes that already hold most of the data for a key;

- handles node failures: it detects failed jobs and reruns them on other machines, which is possible because inputs reside in persistent storage (the distributed file system);
- handles slow machines (and reduces the latency added by failures) by duplicating jobs on multiple machines.

By providing a data-parallel model, MapReduce greatly simplified cluster programming: automatic division of a job into tasks, locality-aware scheduling, load balancing, and recovery from failures and stragglers.

MapReduce permits only a very simple program structure, a map followed by a reduce by key (DryadLINQ generalizes this to DAGs), and iterative algorithms must load from disk on each iteration. PageRank, for instance, runs a MapReduce job per iteration, each reading the previous iteration's graph from HDFS and writing the new one. Users soon needed more complex, multi-stage applications (iterative machine learning and graph processing) and interactive ad-hoc queries, each of which rereads the input from HDFS. Given the node bandwidths above, keeping intermediate data in DRAM would be far faster, and despite huge datasets, many working sets fit in memory: in 2009, 96% of Facebook jobs, 82% of Microsoft jobs, and 97.5% of Yahoo! jobs needed at most 32 GB of memory.

9.3 Spark and resilient distributed datasets

Spark (Zaharia et al., NSDI 2012) provides in-memory, fault-tolerant distributed computing for cluster-scale computations with significant reuse of intermediate datasets: iterative machine learning and graph algorithms, and interactive data mining that loads a large dataset into the aggregate memory of the cluster and runs multiple ad-hoc queries. It avoids writing intermediates to a persistent distributed file system. The challenge is implementing fault tolerance efficiently for large in-memory computations. Options include replicating all computations (expensive, lowering peak throughput), periodic checkpointing and rollback to the last checkpoint, and logging updates (commands and data), which has high overhead. MapReduce checkpoints by writing results to the file system after each map or reduce step, its scheduler's list of outstanding jobs acts as a log, and the functional structure of programs allows restart at the granularity of a single mapper or reducer invocation.

Definition: A RESILIENT DISTRIBUTED DATASET (RDD) is Spark's key abstraction: a read-only (immutable) ordered collection of records. RDDs can only be created by deterministic TRANSFORMATIONS of data in persistent storage or of existing RDDs. ACTIONS on RDDs return data to the application.

```
// create RDD from file system data
val lines = spark.textFile("hdfs://cs149log.txt");

// create RDD using filter() transformation on lines
val mobileViews = lines.filter((x: String) => isMobileClient(x));

// instruct Spark runtime to try to keep mobileViews in memory
mobileViews.persist();

// create a new RDD by filtering mobileViews
// then count number of elements in new RDD via count() action
val numViews = mobileViews.filter(_.contains("Safari")).count();

// 1. create new RDD by filtering only Chrome views
// 2. for each element, split string and take timestamp of
//    page view
// 3. convert RDD to a scalar sequence (collect() action)
val timestamps = mobileViews.filter(_.contains("Chrome"))
                            .map(_.split(" ")(0))
                            .collect();
```

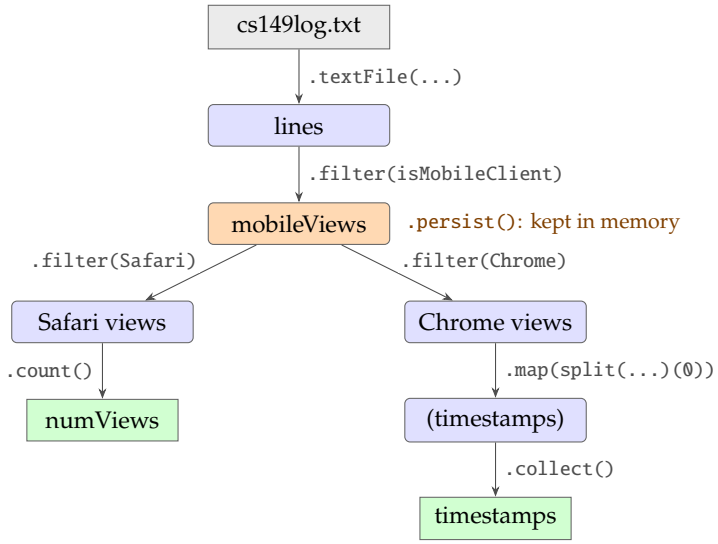


Figure 24: The lineage of the RDDs in the example: transformations create RDDs, and actions return values to the application.

The MapReduce example becomes

```
val perAgentCounts = spark.textFile("hdfs://cs149log.txt")
                          .filter(x => isMobileClient(x))
                          .map(x => (parseUserAgent(x),1))
                          .reduceByKey((x,y) => x+y)
                          .collect();
```

which returns an array of (string, int) pairs. The LINEAGE of an RDD is the sequence of RDD operations needed to compute it (Figure 24).

Transformation	Type
map($f : T \Rightarrow U$)	RDD[T] $\Rightarrow$ RDD[U]
filter($f : T \Rightarrow$ Bool)	RDD[T] $\Rightarrow$ RDD[T]
flatMap($f : T \Rightarrow$ Seq[U])	RDD[T] $\Rightarrow$ RDD[U]
sample(*fraction* : Float)	RDD[T] $\Rightarrow$ RDD[T] (deterministic sampling)
groupByKey()	RDD[(K, V)] $\Rightarrow$ RDD[$(K,$ Seq[V]$)$]
reduceByKey($f : (V, V) \Rightarrow V$)	RDD[(K, V)] $\Rightarrow$ RDD[(K, V)]
union()	(RDD[T], RDD[T]) $\Rightarrow$ RDD[T]
join()	(RDD[(K, V)], RDD[(K, W)]) $\Rightarrow$ RDD[$(K, (V, W))$]
cogroup()	(RDD[(K, V)], RDD[(K, W)]) $\Rightarrow$ RDD[$(K,$ (Seq[V], Seq[W]))]
crossProduct()	(RDD[T], RDD[U]) $\Rightarrow$ RDD[(T, U)]
mapValues($f : V \Rightarrow W$)	RDD[(K, V)] $\Rightarrow$ RDD[(K, W)] (preserves partitioning)
sort(c : Comparator[K])	RDD[(K, V)] $\Rightarrow$ RDD[(K, V)]
partitionBy(p : Partitioner[K])	RDD[(K, V)] $\Rightarrow$ RDD[(K, V)]
Action	**Type**
count()	RDD[T] $\Rightarrow$ Long
collect()	RDD[T] $\Rightarrow$ Seq[T]
reduce($f : (T, T) \Rightarrow T$)	RDD[T] $\Rightarrow$ T
lookup($k : K$)	RDD[(K, V)] $\Rightarrow$ Seq[V] (on hash/range partitioned RDDs)
save(*path* : String)	outputs the RDD to a storage system, e.g., HDFS

9.4 Implementing RDDs

Should RDDs be thought of as arrays? For the program

```
val lines = spark.textFile("hdfs://cs149log.txt");
val lower = lines.map(_.toLower());
val mobileViews = lower.filter(x => isMobileClient(x));
val howMany = mobileViews.count();
```

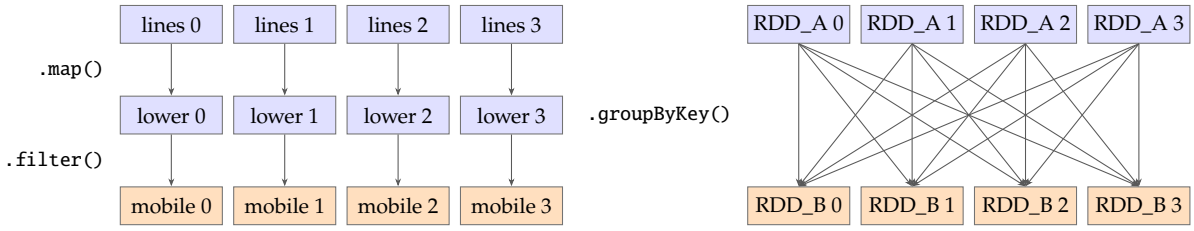
parallel performance requires parallelism and locality, so each RDD is divided into PARTITIONS, one per file block, kept on the node holding the block. Materializing every RDD in memory, however, would be huge, larger than the original file on disk.

Recall two optimizations that globally restructure the order of computation to improve producer-consumer locality (arithmetic intensity): LOOP FUSION (Lecture 6), and TILING, as in a two-pass separable blur over a 1024×1024 image that, instead of blurring the whole image horizontally into a $W \times (H + 2)$ temporary buffer and then vertically, processes chunks of rows so the temporary buffer is only $W \times (CHUNK_SIZE + 2)$. The four-line Spark program can likewise run as a streaming loop that keeps only one line in memory and reads the input once:

```
int count = 0;
while (!inputFile.eof()) {
  string line = inputFile.readLine();
  string lower = line.toLower;
  if (isMobileClient(lower))
    count++;
}
```

Definition: In a NARROW DEPENDENCY, each partition of the parent RDD is referenced by at most one partition of the child RDD. Narrow dependencies allow operations to be fused (apply `map` and then `filter` to each input element at once), and here no communication between nodes is needed except one integer per node for the final `count()`. In a WIDE DEPENDENCY, a parent partition is referenced by multiple child partitions (Figure 25).

This is why `map` and `filter` can be fused but `groupByKey` and `sort` cannot. `groupByKey`, $\text{RDD}[(K, V)] \to \text{RDD}[(K, \text{Seq}[V])]$, creates wide dependencies: all of the parent must be computed before the child, it may induce all-to-all communication, and a node failure may trigger significant recomputation of ancestor lineage.



(a) narrow dependencies (b) wide dependencies

Figure 25: (a) Narrow dependencies: `map` and `filter` can be fused per partition. (b) Wide dependencies: `groupByKey` needs data from every parent partition.

The cost of operations depends on partitioning. Consider

$$join : \text{RDD}[(K, V)],\ \text{RDD}[(K, W)] \to \text{RDD}[(K, (V, W))].$$

If both RDDs are hash partitioned by key (say, username) but with different hash partitions, the join creates wide dependencies; if they share the same partitioning, matching keys are co-located and the join creates only narrow dependencies. `partitionBy()` tells Spark how to partition an RDD

(HashPartitioner, RangePartitioner):

```
// create RDD from file system data
val lines = spark.textFile("hdfs://cs149log.txt");
// clientInfo holds (useragent, "yes"/"no") pairs
val clientInfo = spark.textFile("hdfs://clientssupported.txt");

// create RDD using filter() transformation on lines
val mobileViews = lines.filter(x => isMobileClient(x)).map(x => parseUserAgent(x));

// HashPartitioner maps keys to integers
val partitioner = spark.HashPartitioner(100);

// inform Spark of partition
// .persist() also instructs Spark to try to keep dataset in memory
val mobileViewPartitioned = mobileViews.partitionBy(partitioner)
                                       .persist();
val clientInfoPartitioned = clientInfo.partitionBy(partitioner)
                                      .persist();

// join useragents with whether they are supported or not supported
// Note: this join only creates narrow dependencies due to the
// explicit partitioning above
val joined = mobileViewPartitioned.join(clientInfoPartitioned);
```

`.persist()` tells Spark the RDD's contents should be retained in memory; `.persist(RELIABLE)` stores them in durable storage, like a checkpoint.

Resilience via lineage. RDD transformations are bulk, deterministic, and functional, so the runtime can always reconstruct an RDD's contents from its lineage, which is a log of transformations. The log is cheap because it records bulk data-parallel operations, unlike logging fine-grained operations as in a database. If a node crashes and loses, say, partitions 2 and 3 of `timestamps`, the runtime reloads the needed subset of input from disk (file system data is replicated, so blocks 2 and 3 remain accessible) and recomputes the entire sequence of operations in the lineage to regenerate those partitions, possibly on other nodes.

Performance. On 100 GB of data on a 100-node cluster, Spark's later iterations of logistic regression take about 3 s against 76 s for Hadoop and 62 s for HadoopBM (Hadoop with binary input in an in-memory HDFS); for k-means they take 33 s against 106 s and 87 s. The Hadoop baseline parses text input in every iteration; HadoopBM's first iteration is slow because it runs an extra job to copy the binary input into in-memory HDFS; and accessing HDFS, even in memory, has high overhead (multiple memory copies in the file system, a checksum, and conversion from serialized form to Java objects).

Scale out is not the whole story. Distributed systems for the cloud address scale-out parallelism, resiliency to failures, and the complexity of managing clusters, and have driven the explosion of big-data analytics. But in a comparison of 20 iterations of PageRank on the Twitter graph (41.7M nodes, 1.47B edges), Spark on 128 cores took 857 s and GraphLab on 128 cores 249 s, while a single thread took 300 s from SSD and 275 s from RAM, and further optimization of the single-threaded baseline brought the time down to 110 s. McSherry et al. define COST, the configuration that outperforms a single thread, and observe that many published systems have unbounded COST: no configuration outperforms the best single-threaded implementation. With growing DRAM and faster SSDs, efforts aim to improve Spark's CPU utilization and reduce its COST through efficient code generation (SIMD kernels, GPUs and other accelerators), which requires changing RDD storage layouts (for example structure of arrays instead of array of structures), as in Project Tungsten, Weld, and IBM's SparkGPU. High-performance computing ideas influence performance-oriented distributed systems, and scientific computing can in turn learn about elasticity and utility computing from distributed systems.

Spark thus introduces an opaque sequence abstraction, the RDD, to encapsulate the intermediates of cluster computations, which frameworks like Hadoop MapReduce stored in the file system ("files are a poor abstraction for intermediate variables in large-scale data-parallel programs"). RDDs are read-only and created by deterministic data-parallel operators; lineage is tracked and used for locality-aware scheduling and for fault tolerance by recomputing lost partitions rather than restoring checkpoints (although `.persist(RELIABLE)` allows checkpointing when lineage is long); and bulk operations keep the overhead of lineage tracking low. Because all collections are RDDs scheduled by the Spark scheduler, domain-specific frameworks built on Spark compose: Spark SQL (applying transformations to RDDs produced by SQL queries), a machine learning library, and a GraphLab-like graph library.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 9 (Fall 2023).

Lecture 10: Efficiently Evaluating DNNs on GPUs

Deep neural networks come in many topologies (VGG, ResNet, Inception, MobileNet, fully convolutional segmentation networks, many variants on common backbones) and must run efficiently on many target devices. This unit is about evaluating them efficiently.

10.1 Convolutional neural networks

A DNN is a circuit. An expression such as $\max(\max(0, ab + cd) + ef + gh,\ ij)$ is a small network of multiplies, adds, and max operations. The basic unit ("neuron") with n inputs has $n + 1$ parameters (weights and a bias) and computes $f\left(\sum_i x_i w_i + b\right)$; with the sigmoid $f(x) = 1/(1 + e^{-x})$ it is a binary classifier, and a common choice of f is the RECTIFIED LINEAR UNIT (ReLU) $f(x) = \max(0, x)$. A FULLY CONNECTED LAYER, in which every output depends on every input, is a matrix-vector product followed by an element-wise f: $y = f(Wx + b)$. In a sparsely (locally) connected layer each unit receives inputs from only a few nearby inputs.

The C code

```
int WIDTH = 1024;
int HEIGHT = 1024;
float input[(WIDTH+2) * (HEIGHT+2)];
float output[WIDTH * HEIGHT];

float weights[] = {1.f/9, 1.f/9, 1.f/9,
                   1.f/9, 1.f/9, 1.f/9,
                   1.f/9, 1.f/9, 1.f/9};

for (int j=0; j<HEIGHT; j++) {
  for (int i=0; i<WIDTH; i++) {
    float tmp = 0.f;
    for (int jj=0; jj<3; jj++)
      for (int ii=0; ii<3; ii++)
        tmp += input[(j+jj)*(WIDTH+2) + (i+ii)] * weights[jj*3 + ii];
    output[j*WIDTH + i] = tmp;
  }
}
```

performs a 3×3 blur. A CONVOLUTIONAL LAYER is locally connected, and all its units share the same parameters (the same weights and bias). Filters act as pattern detectors: the filter with rows $(-1, 0, 1), (-2, 0, 2), (-1, 0, 1)$ responds to horizontal gradients and its transpose to vertical gradients, and the magnitude of an output pixel is the filter's response to the region around the corresponding

input pixel. A layer applies many filters at once, for example 96 filters of size $11 \times 11 \times 3$ on an RGB image: a $W \times H$ single-channel input convolved with $3 \times 3 \times$ *num_filters* weights gives $W \times H \times$ *num_filters* responses. Layers follow one another: convolution, then ReLU, then pooling (the maximum response in each 2×2 region), which reduces the data to $W/2 \times H/2 \times$ *num_filters*.

10.2 Approach 1: more efficient topologies

Algorithmic innovation in network topology (ResNet, Inception, MobileNet) has greatly improved accuracy per unit cost. MobileNet factors a standard convolution of *NUM_FILTERS* filters of size $3 \times 3 \times$ *NUM_CHANNELS* into a DEPTHWISE SEPARABLE CONVOLUTION: *NUM_CHANNELS* separate $3 \times 3 \times 1$ convolutions, one per input channel, followed by *NUM_FILTERS* $1 \times 1 \times$ *NUM_CHANNELS* (pointwise) convolutions that combine the results.

Network	ImageNet top-1	Parameters	Cost/image (MADDs)	Year
VGG-16	71.5%	138M	15B	2014
GoogleNet	70%	6.8M	1.5B	2015
ResNet-18	73% (10-crop)	11.7M	1.8B	2016
MobileNet-224	70.5%	4.2M	0.6B	2017

From 2014 to 2017 the cost at similar accuracy fell by about 25×.

10.3 Approach 2: implementing layers efficiently

A direct implementation of a batched convolution layer is a loop nest:

```
// input and output activations
float input[IMAGE_BATCH_SIZE][INPUT_HEIGHT][INPUT_WIDTH][INPUT_DEPTH];
float output[IMAGE_BATCH_SIZE][INPUT_HEIGHT][INPUT_WIDTH][LAYER_NUM_FILTERS];
float layer_weights[LAYER_NUM_FILTERS][LAYER_CONVY][LAYER_CONVX][INPUT_DEPTH];
float layer_biases[LAYER_NUM_FILTERS];

// assumes convolution stride is 1
for (int img=0; img<IMAGE_BATCH_SIZE; img++)
  for (int j=0; j<INPUT_HEIGHT; j++)
    for (int i=0; i<INPUT_WIDTH; i++)
      for (int f=0; f<LAYER_NUM_FILTERS; f++) {
        float tmp = layer_biases[f];
        // sum over filter responses of input channels
        for (int kk=0; kk<INPUT_DEPTH; kk++)
          for (int jj=0; jj<LAYER_FILTER_Y; jj++)     // spatial convolution (Y)
            for (int ii=0; ii<LAYER_FILTER_X; ii++)   // spatial convolution (X)
              tmp += layer_weights[f][jj][ii][kk] * input[img][j+jj][i+ii][kk];
        output[img][j][i][f] = tmp;
      }
```

It has seven loops with significant data reuse: filter weights are reused across the convolution, and input values are reused across filters.

Explicit GEMM. A 3×3 convolution can be written as a matrix product by constructing a matrix from the input image ("im2col"): each row holds the 3×3 neighborhood of one output pixel (with zero padding at the border), so the input of size $W \times H$ becomes a $WH \times 9$ matrix, an $O(N)$ storage overhead for a filter with N elements. Multiplying by a $9 \times \textit{num_filters}$ weight matrix computes all filters at once:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & x_{00} & x_{01} & 0 & x_{10} & x_{11} \\ 0 & 0 & 0 & x_{00} & x_{01} & x_{02} & x_{10} & x_{11} & x_{12} \\ 0 & 0 & 0 & x_{01} & x_{02} & x_{03} & x_{11} & x_{12} & x_{13} \\ & & & & \vdots & & & & \\ x_{00} & x_{01} & x_{02} & x_{10} & x_{11} & x_{12} & x_{20} & x_{21} & x_{22} \\ & & & & \vdots & & & & \end{bmatrix} \begin{bmatrix} w_{00} & w_{01} & \cdots & w_{0K} \\ w_{10} & w_{11} & \cdots & w_{1K} \\ \vdots & \vdots & & \vdots \\ w_{80} & w_{81} & \cdots & w_{8K} \end{bmatrix}.$$

With multiple input channels, the response of each filter is summed over channels, equivalent to a $3 \times 3 \times \textit{num_channels}$ convolution on $W \times H \times \textit{num_channels}$ data: the rows concatenate the neighborhoods of all channels ($9 \cdot \textit{num_channels}$ columns). With filters of spatial support $R \times S$, C input channels, K filters, and batch size N, the layer becomes a GEMM with $M = NPQ$ (one row per output pixel of the $P \times Q$ output), $N_{\text{GEMM}} = K$, and $K_{\text{GEMM}} = CRS$. High-performance GEMM libraries exist, but using them off the shelf requires materializing the input matrix, which increases DRAM traffic by a factor of $R \times S$ (to read the activation tensor and build the convolution matrix)

and needs a large amount of auxiliary storage.

Dense matrix multiplication. The naive loop

```
float A[M][K];
float B[K][N];
float C[M][N];

// compute C += A * B
#pragma omp parallel for
for (int j=0; j<M; j++)
  for (int i=0; i<N; i++)
    for (int k=0; k<K; k++)
      C[j][i] += A[j][k] * B[k][i];
```

has low arithmetic intensity, since it does not exploit temporal locality in its accesses to A and B. Blocking computes a partial result for a block of C while the required blocks of A and B remain in cache:

```
#pragma omp parallel for
for (int jblock=0; jblock<M; jblock+=BLOCKSIZE_J)
  for (int iblock=0; iblock<N; iblock+=BLOCKSIZE_I)
    for (int kblock=0; kblock<K; kblock+=BLOCKSIZE_K)
      for (int j=0; j<BLOCKSIZE_J; j++)
        for (int i=0; i<BLOCKSIZE_I; i++)
          for (int k=0; k<BLOCKSIZE_K; k++)
            C[jblock+j][iblock+i] += A[jblock+j][kblock+k] * B[kblock+k][iblock+i];
```

The block sizes are chosen so that blocks of A, B, and C stay resident; larger blocks give more reuse, but only while they fit. Hierarchical blocking adds further levels of loops for the L2 and L1 caches, and a final level for register locality.

Within a block, SIMD parallelism can be exploited in several ways:

1. vectorize the i loop: load a vector of C, and for each k multiply a splatted (broadcast) element of A by a vector of B and accumulate; this improves spatial locality in accessing B but enlarges the working set by the SIMD width and still walks over B in large steps;
2. if the i dimension is small, pre-transpose the block of B into a temporary buffer and vectorize the innermost k loop as SIMD dot products of a row of A with a row of B^T;
3. with pre-transposed blocks of A and C, keep a $SIMD_WIDTH \times SIMD_WIDTH$ chunk of C^T in registers and accumulate outer products, issuing independent multiply-adds in the innermost loop.

Different layers of one DNN have different matrix dimensions (in MobileNet the weights and

activations range from $224 \times 224 \times 3$ inputs with 32 filters to $7 \times 7 \times 1024$ activations with 1024 filters), so they may benefit from different scheduling strategies, and SIMD widths matter: a hard problem for library implementers.

Implicit GEMM. Instead of materializing the full convolution matrix, an implementation can index directly into the weight and activation tensors. A better version materializes only a sub-block of the convolution matrix at a time in GPU on-chip shared memory and uses a well-tuned shared-memory GEMM on it. This needs no additional off-chip storage and does not increase DRAM traffic. NVIDIA's CUTLASS provides such building blocks for custom high-performance layers (for unusual sizes not heavily tuned in cuDNN): fast shared-memory and warp-level GEMMs, iterators for fast block loading and tensor indexing, and tensor reductions. The GPU needs a lot of parallel work: a layer with batch size $N = 1$ and a 64×64 output of 128 channels has $64 \cdot 64 \cdot 128 = 524$K outputs (2 MB in float32), while $N = 32$ with 256×256 outputs has 256M outputs (1 GB), and the larger problem achieves much higher throughput. One can also simply implement the direct loop nest well.

Algorithmic improvements. Convolution can be computed in the Fourier domain, where it becomes element-wise multiplication; the overhead is an FFT of the inputs and an inverse FFT of the responses ($O(N \lg N)$), and the inverse transform is amortized over all input channels because of the summation over inputs. Winograd convolution is work efficient. For two outputs of a 3-tap 1D convolution with weights w_0, w_1, w_2,

$$\begin{bmatrix} y_0 \\ y_1 \end{bmatrix} = \begin{bmatrix} x_0 & x_1 & x_2 \\ x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} m_1 + m_2 + m_3 \\ m_2 - m_3 - m_4 \end{bmatrix},$$

$$m_1 = (x_0 - x_2)w_0, \quad m_2 = (x_1 + x_2)\frac{w_0+w_1+w_2}{2}, \quad m_3 = (x_2 - x_1)\frac{w_0-w_1+w_2}{2}, \quad m_4 = (x_1 - x_3)w_2.$$

The filter-dependent factors can be precomputed, so this takes 4 multiplies and 8 additions (4 to compute the m's and 4 to form the outputs), against 6 multiplies and 4 adds directly. In 2D the savings are larger: for a 3×3 filter, 2.25× fewer multiplies for a 2×2 block of output.

Vendor libraries (NVIDIA cuDNN, Intel oneAPI Deep Neural Network Library) offer high-performance implementations of key layers, on which frameworks such as TensorFlow build their operations. cuDNN's convolution entry point offers many algorithms: implicit GEMM (with or without precomputed indices), explicit GEMM, direct convolution, FFT (optionally tiled), and Winograd (fused or non-fused).

10.4 Fusion

In the sequence convolution ($N \times H \times W \times C \to N \times H \times W \times K$), then scale and bias, then 2×2 max pooling ($\to N \times H/2 \times W/2 \times K$), writing 1 GB of convolution output to memory and reading it back between each operation costs a great deal of bandwidth. But the per-element scale and bias can be applied right after each element is computed by the convolution, and the max pool output can be computed once each 2×2 region of output is complete, so the three operations can be fused into one, `output[img][j][i][f] = tmp*scale + bias` in the loop nest above. Fusing the max pool requires restructuring the loops so that each 2×2 output region is computed together and reduced before being written.

Attention. A transformer maps a sequence of tokens to a sequence of tokens, for example for next-word prediction. Its attention module computes, for $Q, K, V \in \mathbb{R}^{N \times d}$,

$$S = QK^T \in \mathbb{R}^{N \times N}, \qquad P = \text{softmax}(S) \in \mathbb{R}^{N \times N}, \qquad O = PV \in \mathbb{R}^{N \times d},$$

with the softmax taken over each row x of S:

$$\text{softmax}(x) = \frac{f(x)}{\ell(x)}, \qquad m(x) = \max_i x_i,$$

$$f(x) = \left[e^{x_1 - m(x)} \ \cdots \ e^{x_B - m(x)}\right], \qquad \ell(x) = \sum_i f(x)_i = \sum_i e^{x_i - m(x)}.$$

N can be thousands for long sequences, and a naive implementation uses N^2 space. The softmax can be computed in chunks: splitting $x = [x^{(1)}\ x^{(2)}]$,

$$m(x) = \max\left(m(x^{(1)}), m(x^{(2)})\right), \quad f(x) = \left[e^{m(x^{(1)}) - m(x)} f(x^{(1)}) \ \ e^{m(x^{(2)}) - m(x)} f(x^{(2)})\right],$$

$$\ell(x) = e^{m(x^{(1)}) - m(x)} \ell(x^{(1)}) + e^{m(x^{(2)}) - m(x)} \ell(x^{(2)}).$$

Definition: Fused attention processes blocks: for each j and each i, load blocks Q_i, K_j^T, V_j, and O_i; compute $S_{ij} = Q_i K_j^T$; compute row-wise $m_{ij} = m(S_{ij})$, $P_{ij} = f(S_{ij})$, and $\ell_{ij} = \ell(S_{ij})$; and multiply $P_{ij} V_j$ and accumulate it into O_i with the rescalings above (Figure 26).

This never materializes the N^2 matrices, saving memory footprint, and saves bandwidth through high arithmetic intensity: each step reads three blocks, performs two matrix multiplications and a few row summations, and accumulates into an O block resident in cache. It performs additional computation compared with the original, since prior values of O must be rescaled at each step.

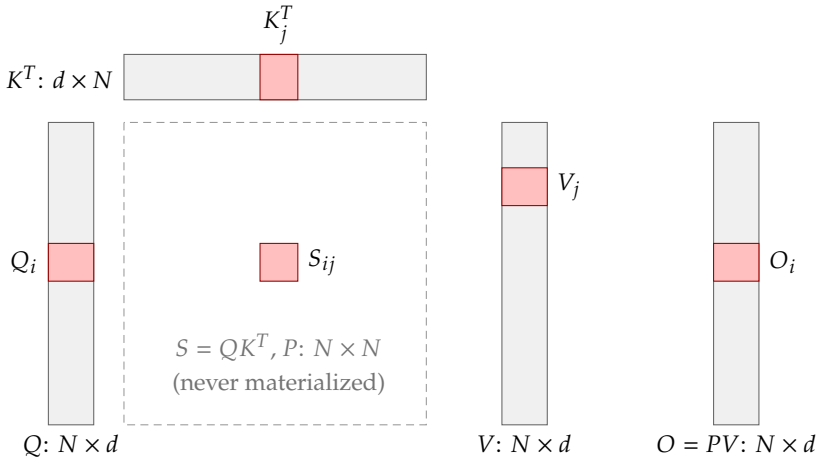


Figure 26: Fused attention: the blocks Q_i, K_j^T, V_j are loaded, the block S_{ij} is computed on chip, and $P_{ij}V_j$ is accumulated into O_i.

Older frameworks had library writers hard-code a few fused operations (as in TensorFlow). More flexibly, the cuDNN "backend" lets a compiler generate new implementations that fuse multiple operations into a single node that executes efficiently, without run-time overhead or communicating intermediate results through memory (Halide's "compute at", Lecture 14), and many compiler-based efforts aim to schedule key DNN operations automatically. Another trick is low-precision values for weights and activations: 16-bit and 8-bit values are common, and in the extreme, 1-bit.

DNN evaluation is optimized in three ways: by better algorithms (manually designed models, with parameters such as depth, filter width, number of filters per layer, and stride, and often automatic search for efficient topologies), by software optimization through good scheduling of performance-critical operations (loop blocking and tiling, fusion), typically by hand though much research aims to automate it, and by approximation that compresses models (lower precision, and automatic sparsification or pruning).

10.5 Hardware for DNNs

GPUs are a good platform for DNNs: the computations have high arithmetic intensity, similar to dense matrix multiplication; they benefit from flop-rich GPU architectures; and highly optimized kernel libraries such as cuDNN exist. But a general-purpose processor may not be needed. Complex instructions amortize the overhead of instruction-stream control over many operations: fused multiply-add ($ax + b$), a 4-component dot product, and a 4×4 matrix multiply-accumulate $AB + C$. Bill Dally estimates the overhead of programmability (instruction stream, control) relative to the arithmetic as 2000% for a half-precision FMA, 500% for a half-precision 4-wide dot product, and 27% for a half-precision 4×4 matrix multiply-accumulate.

Example: *NVIDIA A100 (Ampere).* Each SM has 64 fp32 mul-add ALUs, 32 int32 ALUs, and 4 TENSOR CORES, each executing an 8×4 by 4×8 matrix multiply-accumulate $AB + C$ with A, B in fp16

and accumulation in fp32. With 108 SMs (6,912 fp32 ALUs and 432 tensor cores) at up to 1.4 GHz, the GA100 delivers 19.5 TFLOPs in fp32 plus 312 TFLOPs of mixed fp16/fp32 in the tensor cores.

To maximize efficiency, many hardware accelerators target DNN inference and training: the Huawei Kirin NPU, Google TPU, Apple Neural Engine, GraphCore IPU, Intel's deep learning inference accelerator, SambaNova SN10, Cerebras Wafer Scale Engine, and GPUs with tensor cores.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 10 (Fall 2023).

Lecture 11: Cache Coherence

Caches occupy a large part of a modern CPU (about 30% of the die area of an Intel Core i7), and they exploit locality. The Intel Skylake hierarchy (2015) has, per core, a 32 KB 8-way set-associative write-back L1 data cache (4-cycle latency, two 32-byte loads and one 32-byte store per clock) and a 256 KB 4-way write-back L2 (12-cycle latency, 64 B per clock); the cores share an 8 MB inclusive 16-way L3 with one bank per core on a ring interconnect (42-cycle latency, 32 B per clock per bank). Lines are 64 bytes, and a core supports 72 outstanding loads and 56 outstanding stores. Misses are classified by the "three Cs" model as cold, capacity, and conflict misses.

11.1 Cache design and write policies

A cache line holds the line state (for example a dirty bit), a tag, and the data (64 bytes on modern Intel processors). A write-through cache propagates every write to memory immediately; a write-back cache updates memory only when a dirty line is evicted. A write-allocate cache brings the line into the cache on a write miss; a write-no-allocate cache does not.

Example: *A write miss in a write-allocate, write-back cache* (uniprocessor), executing `int x = 1;`:

1. the processor writes to an address that misses in the cache;
2. the cache selects a location for the line, and if a dirty line occupies it, writes that line out to memory;
3. the cache loads the line from memory ("allocates the line");
4. the whole line is fetched, and the 32 bits of `x` are updated;
5. the line is marked dirty.

11.2 The coherence problem

In the shared address space model, threads communicate implicitly through memory: thread 1 stores to `X`, and later thread 2 reads `X` and observes the update. A reasonable expectation of memory is that reading address X returns the last value written to X by any processor. But modern processors replicate memory contents in local caches, so processors can observe different values for the same location.

With write-back caches and an initial value of 0 at address X:

Action	P1 $	P2 $	P3 $	P4 $	mem[X]
					0
P1 load X	0 (miss)				0
P2 load X	0	0 (miss)			0
P1 store X	1	0			0
P3 load X	1	0	0 (miss)		0
P3 store X	1	0	2		0
P2 load X	1	0 (hit)	2		0
P1 load Y (evicts X)		0	2		1

P3 reads a stale 0 after P1 wrote 1, P2 still sees 0 after P3 wrote 2, and memory ends up holding P1's value. This is not a mutual exclusion problem and cannot be fixed by adding locks: it is created by replicating the data at address X in local caches. The coherence problem exists because the abstraction of a single shared address space is implemented by both global storage (main memory) and per-processor local storage (caches).

On a uniprocessor this behavior is simple to provide, since writes come from one source, except for I/O devices using DMA. If a processor writes to a buffer and tells a network card to send it, the card may transfer stale data if the processor's cached writes have not been flushed; if the card copies a received message into memory by DMA, the CPU may read stale cached data. Common solutions are uncached stores to shared buffers (in driver code), and operating-system support that marks pages holding shared buffers as uncacheable or explicitly flushes them when I/O completes. DMA transfers are infrequent compared with loads and stores, so these heavyweight software solutions are acceptable.

What does "last" mean when two processors write at the same time, or when a read follows a write too closely for the write to be communicated? In a sequential program "last" is determined by program order, not time, and this holds within one thread; across threads a meaningful definition of order is needed.

Definition 11.1 [Coherence]. A memory system is coherent if the results of a parallel program's execution are such that, for each memory location, there is a hypothetical serial order of all program operations to that location (by all processors) that is consistent with the results of execution, and

1. memory operations issued by any one processor occur in the order issued by that processor;
2. the value returned by a read is the value written by the last write to the location in the serial order.

For example, the operations on X might serialize as P0 writes 5; P1 reads 5; P2 reads 5; P0 reads 5; P1 writes 25; P0 reads 25. Implementations maintain two invariants for every address x over time (Figure 27):

- SINGLE-WRITER, MULTIPLE-READER (SWMR): in a read-write epoch a single processor may write x (and read it); in a read-only epoch any number of processors may only read it;
- DATA-VALUE INVARIANT (WRITE SERIALIZATION): the value at the start of an epoch equals the value at the end of the last read-write epoch.

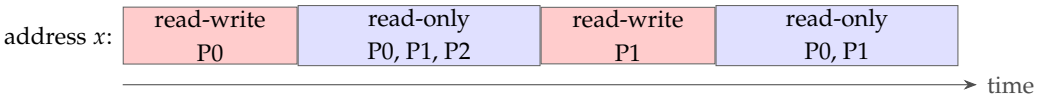


Figure 27: Coherence as a sequence of read-write and read-only epochs for one address.

Coherence can be implemented in software at coarse grain (virtual memory pages), with the operating system using the page-fault mechanism to propagate writes, for example across a cluster of workstations; false sharing makes this slow. Hardware solutions work at the granularity of cache lines and are either snooping based or directory based.

A single cache shared by all processors eliminates the problem of replicating state, but scales poorly: the point of a cache is to be local and fast, and many clients cause interference (conflict misses) and contention. Shared caches do have benefits: they facilitate fine-grained sharing of overlapping working sets, and loads by one processor may prefetch lines for another (in `x[i] = y[i] + y[i+1] + y[i+2]`, neighboring iterations share `y` values). The Sun Niagara 2 connects eight cores to shared L2 banks through a crossbar.

11.3 Snooping coherence

Definition: In a SNOOPING cache-coherence scheme, all coherence-related activity is broadcast to all processors, more precisely to their cache controllers, which monitor ("snoop") memory operations and follow a CACHE COHERENCE PROTOCOL to maintain coherence. A cache controller must now respond to both its local processor's loads and stores and the coherence activity broadcast over the interconnect. If every controller follows the protocol, the caches cooperate to keep memory coherent.

A very simple protocol. With write-through caches and coherence at cache-line granularity, a cache controller broadcasts an invalidation message on every write, so the next read by another processor misses and retrieves the updated value from memory:

Action	Interconnect activity	P0 $	P1 $	mem[X]
				0
P0 load X	cache miss for X	0		0
P1 load X	cache miss for X	0	0	0
P0 write 100 to X	invalidation for X	100		100
P1 load X	cache miss for X	100	100	100

But write-through is inefficient: every write goes to memory, demanding very high bandwidth. Write-back caches absorb most writes as cache hits, which requires a more sophisticated protocol. The dirty state of a line now indicates exclusive ownership (a read-write epoch): the cache with a *modified* line is the only one with a valid copy and may write it, and as *owner* it is responsible for supplying the data when another processor tries to load it, since memory is stale. A shared bus provides the two properties these protocols rely on: every controller observes every transaction, and transactions are serialized.

Invalidation-based write-back protocols. A line in the modified state can be modified without notifying other caches, and a processor can only write lines in the modified state; to obtain one it sends a message telling all other caches it wants exclusive access, and a cache controller that sees such a request for a line it holds must invalidate its copy.

Definition: The MSI protocol has three line states: INVALID (I), as in a uniprocessor cache; SHARED (S), valid in one or more caches with memory up to date; and MODIFIED (M), valid in exactly one cache ("dirty" or "exclusive"). Its inputs are two processor operations, PrRd and PrWr, and three bus transactions: BusRd (obtain a copy with no intent to modify), BusRdX (obtain a copy with intent to modify), and BusWB (write a dirty line back to memory).

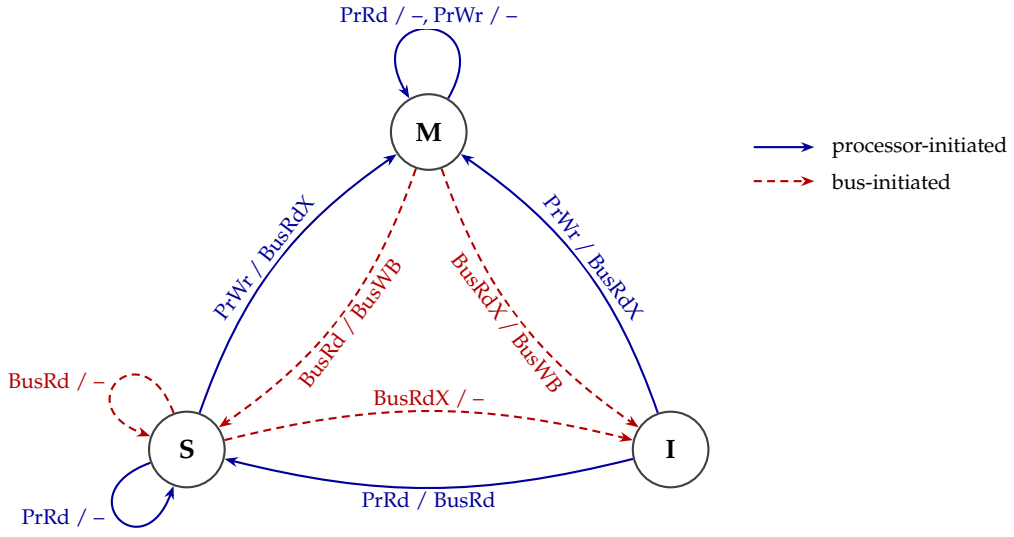


Figure 28: MSI state transitions for one cache line in one cache. An edge labeled A / B fires when the controller observes A and performs B.

A read obtains the line in S, even if it is the only cached copy. A write needs exclusive ownership first: BusRdX causes other caches to invalidate (and a cache holding the line in M to write it back), and a write that hits in S still issues BusRdX to upgrade to M, because other caches may hold the line in S and must invalidate it. When a cache snoops a BusRdX for a line it holds, it must invalidate the line, or the line would no longer be exclusive to the writer. All caches run this logic independently (Figure 28).

Example: *MSI in action.*

Action	P1	P2	P3	Bus	Data from
1. P1 read x	S	–	–	BusRd	memory
2. P3 read x	S	–	S	BusRd	memory
3. P3 write x	I	–	M	BusRdX	memory
4. P1 read x	S	–	S	BusRd	P3's cache
5. P2 read x	S	S	S	BusRd	memory
6. P2 write x	I	M	I	BusRdX	memory

In step 4 the modified line moves to S, and P3 supplies the data (and writes it back) because memory is stale.

MSI satisfies the invariants: only one cache can be in M, all others having been invalidated, while multiple caches may be in the read-only S state (SWMR); and on BusRd and BusRdX the data comes from the cache holding the line in M, while the bus serializes all transactions (write serialization). Communication increases memory latency.

Definition: MSI needs two interconnect transactions for the common case of reading an address and then writing it (BusRd to move from I to S, BusRdX to move from S to M), even when there is no sharing at all. The MESI protocol adds the state EXCLUSIVE CLEAN (E): the line is unmodified, but only this cache has a copy. This decouples exclusivity from ownership (the line is not dirty, so memory holds a valid copy), and the upgrade from E to M needs no bus transaction. On a read miss the line enters E if no other cache asserts that it shares the line, and S otherwise (Figure 29).

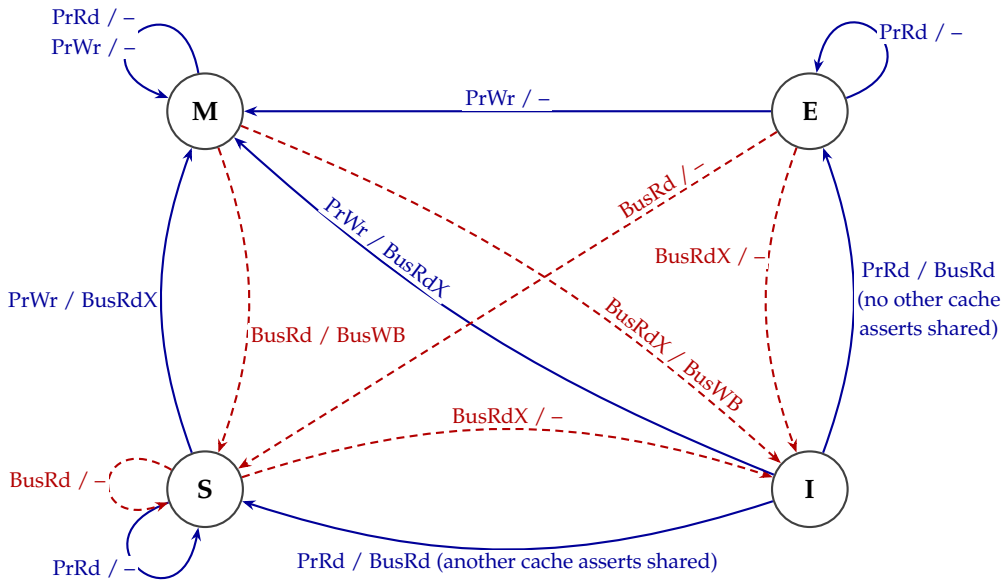


Figure 29: MESI state transitions. Solid edges are processor-initiated, dashed edges bus-initiated.

11.4 Directory-based coherence

Snooping broadcasts coherence messages to learn the state of a line in other caches, which does not scale. A DIRECTORY stores the status of each line in one place: the directory entry for a line records its state in all caches, caches consult the directory as needed, and coherence is maintained by point-to-point messages on a "need to know" basis rather than by broadcast. The SWMR and write-serialization invariants must still hold. In an Intel Core i7, the inclusive L3 cache (any line in an L2 is guaranteed to be in L3) serves as the centralized directory and serialization point for its lines: it keeps the list of L2 caches holding each line ($P = 4$ bits per line for M L3 lines), and coherence messages go only to the L2s that hold the line. The interconnect is a ring, not a bus.

11.5 Implications for the programmer

Communication time is a key parallel overhead: it appears as increased memory access time, with average memory access time

$$\text{AMAT} = \sum \text{frequency of access} \times \text{latency of access}, \qquad \text{AMAT}_{\text{multiprocessor}} > \text{AMAT}_{\text{uniprocessor}},$$

because of higher miss rates, extra main-memory accesses in uniform memory access (UMA) systems, and higher-latency remote accesses in NUMA systems; even a fraction of a percent of these can be significant. Approximate latencies on a Core i7 Xeon 5500:

Data source	Latency
L1 hit	~4 cycles
L2 hit	~10 cycles
L3 hit, line unshared	~40 cycles
L3 hit, shared line in another core	~65 cycles
L3 hit, modified in another core	~75 cycles
Local DRAM	~30 ns (~120 cycles)
Remote DRAM	~100 ns (~400 cycles)

Tools such as VTune report memory-system performance.

Definition: False sharing occurs when processors write to different addresses that map to the same cache line. The line "ping-pongs" between the writers' caches, generating substantial coherence traffic. There is no inherent communication: it is entirely artifactual, caused by cache lines larger than the data each processor writes.

Example: *False sharing.* Threads that each increment their own counter `(*counter)++` many times run with `int counter[MAX_THREADS]`, so neighboring counters share cache lines, or with each counter padded to its own line:

```
struct padded_t {
  int counter;
  char padding[CACHE_LINE_SIZE - sizeof(int)];
};
```

With 8 threads on a 4-core system, the unpadded version takes 14.2 s and the padded one 4.7 s.

Simulations of a 1 MB cache on four applications (Barnes-Hut, Radiosity, Ocean, and Radix sort), with misses classified as cold, capacity/conflict, true sharing, false sharing, and upgrade, show the effect of line size from 8 to 256 bytes: larger lines reduce cold and true-sharing misses through spatial locality, but false-sharing misses grow, and for Radix sort they dominate at 128- and 256-byte lines.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 11 (Fall 2023).

Lecture 12: Memory Consistency

Intuition says a load returns the latest value written, but what is "latest"? Coherence concerns only one memory location; consistency concerns the apparent ordering of operations on all locations, that is, the order in which memory operations performed by one thread become visible to other threads. It affects programmability (how programmers reason about the allowed behavior of multithreaded programs) and performance (it limits the hardware and software optimizations, such as reordering memory operations to hide latency, that can be used). It matters to anyone who implements a synchronization library, works on kernels or drivers, or implements lock-free data structures (Lecture 13).

12.1 Coherence versus consistency

Memory coherence (Lecture 11) defines requirements for the observed behavior of reads and writes to the *same* location: all processors must agree on the order of reads and writes to X, so all operations on X can be placed on one timeline consistent with every processor's observations (P0 writes 5, P1 reads 5, P2 writes 10, P2 writes 11, P1 reads 11). Memory consistency defines the behavior of reads and writes to *different* locations as observed by other processors. Coherence only guarantees that writes to X eventually propagate to other processors; consistency deals with when they propagate relative to reads and writes of other addresses.

Put differently, coherence aims to make the memory system of a parallel computer behave as if the caches were not there, just as in a uniprocessor; a system without caches would not need coherence. A consistency model specifies the allowed behavior of loads and stores to different addresses whether or not caches are present. Multiprocessors reorder memory operations in unintuitive ways, and this is required for performance; application programmers rarely see it, but system (OS and compiler) developers see it all the time.

Definition: A program defines a sequence of loads and stores, its program order. There are four types of memory operation ordering: $W_X \rightarrow R_Y$ (a write to X must commit before a subsequent read from Y), $R_X \rightarrow R_Y$, $R_X \rightarrow W_Y$, and $W_X \rightarrow W_Y$. "A write must commit before a subsequent read" means that when the write precedes the read in program order, its result is visible by the time the read occurs.

Example: *What can be printed?* Initially $A = B = 0$:

Proc 0	Proc 1
`(1) A = 1`	`(3) B = 1`
`(2) print B`	`(4) print A`

Reading the output in the order the prints execute, interleavings that respect program order print

"01" (for example (1)(2)(3)(4)) or "11" (for example (1)(3)(2)(4)). The program should not print "10": if the first print outputs 1, the other processor's write precedes it, and so does that processor's own write, so the second print also outputs 1. Nor should it print "00". A HAPPENS-BEFORE GRAPH shows the order in which events must execute to produce an outcome; a cycle means the outcome is impossible, since an event would have to happen before itself (Figure 30).

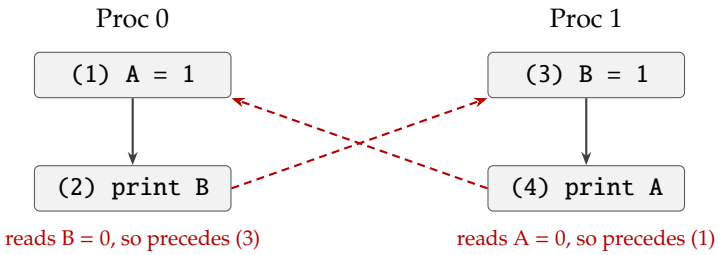


Figure 30: Happens-before graph for printing "00": program order (solid) plus the orders forced by reading the initial values (dashed) form a cycle.

Definition 12.1 [SEQUENTIAL CONSISTENCY]. A memory system is SEQUENTIALLY CONSISTENT (Lamport, 1976) if all operations are executed in some sequential order, as if manipulating a single shared memory, and each thread's operations happen in program order: there is one chronology of all memory operations, to all addresses, consistent with the observed values. A sequentially consistent memory system maintains all four orderings $W_X \to R_Y$, $R_X \to R_Y$, $R_X \to W_Y$, $W_X \to W_Y$.

A metaphor: all processors issue loads and stores in program order, and memory acts as a switch that chooses a processor at random, performs one of its memory operations to completion, then chooses another processor. For Proc 0 running `A=1; r1=B` and Proc 1 running `B=1; r2=A`, one sequentially consistent execution is `A=1`, `B=1`, `r2=A` (reads 1), `r1=B` (reads 1).

12.2 Relaxed consistency

RELAXED MEMORY CONSISTENCY models allow certain orderings to be violated, to gain performance by hiding memory latency: overlapping independent memory operations with other operations. In a cache-coherent system a memory access may entail much more work than reading bits from memory (finding the data, sending invalidations, and so on). Under sequential consistency, Proc 0 must wait for `A=1`, which may take hundreds of cycles, before `r1=B`, although the two do not conflict.

Write buffers. With a WRITE BUFFER, each processor places its writes in its own buffer, from which they drain to the cache and memory later, and its reads proceed without waiting (Figure 31). This changes memory behavior: with initially $A = B = 0$, Proc 0 executing `A=1; r1=B` and Proc 1 executing `B=1; r2=A` can end with $r_1 = r_2 = 0$, impossible under sequential consistency. Relaxing $W \to R$ ordering hides write latency almost fully: normalizing sequentially consistent execution time to 100, the relaxed versions take 71 (MP3D), 95 (LU), and 80 (PTHOR). Every modern processor (Intel x86, ARM, SPARC) uses write buffers, so they need a weaker memory model.

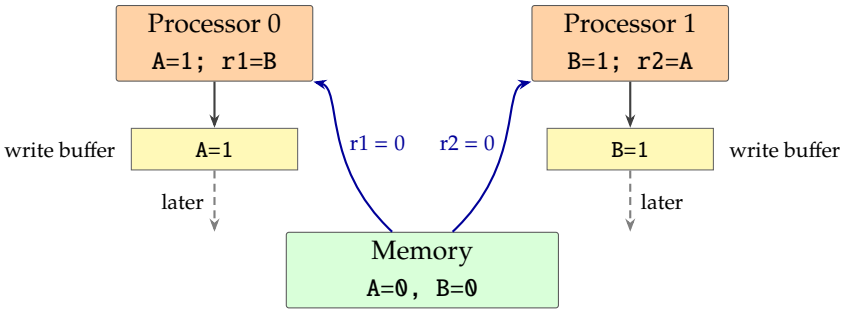


Figure 31: Write buffers let each processor's read bypass its own pending write, so both reads can return 0.

Definition: Allowing reads to move ahead of writes relaxes only $W_X \to R_Y$; the $W_X \to W_Y$ constraint remains, so writes by the same thread occur in program order.

- In TOTAL STORE ORDERING (TSO), a processor may read B before its own write to A is seen by all processors (it can move its own reads in front of its own writes), but other processors cannot read the new value of A until the write is observed by all processors. x86 uses an incompletely specified form of TSO, slightly harder to reason about than sequential consistency.
- In PROCESSOR CONSISTENCY (PC), any processor may read the new value of A before the write is observed by all processors.

Keep the two issues separate. The coherence problem exists because hardware duplicates data in multiple caches, and the copies must be kept coherent. Relaxed consistency arises from the optimization of reordering memory operations, and is unrelated to whether caches exist.

Definition: PARTIAL STORE ORDERING (PSO) also relaxes $W_X \to W_Y$. Then a program may not behave as under sequential consistency: if thread 1 executes `A = 1; flag = 1;` and thread 2 executes `while (flag == 0); print A;`, thread 2 may observe the change to `flag` before the change to `A` and print 0.

More aggressive reorderings are useful because processors reorder writes in a write buffer (for example when one write misses in the cache while another hits), relaxing $W_X \to W_Y$, and reorder independent instructions in out-of-order execution, relaxing $R_X \to W_Y$ and $R_X \to R_Y$. All are valid optimizations for a single instruction stream. Models that allow all reorderings give no ordering guarantees on data at all; they overlap multiple reads and writes in the memory system, executing reads as early and writes as late as possible to hide latency. Examples are WEAK ORDERING (WO) and RELEASE CONSISTENCY (RC).

12.3 Synchronization

Every architecture provides synchronization primitives that make memory ordering stricter. A MEMORY FENCE (memory barrier) instruction prevents reordering across it: all memory operations before the fence complete before any operation after it begins, while operations between fences may

be reordered. Fences are expensive. Other synchronization primitives act per address: read-modify-write and compare-and-swap, transactional memory, and so on. Intel x86/x64, approximately TSO, provides `_mm_lfence` (wait for all loads to complete), `_mm_sfence` (wait for all stores to complete), and `_mm_mfence` (wait for all memory operations to complete) for when software needs an ordering the model does not guarantee. ARM processors have a very relaxed consistency model.

Definition: Two memory accesses by different processors CONFLICT if they access the same location and at least one is a write. A program is UNSYNCHRONIZED if conflicting accesses are not ordered by synchronization (a fence, an operation with release/acquire semantics, a barrier, and so on); such a program contains DATA RACES, and its output depends on the relative speed of the processors.

Every example above involves a data race. SYNCHRONIZED PROGRAMS, which are data-race free, yield sequentially consistent results on non-SC systems: if there are no data races, reordering does not matter, because accesses are ordered by synchronization and synchronization forces sequential consistency. In practice most programs are synchronized through locks, barriers, and similar primitives in synchronization libraries, rather than through ad hoc reads and writes of shared variables. The cost of relaxed consistency is software complexity: the programmer or compiler must insert synchronization where specific orderings are needed, but libraries encapsulate this behind intuitive primitives (lock/unlock, barrier, or lower-level fences). This optimizes for the common case, since most memory accesses do not conflict.

12.4 Language memory models

Compilers reorder memory operations too. A compiler may transform

```
X = 0; for i=0 to 100:  { X = 1; print X }  into  X = 1; for i=0 to 100:  print X
```

which is invisible to a single thread: both print 1111 . . .. But if another thread concurrently executes `X = 0`, the original program prints something like 11110111 . . . (the next iteration restores $X = 1$), while the transformed one prints 1111000 . . .. Languages must therefore provide a contract about how memory operations may be reordered by the compiler. Modern (C11, C++11) and older (Java 5) languages guarantee sequential consistency for data-race-free programs ("SC for DRF"), with compilers inserting the synchronization needed for the hardware memory model, and give no guarantees for programs with data races, which most programmers consider buggy anyway. The advice is to use a synchronization library.

A memory consistency model thus defines the reorderings of memory operations allowed to hardware and compilers, a contract between them and application software. Weak models are used for performance, although sequential consistency can perform well with many more hardware resources, and the details of the model can be hidden in a synchronization library for data-race-free programs.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 12 (Fall 2023).

Lecture 13: Fine-Grained Synchronization and Lock-Free Programming

This unit covers how locks are implemented, how to use them with fine granularity, and lock-free data structure designs.

13.1 Deadlock, livelock, and starvation

Definition: Deadlock is a state in which a system has outstanding operations to complete but no operation can make progress. It can arise when each operation has acquired a shared resource that another needs, and no thread can progress unless some thread relinquishes a resource (as in a traffic gridlock where no car can move until one backs up). Livelock is a state in which the system executes many operations but no thread makes meaningful progress, for example operations that continually abort and retry. Starvation is a state in which the system makes overall progress but some processes make none, as when cars that must yield to cross traffic wait while that traffic keeps coming; it is usually not permanent. Deadlock and livelock concern correctness; starvation is a matter of fairness.

Examples of deadlock in computer systems: thread A produces work for B's work queue and B for A's, with finite queues and workers that wait when there is no output space, so two full queues deadlock; or every thread first sends a message with a blocking send to the thread with the next higher id and then receives from the next lower id (as in Lecture 6). Deadlock requires four conditions:

1. *mutual exclusion*: only one processor can hold a given resource at once;
2. *hold and wait*: a processor holds a resource while waiting for other resources it needs to complete an operation;
3. *no preemption*: processors do not give up resources until the operation they wish to perform is complete;
4. *circular wait*: waiting processors have mutual dependencies (a cycle exists in the resource dependency graph).

Example: *MSI in practice.* Addresses X and Y lie on different cache lines and hold 0 initially. Under MSI (where flushing a dirty line to memory is the BusWB of Lecture 11):

Operation	Cache 0	Cache 1
P0: LD X	BusRd, load X in S	observe BusRd, nothing (I)
P0: LD X	hit	nothing
P0: ST X ← 1	BusRdX, load X in M	observe BusRdX, nothing (I)
P0: ST X ← 2	hit	nothing
P1: ST X ← 3	observe BusRdX, flush X, move to I	BusRdX, load X in M
P1: LD X	nothing	hit
P0: LD X	BusRd, load X in S	observe BusRd, flush X, move to S
P0: ST X ← 4	BusRdX, load X in M	observe BusRdX, move to I
P1: LD X	observe BusRd, flush X, move to S	BusRd, load X in S
P0: LD Y	BusRd, load Y in S	observe BusRd, nothing (I)
P0: ST Y ← 1	BusRdX, load Y in M	observe BusRdX, nothing (I)
P1: ST Y ← 2	observe BusRdX, flush Y, move to I	BusRdX, load Y in M

13.2 Implementing locks

Definition: The atomic TEST-AND-SET instruction `ts R0, mem[addr]` loads `mem[addr]` into `R0` and, if `mem[addr]` is 0, sets it to 1. A TEST-AND-SET LOCK is

```
lock:    ts R0, mem[addr]   // load word into R0
         bnz R0, lock       // if 0, lock obtained
unlock:  st mem[addr], #0   // store 0 to address
```

x86 provides compare-and-exchange, atomic with the `lock` prefix: `lock cmpxchg dst, src` (where `dst` is often a memory address and `src` a register) checks whether `dst` equals register `EAX`; if so it sets the zero flag `ZF` = 1 and writes `dst = src`, and otherwise it sets `ZF` = 0 and loads `EAX = dst`.

Coherence traffic. Each test-and-set is a write, so it issues BusRdX and invalidates the line in all other caches. While P1 holds the lock, P2 and P3 repeatedly execute test-and-set: each attempt issues BusRdX, takes the line away (invalidating it in the other caches, including P1's), and fails. To release, P1 must issue BusRdX to set the line to 0, and then another processor's test-and-set succeeds. P1 holds the lock from its successful test-and-set until its release store, but its cache holds a valid copy of the line only briefly, between its own BusRdX and the next attempt by another processor. Measured on N lock/unlock pairs spread over P processors (with the critical-section time removed), the time per lock transfer grows rapidly with the number of processors, because interconnect contention delays the transfer (the holder must wait for the bus to release the lock); contention also slows the critical section itself.

Desirable lock characteristics are:

- *low latency*: a free, uncontended lock is acquired quickly;
- *low interconnect traffic*: when all processors try to acquire the lock at once, they acquire it in succession with as little traffic as possible;
- *scalability*: latency and traffic scale reasonably with the number of processors;
- *low storage cost*;
- *fairness*: avoid starvation or substantial unfairness, ideally granting the lock in the order requested.

The simple test-and-set lock has low latency under low contention, high traffic, poor scaling, low storage cost (one int), and no provision for fairness.

Definition: A TEST-AND-TEST-AND-SET LOCK spins reading the lock and attempts test-and-set only when the lock appears free:

```
1  void Lock(int* lock) {
2    while (1) {
3      while (*lock != 0);              // while another processor has the lock...
4                                       // (assume *lock is NOT register allocated)
5      if (test_and_set(*lock) == 0)    // when lock is released, try to acquire it
6        return;
7    }
8  }

9  void Unlock(int* lock) {
10   *lock = 0;
11 }
```

While P1 holds the lock, the waiters issue one BusRd each and then spin on reads from their local caches with no interconnect traffic. The release store invalidates their copies; they re-read the line and race to test-and-set, and the losers' test-and-set attempts fail. Latency is slightly higher than test-and-set without contention (test, then test-and-set), but traffic is much lower: one invalidation per waiting processor per lock release, $O(P)$ invalidations per release and $O(P^2)$ interconnect traffic if all processors have the lock cached, instead of one invalidation per waiting processor per test. It is more scalable, storage is unchanged (one int), and there is still no fairness.

Definition: The main remaining problem is that on release, all waiting processors attempt test-and-set. A TICKET LOCK avoids this:

```
struct lock {
  int next_ticket;
  int now_serving;
};

void Lock(lock* l) {
  int my_ticket = atomic_increment(&l->next_ticket);  // take a "ticket"
  while (my_ticket != l->now_serving);                // wait for number to be called
}

void unlock(lock* l) {
  l->now_serving++;
}
```

Waiting needs no atomic operation, only reads, so each release causes one invalidation ($O(P)$ traffic), and processors acquire the lock in the order they requested it.

Atomic operations. CUDA provides `atomicAdd`, `atomicSub`, `atomicExch`, `atomicMin`, `atomicMax`, `atomicInc`, `atomicDec`, `atomicCAS`, and the bitwise `atomicAnd`, `atomicOr`, `atomicXor` (on `int`, some on `float`, and 64-bit and unsigned variants). Compare-and-swap performs atomically

$$old \leftarrow *addr; \quad *addr \leftarrow (old = compare)\ ?\ val : old; \quad \text{return } old,$$

and any atomic fetch-and-op can be built from it by retrying until no other thread intervened:

```
void atomic_min(int* addr, int x) {
  int old = *addr;
  int new = min(old, x);
  while (atomicCAS(addr, old, new) != old) {
    old = *addr;
    new = min(old, x);
  }
}
```

A lock is `while (atomicCAS(l, 0, 1) == 1);` with unlock `*l = 0`. Under contention it is more efficient to spin on reads first, `while (*l == 1);` and only then try `atomicCAS(l, 0, 1)`, for the same reason as test-and-test-and-set.

Definition: Load-linked/store-conditional (LL/SC) is a pair of instructions rather than a single atomic one: `load_linked(x)` loads a value, and `store_conditional(x, value)` stores to `x` only if no processor has written `x` since the corresponding load-linked (ARM's `LDREX` and `STREX`). On a cache-coherent processor it can be implemented by watching for an invalidation of the line between the two instructions.

C++11's `atomic<T>` provides atomic read, write, and read-modify-write of entire objects, imple-

mented with a mutex or, for basic types, with processor atomic instructions. It also specifies memory-ordering semantics for operations before and after the atomic operation, sequential consistency by default (see `std::memory_order`). For `atomic<int> i`, `i++` increments atomically, `i.compare_exchange_strong(a, 10)` sets `i` to 10 if it equals `a`, and `i.is_lock_free()` reports whether the implementation is lock free.

13.3 Fine-grained locking

Consider a sorted linked list:

```
struct Node {
  int value;
  Node* next;
};

struct List {
  Node* head;
};

void insert(List* list, int value) {
  Node* n = new Node;
  n->value = value;

  // assume case of inserting before head of
  // of list is handled here (to keep the code simple)
  Node* prev = list->head;
  Node* cur = list->head->next;

  while (cur) {
    if (cur->value > value)
      break;
    prev = cur;
    cur = cur->next;
  }

  n->next = cur;
  prev->next = n;
}

void delete(List* list, int value) {
  // assume case of deleting first node in list
  // is handled here (to keep the code simple)
  Node* prev = list->head;
  Node* cur = list->head->next;
```

```
  while (cur) {
    if (cur->value == value) {
      prev->next = cur->next;
      delete cur;
      return;
    }
    prev = cur;
    cur = cur->next;
  }
}
```

With the list 3, 5, 10, 11, 18, if thread 1 inserts 6 while thread 2 inserts 7, both compute the same `prev` (node 5) and `cur` (node 10), and one insertion is lost. If thread 1 inserts 6 while thread 2 deletes 10, thread 1 may link 6 to the deleted node 10, losing the insertion.

Solution 1: a single lock. Adding a `Lock lock` to `List` and holding it for the whole of each operation is simple and correct, but serializes all operations on the data structure, which may limit parallel performance.

Solution 2: hand-over-hand locking. Give each node a lock and traverse "hand over hand", always holding the locks of `prev` and `cur`, acquiring the next node's lock before releasing the oldest. Thread 0 deleting 11 and thread 1 deleting 10 can then proceed in different parts of the list; thread 1 cannot overtake thread 0.

```
struct Node {
  int value;
  Node* next;
  Lock* lock;
};

struct List {
  Node* head;
  Lock* lock;
};

void insert(List* list, int value) {
  Node* n = new Node;
  n->value = value;

  // assume case of insert before head handled
  // here (to keep the code simple)
```

```
15    Node* prev, *cur;

16    lock(list->lock);
17    prev = list->head;
18    lock(prev->lock);
19    unlock(list->lock);
20    cur = prev->next;
21    if (cur) lock(cur->lock);

22    while (cur) {
23      if (cur->value > value)
24        break;

25      Node* old_prev = prev;
26      prev = cur;
27      cur = cur->next;
28      unlock(old_prev->lock);
29      if (cur) lock(cur->lock);
30    }

31    n->next = cur;
32    prev->next = n;

33    unlock(prev->lock);
34    if (cur) unlock(cur->lock);
35  }

36  void delete(List* list, int value) {
37    // assume case of delete head handled here
38    // (to keep the code simple)

39    Node* prev, *cur;

40    lock(list->lock);
41    prev = list->head;
42    lock(prev->lock);
43    unlock(list->lock);
44    cur = prev->next;
45    if (cur) lock(cur->lock);

46    while (cur) {
47      if (cur->value == value) {
48        prev->next = cur->next;
49        unlock(prev->lock);
50        unlock(cur->lock);
51        delete cur;
52        return;
```

```
    }

    Node* old_prev = prev;
    prev = cur;
    cur = cur->next;
    unlock(old_prev->lock);
    if (cur) lock(cur->lock);
  }
  unlock(prev->lock);
}
```

Fine-grained locking enables parallelism in data structure operations and reduces contention for a global lock, which is overly conservative when operations touch different parts of the list. The code is deadlock free because every thread acquires locks in the same order, along the list, so no circular wait can arise. But correctness is tricky (when is mutual exclusion required? could there be deadlock or livelock?), and there are costs: taking a lock at each traversal step (extra instructions, and traversal now writes memory) and a lock per node of storage. A middle ground trades some parallelism for lower overhead, for example one lock per group of nodes, much like choosing task granularity. A similar exercise is a fine-grained locked binary search tree with insert and delete.

13.4 Lock-free data structures

Definition: A BLOCKING algorithm allows one thread to prevent other threads from completing operations on a shared data structure indefinitely: if thread 0 holds a lock on a list node and is swapped out by the OS, crashes, or is just slow (takes a page fault), no other thread can complete its operation. Any algorithm using locks is blocking, whether the lock implementation spins or uses preemption. A non-blocking algorithm is LOCK FREE if some thread is guaranteed to make progress ("system-wide progress"): preempting one thread at an inopportune time cannot prevent progress by the rest of the system. This does not prevent starvation of any one thread.

In the following, assume a sequentially consistent memory system (or appropriate memory fences, or C++11 `atomic<>`).

Single-reader, single-writer bounded queue. With exactly one producer and one consumer, the threads never synchronize or wait on each other except when the queue is empty (pop fails) or full (push fails):

```
struct Queue {
  int data[N];
  int head;  // head of queue
  int tail;  // next free element
```

```
};

void init(Queue* q) {
  q->head = q->tail = 0;
}

// return false if queue is full
bool push(Queue* q, int value) {
  // queue is full if tail is element before head
  if (q->tail == MOD_N(q->head - 1))
    return false;
  q->data[q->tail] = value;
  q->tail = MOD_N(q->tail + 1);
  return true;
}

// returns false if queue is empty
bool pop(Queue* q, int* value) {
  // if not empty
  if (q->head != q->tail) {
    *value = q->data[q->head];
    q->head = MOD_N(q->head + 1);
    return true;
  }
  return false;
}
```

Single-reader, single-writer unbounded queue. A linked version keeps `tail` pointing to the last element added (if non-empty), `head` pointing to the element *before* the head of the queue, and a `reclaim` pointer; all node allocation and deletion is done by the producer thread:

```
struct Node {
  Node* next;
  int value;
};

struct Queue {
  Node* head;
  Node* tail;
  Node* reclaim;
};

void init(Queue* q) {
  q->head = q->tail = q->reclaim = new Node;
```

```
}

void push(Queue* q, int value) {
  Node* n = new Node;
  n->next = NULL;
  n->value = value;

  q->tail->next = n;
  q->tail = q->tail->next;

  while (q->reclaim != q->head) {
    Node* tmp = q->reclaim;
    q->reclaim = q->reclaim->next;
    delete tmp;
  }
}

// returns false if queue is empty
bool pop(Queue* q, int* value) {
  if (q->head != q->tail) {
    *value = q->head->next->value;
    q->head = q->head->next;
    return true;
  }
  return false;
}
```

For example, after push 3 and push 10, two pops return 3 and 10 and advance `head` to the node holding 10 (now equal to `tail`), and a third pop returns false. The next push of 5 appends a node and reclaims the dummy node and the node holding 3, which the consumer has passed.

Lock-free stack.

```
struct Node {
  Node* next;
  int value;
};

struct Stack {
  Node* top;
};

void init(Stack* s) {
  s->top = NULL;
```

```
}

void push(Stack* s, Node* n) {
  while (1) {
    Node* old_top = s->top;
    n->next = old_top;
    if (compare_and_swap(&s->top, old_top, n) == old_top)
      return;
  }
}

Node* pop(Stack* s) {
  while (1) {
    Node* old_top = s->top;
    if (old_top == NULL)
      return NULL;
    Node* new_top = old_top->next;
    if (compare_and_swap(&s->top, old_top, new_top) == old_top)
      return old_top;
  }
}
```

As long as no other thread has modified the stack, a thread's modification proceeds. Unlike fine-grained locking, which locks part of the data structure, threads hold no lock at all.

Definition: The ABA PROBLEM: compare-and-swap checks only that a value is unchanged, not that nothing happened. In the stack, thread 0 begins a pop with `old_top` = A and `new_top` = B; meanwhile thread 1 pops A, pushes D, modifies node A, and pushes A again. Thread 0's CAS then succeeds, because `top` is A again, and sets `top` to B: D is lost and the stack is corrupted (Figure 32; A, B, C, D are node addresses, not values).

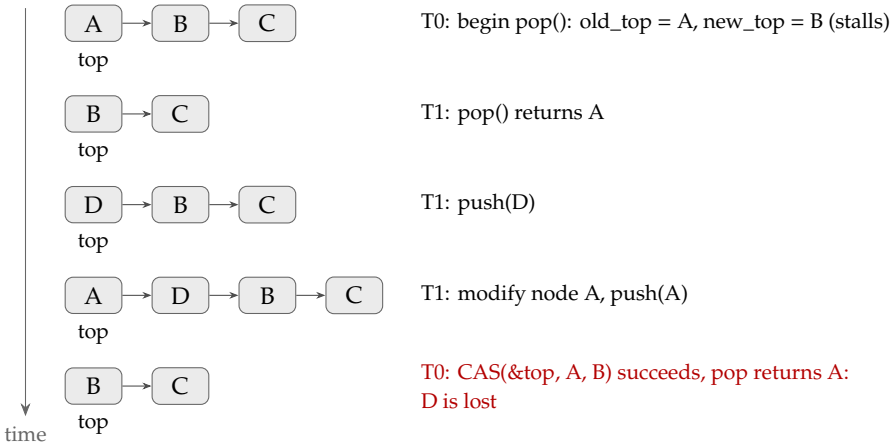


Figure 32: The ABA problem in the lock-free stack.

One solution keeps a counter of pop operations in the stack (`int pop_count`) and pops with a DOUBLE COMPARE-AND-SWAP that succeeds only if neither `top` nor `pop_count` changed, incrementing the count:

```
Node* pop(Stack* s) {
  while (1) {
    int pop_count = s->pop_count;
    Node* top = s->top;
    if (top == NULL)
      return NULL;
    Node* new_top = top->next;
    if (double_compare_and_swap(&s->top,       top,       new_top,
                                &s->pop_count, pop_count, pop_count+1))
      return top;
  }
}
```

This requires machine support for double compare-and-swap (DCAS) or a doubleword CAS; ABA can also be solved by careful node allocation or element reuse policies. x86 provides a double-wide compare-and-swap, which is not quite DCAS but suffices if the stack's `top` and `pop_count` are contiguous in memory: `cmpxchg8b` (two 32-bit values) and `cmpxchg16b` (two 64-bit values).

Referencing freed memory. If `pop` also frees the popped node and returns its value, another thread may still be reading `old.top->next` of a node that has been freed by the thread that popped it. A HAZARD POINTER avoids freeing a node until no other thread holds a reference to it: each thread publishes in a per-thread `hazard` pointer the node it is accessing (set to `s->top` when reading the top, and cleared afterward), and instead of deleting a popped node it *retires* it onto a per-thread list. When the list exceeds a threshold, the thread deletes each retired node not pointed to by any thread's hazard pointer.

Lock-free linked list. Insertion after a given node, assuming insertion is the only operation, retries a compare-and-swap on `prev->next`:

```
// insert new node after specified node
void insert_after(List* list, Node* after, int value) {
  Node* n = new Node;
  n->value = value;

  // assume case of insert into empty list handled
  // here (to keep the code simple)

  Node* prev = list->head;
  while (prev->next) {
```

```
    if (prev == after) {
      while (1) {
        Node* old_next = prev->next;
        n->next = old_next;
        if (compare_and_swap(&prev->next, old_next, n) == old_next)
          return;
      }
    }
    prev = prev->next;
  }
}
```

Compared with fine-grained locking there is no overhead of taking locks and no per-node storage. Supporting deletion significantly complicates the structure: if B is deleted (a CAS on `A->next`) while E is inserted after B (a CAS on `B->next`), both CASes succeed, and B points to E although B is no longer in the list (see Harris 2001; Fomitchev 2004).

Measured lock-free implementations of a queue, a deque, and a linked list (Hunt 2011), normalized to versions using pthread mutexes, are sometimes faster and sometimes slower depending on the thread count and the mix of operations. When a program has the machine to itself, as is typical in scientific computing, graphics, machine learning, and data analytics, well-written code with locks can be as fast as, or faster than, lock-free code. But locks can cause tricky performance problems when a program has many threads (databases, web servers) and page faults or preemption occur inside critical sections: priority inversion, convoying, and crashing in a critical section.

Fine-grained locking reduces contention but increases code complexity and execution overhead; lock-free structures avoid lock overheads but are tricky to implement correctly (and correctness has its own overheads), and they still need appropriate memory fences on relaxed hardware. Lock freedom does not eliminate contention: compare-and-swap can fail repeatedly under heavy contention. In these designs the CAS determines whether another thread modified the data structure while the calling thread was in the middle of an operation; transactional memory (Lectures 15 and 16) generalizes this, speculating that an operation will complete before another thread modifies the structure and aborting it if one does.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 13 (Fall 2023).

Lecture 14: Domain-Specific Programming Languages

Programmers who can optimize performance in C++ with threads, ISPC, or CUDA are hard to find, and that work has low productivity. The ideal parallel programming language would combine performance, productivity, and generality; popular languages achieve at most two of the three. The way forward is domain-specific languages, which give up generality to obtain performance on heterogeneous parallel machines and productivity. The DSL hypothesis: it is possible to write one program and run it efficiently on a range of heterogeneous parallel systems.

Definition: A domain-specific language (DSL) is a programming language with restricted expressiveness for a particular domain, typically high level, declarative, and deterministic. It raises the level of abstraction with primitives specific to the domain. These are *productive*: intuitive to use, portable across machines, and matching behaviors frequently used to solve problems in the domain. They are also *performant*: the system uses domain knowledge to provide efficient, optimized implementations, knowing for a given machine which algorithms and parallelization strategies to use. Optimization can go beyond mapping software onto hardware, since the hardware itself can be optimized for the abstractions. The cost is a loss of generality and completeness.

14.1 Optimizing image processing by hand

Halide (Ragan-Kelley, Adams et al., SIGGRAPH 2012, PLDI 2013) is a DSL for image processing, used to implement camera pipelines on Google phones (HDR+, aspects of portrait mode) and in industry at Instagram, Adobe, and elsewhere.

Consider first a 3×3 box blur in C:

```
int WIDTH = 1024;
int HEIGHT = 1024;
float input[(WIDTH+2) * (HEIGHT+2)];
float output[WIDTH * HEIGHT];

float weights[] = {1.f/9, 1.f/9, 1.f/9,
                   1.f/9, 1.f/9, 1.f/9,
                   1.f/9, 1.f/9, 1.f/9};

for (int j=0; j<HEIGHT; j++) {
  for (int i=0; i<WIDTH; i++) {
    float tmp = 0.f;
    for (int jj=0; jj<3; jj++)
      for (int ii=0; ii<3; ii++)
        tmp += input[(j+jj)*(WIDTH+2) + (i+ii)] * weights[jj*3 + ii];
    output[j*WIDTH + i] = tmp;
  }
}
```

Its total work is $9 \times \mathit{WIDTH} \times \mathit{HEIGHT}$, and $N^2 \times \mathit{WIDTH} \times \mathit{HEIGHT}$ for an $N \times N$ filter. A 2D separable filter such as a box filter can instead be evaluated as two 1D filters, horizontal then vertical:

```
int WIDTH = 1024;
int HEIGHT = 1024;
float input[(WIDTH+2) * (HEIGHT+2)];
float tmp_buf[WIDTH * (HEIGHT+2)];
float output[WIDTH * HEIGHT];

float weights[] = {1.f/3, 1.f/3, 1.f/3};

for (int j=0; j<(HEIGHT+2); j++)
  for (int i=0; i<WIDTH; i++) {
    float tmp = 0.f;
    for (int ii=0; ii<3; ii++)
      tmp += input[j*(WIDTH+2) + i+ii] * weights[ii];
    tmp_buf[j*WIDTH + i] = tmp;
  }

for (int j=0; j<HEIGHT; j++) {
  for (int i=0; i<WIDTH; i++) {
    float tmp = 0.f;
    for (int jj=0; jj<3; jj++)
      tmp += tmp_buf[(j+jj)*WIDTH + i] * weights[jj];
    output[j*WIDTH + i] = tmp;
  }
}
```

This costs $6 \times WIDTH \times HEIGHT$ work ($2N$ per pixel for an $N \times N$ filter) but needs $WIDTH \times HEIGHT$ extra storage and has half the arithmetic intensity of the 2D blur, because every intermediate value is written to and read back from `tmp_buf`.

Locality. The intrinsic bandwidth requirement of the blur is to read each input element and write each output element once. In the first pass, input data is reused three times, immediately in the next two iterations of the i loop and never again: perfect cache behavior and perfect use of cache lines. The loads and stores of `tmp_buf` are overhead, an artifact of the two-pass implementation rather than intrinsic to the computation. In the second pass, each value of `tmp_buf` is also reused three times, but three rows of the image are accessed in between, so data is never loaded twice only if the cache can hold three rows.

Chunking. Version 1 allocates only three rows of `tmp_buf`: for each output row, it produces the three rows of horizontal blur that row needs and combines them, so loads of `tmp_buf` hit in cache. But each output row costs $3 \times 3 \times WIDTH$ work for step 1 plus $3 \times WIDTH$ for step 2, $12 \times WIDTH \times HEIGHT$ in total, more than the 2D blur, because each horizontal row is recomputed three times. Version 2 sizes `tmp_buf` to $WIDTH \times (CHUNK_SIZE + 2)$ rows so the whole buffer fits in cache (capturing all producer-consumer locality) and produces $CHUNK_SIZE$ output rows at a

time. With *CHUNK_SIZE* = 16, a chunk costs $18 \times 3 \times WIDTH$ for step 1 and $16 \times 3 \times WIDTH$ for step 2, so the image costs $(34/16) \times 3 \times WIDTH \times HEIGHT \approx 6.4 \times WIDTH \times HEIGHT$, tending to the ideal 6 as the chunk grows.

The loops are still neither parallelized across cores nor vectorized, and other basic optimizations (loop unrolling and so on) remain. A fully optimized C++ version, partitioning the image vertically across cores, iterating in 256×32 tiles to maximize cache hits, using SIMD intrinsics, and fusing the two passes so intermediate data is read from cache, is about 10× faster on a quad-core CPU than the original two-pass code. But it is specific to SSE (not AVX2), runs only on CPUs, and is very hard to read.

14.2 Halide

Halide is a simple DSL embedded in C++ for describing sequences of image processing operations:

```
Var x, y;
Func blurx, blury, bright, out;
Halide::Buffer<uint8_t> in = load_image("myimage.jpg");
Halide::Buffer<uint8_t> lookup = load_image("s_curve.jpg");  // 255-pixel 1D image

// perform 3x3 box blur in two-passes
blurx(x,y) = 1/3.f * (in(x-1,y) + in(x,y) + in(x+1,y));
blury(x,y) = 1/3.f * (blurx(x,y-1) + blurx(x,y) + blurx(x,y+1));

// brighten blurred result by 25%, then clamp
bright(x,y) = min(blury(x,y) * 1.25f, 255);

// access lookup table to contrast enhance
out(x,y) = lookup(bright(x,y));

// execute pipeline to materialize values of out in range (0:1024,0:1024)
Halide::Buffer<uint8_t> result = out.realize(1024, 1024);
```

Definition: A Halide FUNCTION is an infinite (but discrete) set of values defined on an N-D domain, mapping integer coordinates to values (such as pixel colors). A Halide EXPRESSION is a side-effect-free expression describing how to compute a function's value at a point in terms of the values of other functions. An application is a DAG of functions (here `in` and `lookup` feed `blurx`, `blury`, `bright`, and `out`).

The representation adopts a local, "pointwise" view of algorithms. It is declarative: it does not define the order of iteration or which values of the domain are stored, only what is needed to compute them, and iteration over the domain is implicit (no explicit loops). Real pipelines are long: 2 functions for the two-pass blur, 9 for unsharp mask, 13 for Harris corner detection, 30 for camera RAW processing, 103 for a local Laplacian filter, 64 for VGG-16 evaluation, and over 2000 in

Google's HDR+ pipeline. One serial implementation of the two-function blur allocates `blurx` as a $1024 \times (1024 + 2)$ buffer, computes all of it, and then computes all of `out`.

Choosing the right representation is key to any system. Good representations are productive, embodying the natural way of thinking about a problem, and they let the system provide useful services: guarantees (correctness, resource bounds, conservation of quantities, type checking) and performance (parallelization, vectorization, use of specialized hardware). The job then becomes generating an efficient implementation of a specific Halide program.

Schedules. Halide adds a second representation, the SCHEDULE, which says how to compute the algorithm:

```
// the "algorithm description" (declaration of what to do)
blurx(x,y) = (in(x-1, y) + in(x,y) + in(x+1,y)) / 3.0f;
out(x,y)   = (blurx(x,y-1) + blurx(x,y) + blurx(x,y+1)) / 3.0f;

// "the schedule" (how to do it)
out.tile(x, y, xi, yi, 256, 32).vectorize(xi,8).parallel(y);
blurx.compute_at(out, x).vectorize(x, 8);
```

This evaluates `out` in a 2D tiled order (loops named `x`, `y`, `xi`, `yi`) with 256×32 tiles, vectorizes the `xi` loop 8 wide, parallelizes the `y` loop with threads, and produces the elements of `blurx` on demand for each tile of output, vectorizing its innermost loop. Scheduling primitives let the programmer sketch at a high level how to map the algorithm onto a parallel machine, while the compiler emits the low-level, platform-specific code.

Iteration primitives specify both the order of a loop nest and how to parallelize it. Given `out.tile(x, y, xi, yi, 256, 32).vectorize(xi,8).parallel(y)`, the compiler generates loops over tiles in `y` (parallelized with threads) and `x`, and within each tile over `yi` from 0 to 32 and `xi` from 0 to 256 by 8 (vectorized with SIMD instructions), computing `out(x*256+xi, y*32+yi)`.

Producer-consumer interleaving primitives say where a producer is computed relative to its consumer's loop nest:

- `blurx.compute_root()`: allocate a buffer for all of `blurx`, compute all of it, then compute all of `out`;
- `blurx.compute_at(out, xi)`: inside `out`'s `xi` loop, allocate a 3-element buffer and compute the three elements of `blurx` needed by that output pixel (the compiler analyzes that each iteration needs 3 elements), which maximizes locality but recomputes each `blurx` value three times;
- `blurx.compute_at(out, x)`: inside the tile loop, allocate a 258×34 buffer, compute the tile of `blurx` needed for one 256×32 tile of output, and then consume it.

With the full schedule above, the generated code loops over tiles in parallel; for each tile it allocates the 258×34 `blurx` buffer, computes it with 8-wide SIMD instructions (generating boundary code, since 258 is not a multiple of 8), and then computes the 256×32 outputs with 8-wide SIMD instructions.

The philosophy of Halide: the programmer describes the image processing algorithm, and also knows how to schedule it efficiently (which is slow and tedious by hand), so Halide gives a language for high-level scheduling decisions: loop structure, unrolling, vectorization, and multi-core parallelization. The compiler is not smart; it mechanically carries out the schedule using the mechanisms of the target machine (pthreads, AVX intrinsics, and so on). To enable these services the language is constrained to computation on regular N-D domains, to feed-forward pipelines (with special support for reductions and fixed recursion depth), and to dependencies that the compiler can infer.

In the initial results, a camera RAW pipeline that took 463 lines of hand-tuned ARM NEON assembly needed 2.75× less code in Halide and ran 5% faster; a bilateral filter of 122 lines of C++ became 34 lines of algorithm plus 6 lines of schedule, 5.9× faster on a CPU, and on a GPU 2× faster than hand-written CUDA. Halide helps expert developers optimize image processing code rapidly: it does not decide how to optimize a program, but lets a programmer with strong optimization knowledge express the strategy, and the compiler handles the details of mapping it to a machine. Very few programmers turned out to write good schedules (of 80+ programmers writing Halide at Google, very few were trusted with schedules), so later work automatically generates schedules by analyzing the program (Adams et al., 2019); it is now hard to write a schedule by hand that beats the autoscheduler for image processing applications. Already in early results (Mullapudi 2016), the automatically generated schedules for non-local means denoising and lens blur outperformed those that two experts wrote over up to two hours, although for a max filter one expert surpassed the autoscheduler after about 25 minutes. Darkroom, Rigel, and Aetherling go further, synthesizing ASIC or FPGA line-buffered pipelines, modeled on image signal processors, directly from a Halide-like description, for very high-efficiency image processing.

Other domain-specific systems include frameworks for data-parallel big-data computation on distributed systems (MapReduce, Spark), graph processing (GraphLab, Ligra), deep neural network definition and training, real-time 3D graphics, numerical computing, the model-view-controller paradigm for web applications, physical simulation (Simit, Ebb), and nonlinear least squares optimization (Opt). Modern machines are parallel and heterogeneous, the only way to increase compute capability in an energy-constrained world, but most software uses a small fraction of their peak; tuning is very hard and does not port across machines. Domain-specific programming environments trade generality for productivity, performance, and portability, exploiting explicit dependencies, domain restrictions, and domain knowledge to synthesize efficient implementations.

14.3 Liszt

Liszt (DeVito et al., 2011) is a language for solving PDEs on meshes. A Liszt program runs on a mesh and computes the values of fields defined on mesh faces, edges, or vertices, for example heat conduction:

```
var i = 0;
while ( i < 1000 ) {
  Flux(vertices(mesh)) = 0.f;
  JacobiStep(vertices(mesh)) = 0.f;
  for (e <- edges(mesh)) {
    val v1 = head(e)
    val v2 = tail(e)
    val dP = Position(v1) - Position(v2)
    val dT = Temperature(v1) - Temperature(v2)
    val step = 1.0f/(length(dP))
    Flux(v1) += dT*step
    Flux(v2) -= dT*step
    JacobiStep(v1) += step
    JacobiStep(v2) += step
  }
  i += 1
}
```

Fields (`Flux`, `Position`, `Temperature`) hold values at mesh elements; topology functions (`head`, `tail`, `vertices`, `edges`) navigate the mesh; and the `for` loop runs independently for each edge, whose body reads and modifies field values at the adjacent vertices. A Liszt program describes operations on fields of an abstract mesh: the application specifies the mesh type (regular or irregular) and topology, but Liszt chooses the mesh representation based on the mesh type, program behavior, and target machine. The compiler decides the data structure from the operations the code performs.

Parallelizing a program requires identifying parallelism (the absence of dependencies), identifying data locality (partitioning data by its dependencies), and reasoning about synchronization (waiting for the values a computation depends on). For general programs compilers cannot infer dependencies at global scale: in `a[f(i)] += b[i];` the compiler must execute `f(i)` to know whether iterations depend on each other.

Definition: Liszt is constrained to allow dependency analysis. It statically analyzes each top-level `for` loop to infer its STENCIL, the mesh elements accessed in an iteration, which are the iteration's dependencies. It extracts the nested mesh-element reads and the operations on data at those elements. In the heat-conduction loop, edge e reads `head(e)` and `tail(e)`, reading Position and Temperature and writing Flux and JacobiStep there; for example, edge 6's read stencil is vertices D and F.

The compiler uses the dependencies to implement different parallel strategies on different machines.

- *Mesh partitioning* for a cluster with a distributed address space: the mesh and stencil define a graph, which is partitioned (for example by ParMETIS) to store a region of the mesh on each node. Each node also needs the data of neighboring cells, its GHOST CELLS (owned by other nodes); Liszt allocates ghost storage and emits the communication needed to implement the topological operators. Since many operators return sets ("all edges of this face"), elements are always accessed relative to an input vertex, edge, or face.
- *Mesh coloring* for a GPU with a single address space and many tiny threads: the natural parallelization is one edge per CUDA thread, but edges that share a vertex update the same per-vertex field values, which would require atomic updates. Instead, the compiler builds a CONFLICT GRAPH of edges with colliding writes (one run of the program suffices, and the result stays valid while the mesh does not change) and colors it so that no two connected edges share a color. All edges of one color run in a single CUDA launch, in parallel, without atomics (Figure 33).

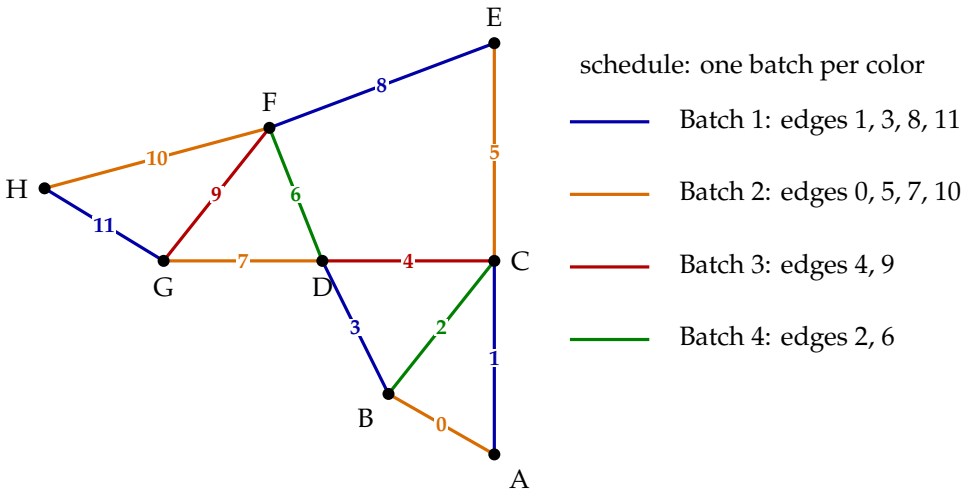


Figure 33: Mesh coloring for the GPU. Edges sharing a vertex conflict; each color is a batch of edges with no shared vertices, run as one CUDA launch without atomics.

On a cluster of 256 nodes with 8 cores each (message passing), Liszt programs for Euler (23M-cell mesh) and Navier-Stokes (21M-cell mesh) scale to 1024 cores as well as hand-written C++ versions (slightly better at 1024 cores), and the same program also runs efficiently on a GPU using a different algorithm (graph coloring): performance portability. Liszt is productive (an abstract mesh of vertices, edges, faces, and fields, concepts scientists already think in, with intuitive topological operators), portable (the same code runs on large CPU clusters, GPUs, and combinations), and fast: the constrained language lets the compiler track dependencies for locality-aware partitioning and for graph coloring that avoids synchronization, choose different parallelization strategies per platform, and customize the mesh representation (for example, not storing edge pointers if the code never uses them).

14.4 Designing domain-specific systems

1. *Good systems identify the most important cases and provide the most benefit there.* The structure of code mimics the natural structure of problems in the domain (Halide's pixel-wise view, per-vertex operations in graph processing); common operations are easy and intuitive to express; and the system performs the domain's most important optimizations. A parallel programming system whose "convenient" abstractions preclude the best-known implementation strategies will almost always fail.
2. *Good systems are simple.* They have a small number of key primitives (Halide's few scheduling primitives, Hadoop's map and reduce), so the compiler and runtime can focus on optimizing them. Good architects ask "do we really need that?": every domain-specific primitive needs a strong performance or expressivity justification.
3. *Good primitives compose.* Composition gives wide application scope even within a domain (Halide's loop ordering and interleaving primitives express a wide range of schedules) and makes optimization generalize: if the system can optimize A and B, it can optimize programs combining them. A feature that does not compose with the existing features should not be added, or should be added differently; a sign of good design is use in applications the designers never anticipated.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 14 (Fall 2023).

Lecture 15: Transactional Memory 1

Lecture 13 built locks, barriers, and lock-free data structures in software from machine-level atomic operations (fetch-and-op, test-and-set, compare-and-swap, LL/SC), and it is easy to write buggy programs with them (atomicity violations, deadlock). Transactional memory raises the level of abstraction for synchronization further. Locks force a trade-off between the degree of concurrency (performance) and the chance of races and deadlock (correctness): coarse-grained locking (one lock for a whole data structure or all of shared memory) has low concurrency but a higher chance of correctness, while fine-grained locking (such as hand-over-hand locking) has high concurrency but a lower chance of correctness.

15.1 Programming with transactions

A bank deposit is a read-modify-write that must be atomic with respect to other operations on the account. With locks and with a transaction:

```
1 void deposit(Acct account, int amount)
2 {
3   lock(account.lock);
4   int tmp = bank.get(account);
5   tmp += amount;
6   bank.put(account, tmp);
7   unlock(account.lock);
8 }
```

```
void deposit(Acct account, int amount)
{
  atomic {
    int tmp = bank.get(account);
    tmp += amount;
    bank.put(account, tmp);
  }
}
```

The `atomic` construct is DECLARATIVE: the programmer states what to do (keep this code atomic), not how (no explicit use or management of locks), and the system implements the synchronization needed. It could implement `atomic` with locks; the implementations here use OPTIMISTIC CONCURRENCY, serializing only in cases of true contention (read-write or write-write conflicts). In general, declarative abstractions say what should be done ("execute these 1000 independent tasks", "perform this set of operations atomically"), and imperative ones say how ("spawn N worker threads that pull work from a shared queue", "acquire a lock, perform operations, release the lock").

Definition: A MEMORY TRANSACTION is an atomic and isolated sequence of memory accesses, inspired by database transactions.

- ATOMICITY (all or nothing): on COMMIT, all memory writes of the transaction take effect at once; on ABORT, none of them appear to take effect, as if the transaction never happened.
- ISOLATION: no other processor can observe the transaction's writes before it commits.

- Serializability: transactions appear to commit in a single serial order, although the semantics do not guarantee which order.

Many properties maintained for a single address in a coherent memory system are thus maintained for the whole set of reads and writes of a transaction: a transaction that reads X, Y, Z and writes A, X has all its writes observed by other processors, or none of them, effectively at the same time.

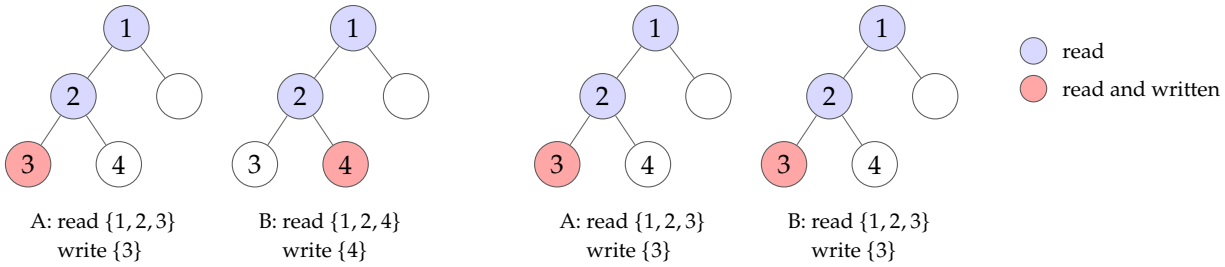
15.2 Motivation

Thread-safe data structures. Java's `HashMap`, a hash table with a linked list per bucket, is not thread safe, but has no lock overhead when synchronization is not needed. Java 1.4's synchronized wrapper converts any map into a thread-safe variant with one coarse-grained lock guarding all accesses:

```
public Object get(Object key) {
  synchronized (myHashMap) {   // per-hashmap lock guards all
                               // accesses to hashMap
    return myHashMap.get(key);
  }
}
```

which is thread safe and easy to program, but limits concurrency and scales poorly. Finer-grained locks (one per bucket) are thread safe but incur lock overhead even when synchronization is not needed; they reduce contention and outperform coarse locks as processors are added, for both a hash table and a balanced tree. A transactional map simply encloses every operation in an atomic block, `atomic { return m.get(key); }`: thread safe and easy to program, with performance and scalability that depend on the workload and the implementation of `atomic`.

Concurrency on a tree. To modify nodes 3 and 4 of a tree safely, hand-over-hand locking makes the two updates contend for the locks on nodes 1 and 2: locks on the path to node 3 can delay the update to node 4. With transactions, A reads $\{1, 2, 3\}$ and writes $\{3\}$ while B reads $\{1, 2, 4\}$ and writes $\{4\}$. Neither writes data accessed by the other, so there are no read-write or write-write conflicts and both proceed concurrently. If both modify node 3, they conflict and must be serialized (Figure 34). In measurements with TCC, a hardware TM system, transactions match fine-grained locks for the hash map and the balanced tree as processors are added.



(a) no conflict (b) conflict on node 3

Figure 34: Read and write sets of two transactions on a tree.

The same idea makes `PushLeft` on a doubly linked list thread safe: allocate and initialize the node outside, and wrap the four pointer reads and updates (`leftSentinel`, `oldLeftNode`, `qn->left`, `qn->right`, `leftSentinel->right`, `oldLeftNode->left`) in one `atomic` block.

Failure atomicity. In a lock-based

```
void transfer(A, B, amount) {
  synchronized(bank) {
    try {
      withdraw(A, amount);
      deposit(B, amount);
    }
    catch(exception1) { /* undo code 1*/ }
    catch(exception2) { /* undo code 2*/ }
    ...
  }
}
```

the programmer must supply undo code case by case and track what to undo and how; some side effects may become visible to other threads, and an uncaught case can deadlock the system. With `atomic { withdraw(A, amount); deposit(B, amount); }` the system handles all exceptions not explicitly managed by the programmer: the transaction aborts and its memory updates are undone, so no partial updates are visible and no locks are held by a failing thread.

Composability. If `transfer` locks `A` and then `B`, thread 0 running `transfer(A, B, 100)` and thread 1 running `transfer(B, A, 200)` can deadlock. Composing lock-based code requires system-wide policies (such as a global lock order) that break software modularity; the programmer is caught between coarse locks (low performance) and fine-grained locks (mistakes lead to deadlock). Transactions compose gracefully, at least in theory: the programmer declares the global intent, the transaction in `transfer` subsumes any in `withdraw` and `deposit` (the outermost transaction defines the atomicity boundary), and the system serializes `transfer(A, B, 100)` with `transfer(B, A, 200)` while running `transfer(A, B, 100)` and `transfer(C, D, 200)` concurrently.

The promise of transactional memory: synchronization that is as easy to use as coarse-grained locks; performance often as good as fine-grained locks, with automatic read-read and fine-grained concurrency, and performance portability (a locking scheme for 4 CPUs may not suit 64), so most of the benefit of expert fine-grained locking comes with much less development time; failure atomicity and recovery (no lost locks, and recovery is abort plus restart); and safe, scalable composition of software modules.

Note. `atomic {}` is not `lock()` plus `unlock()`. Atomic is a high-level declaration of atomicity that does not specify an implementation; a lock is a low-level blocking primitive that provides neither atomicity nor isolation on its own. Locks can implement atomic blocks, but locks have uses beyond atomicity, so not all locks can be replaced by atomic regions. For example, if thread 1 executes `flagA = true; while (flagB == 0);` inside `synchronized(lock1)` and thread 2 executes `flagB = true; while (flagA == 0);` inside `synchronized(lock2)`, the two critical sections run concurrently and both threads proceed; as atomic blocks, isolation hides each flag write from the other thread until commit, so neither loop can exit. Atomic blocks also eliminate many data races but not atomicity violations: if thread 1 sets `ptr = A` in one atomic block and reads `B = ptr->field` in another, thread 2 can set `ptr = NULL` in between, because a logically atomic sequence was split into two blocks.

15.3 Implementing transactional memory

A TM system must provide atomicity and isolation while maintaining as much concurrency as possible. The two key design questions are the DATA VERSIONING POLICY, how the system manages uncommitted (new) and previously committed (old) versions of data for concurrent transactions, and the CONFLICT DETECTION POLICY, how and when it determines that two concurrent transactions conflict.

Definition: In EAGER VERSIONING (undo-log based), a write updates the memory location directly and the old value is kept in an UNDO LOG; commit discards the log, and abort restores memory from it. In LAZY VERSIONING (write-buffer based), writes go to a transaction WRITE BUFFER; commit flushes the buffer to memory, and abort discards it (Figure 35).

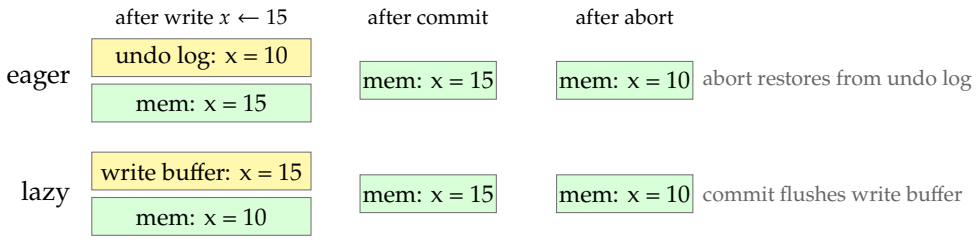


Figure 35: Eager and lazy versioning for a transaction writing $x \leftarrow 15$ when memory holds $x = 10$.

Eager versioning writes to memory immediately, hoping the transaction will not abort, and deals

with aborts when it must: commits are fast (the data is already in memory), but there is undo bookkeeping on every store, aborts are slow, and there are fault-tolerance issues (consider a crash in the middle of a transaction). Lazy versioning writes to memory only when it must: aborts are fast (just clear the buffer) with no fault-tolerance issues, but commits are slow.

Definition: A READ-WRITE CONFLICT occurs when transaction A reads an address written by a pending (not yet committed) transaction B; a WRITE-WRITE CONFLICT occurs when two pending transactions both write the same address. To detect conflicts the system tracks each transaction's READ SET (addresses read) and WRITE SET (addresses written).

PESSIMISTIC (eager) detection checks for conflicts immediately on each load or store, on the theory that if a rollback is coming it is better to do it now and avoid wasted work; a CONTENTION MANAGER decides whether to stall or abort on a conflict, with policies that make the common case fast. OPTIMISTIC (lazy, commit-time) detection checks only when a transaction tries to commit ("hope for the best and sort out conflicts at commit"), giving priority to the committing transaction while others may abort later. The following cases show the difference, with an aggressive contention manager in which the writer wins:

	Scenario	Outcome
Pessimistic detection		
1	T0 reads A, writes B and C; T1 touches none of them	success
2	T0 writes A; T1 then reads A: the check detects the conflict and T1 stalls until T0 commits, then continues	early detect, stall
3	T0 reads A; T1 writes A: the writer wins and T0 restarts; T0's new read of A stalls until T1 commits	abort
4	T0 writes A, T1 writes A (T0 restarts), T0 writes A again (T1 restarts), and so on	no progress (livelock)
Optimistic detection		
1	T0 reads A and writes C; T1 writes B and commits; T0 commits	success
2	T1 reads A; T0 writes A and commits first, so T1's check fails and T1 restarts, rereads A, and commits	abort
3	T0 reads A and commits; T1 writes A and commits later	success
4	T0 and T1 both read and write A; T1 commits first, T0 restarts and then commits	forward progress

Pessimistic detection finds conflicts early (undoing less work and turning some aborts into stalls), but gives no forward-progress guarantee (avoiding livelock in case 4 needs a policy), may abort more in some cases, needs fine-grained communication (a check on every load and store), and puts detection on the critical path. Optimistic detection guarantees forward progress (a committing transaction always succeeds) and allows bulk communication and conflict detection, but detects conflicts late and can still have fairness problems.

System	Versioning and detection
Hardware TM	
Stanford TCC	lazy, optimistic
MIT LTM, Intel VTM	lazy, pessimistic
Wisconsin LogTM	eager, pessimistic
(eager + optimistic)	not practical
Software TM	
Sun TL2	lazy, optimistic reads and writes
MS OSTM	lazy, optimistic reads, pessimistic writes
Intel STM	eager, optimistic reads, pessimistic writes
Intel STM	eager, pessimistic reads and writes

The optimal design remains an open question and may differ for hardware, software, and hybrid systems.

15.4 Software transactional memory

An STM compiler turns an atomic block into calls to software barriers (STM functions) that do the TM bookkeeping:

```
atomic {                    tmTxnBegin()
  a.x = t1                  tmWr(&a.x, t1)
  a.y = t2                  tmWr(&a.y, t2)
  if (a.z == 0) {           if (tmRd(&a.z) != 0) {
    a.x = 0                   tmWr(&a.x, 0);
    a.z = t3                  tmWr(&a.z, t3)
  }                         }
}                           tmTxnCommit()
```

The barriers handle versioning, read- and write-set tracking, and commit, using locks, timestamps, data copying, and so on. Functions called both inside and outside transactions require function cloning or dynamic translation.

The runtime keeps a per-thread TRANSACTION DESCRIPTOR, used for conflict detection, commit, and abort, which includes the read set, the write set, and the undo log or write buffer. Each datum has a TRANSACTION RECORD, a pointer-sized record guarding shared data that tracks its transactional state: *shared*, accessed by multiple readers (using a version number or a shared reader lock), or *exclusive*, accessed by one writer (using a writer lock that points to the owner), much as hardware cache coherence works. Data is mapped to transaction records either by embedding a record in each object (Java/C#, next to the vtable pointer), or by hashing fields or array elements (in Java/C#) or addresses at cache-line or word granularity (in C/C++) into a global table of records.

The granularity of conflict detection is a trade-off. *Object* granularity has a cheap mapping and

exposes optimization opportunities but causes false conflicts (a transaction writing `a.x` and `a.y` conflicts with one reading `a.z`). *Element or field* (word) granularity reduces false conflicts and improves concurrency, at higher time and space overhead. *Cache-line* granularity (multiple words) matches hardware TM and reduces the storage for transaction records, but is hard for programmers and compilers to analyze. Systems can mix granularities per type, for example element level for arrays and object level otherwise.

An example STM algorithm. Intel's McRT STM uses eager versioning, optimistic reads, and pessimistic writes, with timestamps for version tracking: a GLOBAL TIMESTAMP, incremented when a writing transaction commits, and a per-transaction LOCAL TIMESTAMP, the global timestamp when the transaction last validated. The 32-bit transaction record's least significant bit is 0 if writer-locked and 1 if not; the remaining bits hold the version number (timestamp of the last commit) when unlocked, or a pointer to the owning transaction when locked.

- *Read* (optimistic): read memory directly (eager versioning); validate by checking that the record is unlocked and the data version is at most the local timestamp, and if not, validate the whole read set for consistency; insert into the read set; return the value.
- *Write* (pessimistic): validate (unlocked, version at most the local timestamp); acquire the lock; insert into the write set; create an undo-log entry; write in place.
- *Read-set validation*: get the global timestamp; for each item in the read set, abort if it is locked by another transaction or its version exceeds the local timestamp; set the local timestamp to the global timestamp from the first step.
- *Commit*: atomically increment the global timestamp by 2 (the least significant bit is the lock bit); if the old global timestamp exceeds the local timestamp, validate the read set (to catch recently committed transactions); for each item in the write set, release the lock and set the version number to the new global timestamp.

Example: *STM in action.* Transaction X1 copies object `foo` (version 3, $x = 9$, $y = 7$) into `bar` (version 5, $x = 0$, $y = 0$): `t = foo.x; bar.x = t; t = foo.y; bar.y = t;`, while X2 reads `t1 = bar.x; t2 = bar.y;`. X2 must see `bar` as $[0, 0]$ or $[9, 7]$. With a timestamp per object (and no global or local timestamps in this simplified example), X1 records reads $\langle foo, 3\rangle$ twice, locks `bar` and records the write $\langle bar, 5\rangle$ with undo entries $\langle bar.x, 0\rangle$ and $\langle bar.y, 0\rangle$, writes in place, and commits, setting `bar`'s version to 7. X2 read `bar.x` at version 5 and waited while `bar` was locked; its read of `bar.y` sees version 7, inconsistent with its earlier read, so X2 aborts (and on retry reads $[9, 7]$).

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 15 (Fall 2023).

Lecture 16: Transactional Memory 2

A TM implementation chooses a data versioning policy (eager or lazy), a conflict detection policy (optimistic or pessimistic), and a detection granularity (object, word, cache line, . . .), as described in Lecture 15. In software TM the compiler adds code for versioning and conflict detection (an STM barrier is instrumentation code), and the runtime keeps a transaction descriptor per thread (status, read and write sets, . . .) and a transaction record per datum (locked or version). This unit covers the challenges of STM, hardware TM, and real hardware support.

16.1 Challenges and optimization of STM

STM systems face the overhead of software barriers, function cloning, robust contention management, and the memory model (strong versus weak atomicity). Monolithic barriers such as `tmWr(&a.x, t1)` hide redundant logging and locking from the compiler. Decomposing them into finer operations, opening an object for reading or writing and logging a field for undo, exposes the redundancies:

```
atomic {                    txnOpenForWrite(a)
  a.x = t1                  txnLogObjectInt(&a.x, a)
  a.y = t2                  a.x = t1
  if (a.z == 0) {           txnOpenForWrite(a)
    a.x = 0                 txnLogObjectInt(&a.y, a)
    a.z = t3                a.y = t2
  }                         txnOpenForRead(a)
}                           if (a.z != 0) {
                              txnOpenForWrite(a)
                              txnLogObjectInt(&a.x, a)
                              a.x = 0
                              txnOpenForWrite(a)
                              txnLogObjectInt(&a.z, a)
                              a.z = t3
                            }
```

so the compiler can remove them: object a need be opened for writing only once (which also covers reading `a.z`), and `a.x` need be logged only once, leaving

```
txnOpenForWrite(a)
txnLogObjectInt(&a.x, a)
a.x = t1
txnLogObjectInt(&a.y, a)
a.y = t2
if (a.z != 0) {
  a.x = 0
  txnLogObjectInt(&a.z, a)
  a.z = t3
}
```

which has fewer and cheaper STM operations. With such compiler optimizations, single-thread overhead over a thread-unsafe baseline falls below 40% over no concurrency control and below 30% over lock-based synchronization.

Example: *STM barriers.* In an STM with optimistic reads, pessimistic writes, and eager versioning, the region `atomic { obj.f1 = 42; }` becomes

```
tx = GetTxDescriptor();            // Assume a way to get transaction descriptor
OpenForWriteTx(tx, obj);
LogForUndoIntTx(tx, obj, offset);  // record old value in the undo log
obj.f1 = 42;
```

Why hardware support? STM's barriers cost 2–8× per thread: on the three-tier server benchmark Vacation, STM's speedup at 16 processors stays far below ideal. Measured single-thread STM is 1.8–5.6× slower than sequential code (on Vacation and k-means), and most of the time goes to read barriers and commit, since most applications read more data than they write. In the short term this discourages parallel programming; in the long term it wastes energy. STM also lacks strong atomicity, which is costly to provide purely in software.

Hardware support comes in three forms:

	HW-accelerated STM (HASTM, SigTM, USTM)	HTM (TCC, LTM, VTM, LogTM)	Hybrid TM (Sun Rock)
Write versioning	SW	HW	SW
Conflict detection	SW	HW	HW

Hardware-accelerated STM starts from an STM, identifies the bottlenecks, and provides simple hardware primitives while keeping software barriers. Hardware TM implements versioning and conflict detection directly in hardware, with no software barriers. Hybrid TM combines an HTM with an STM, switching modes when needed based on transaction characteristics and available resources.

16.2 Hardware transactional memory

Definition: In HARDWARE TRANSACTIONAL MEMORY (HTM), data versioning is implemented in the caches, which hold the write buffer or the undo log, with new cache-line metadata tracking each transaction's read and write sets. Conflict detection uses the cache coherence protocol: coherence lookups detect conflicts between transactions, with snooping or directory coherence. A register checkpoint is taken at transaction begin to restore the execution context on abort.

Cache lines are annotated with an R BIT, set on loads, marking data read by the transaction, and a W BIT, set on stores, marking data written; the bits may be per word or per line, and are gang-cleared on commit or abort. (Eager versioning needs a second cache write for the undo log.) Coherence requests check the bits: a shared request to a W word is a read-write conflict; an exclusive (intent to write) request to an R word is a write-read conflict; an exclusive request to a W word is a write-write conflict.

A lazy-optimistic HTM. The CPU gains the ability to checkpoint register state (available in many CPUs) and TM state registers (status, pointers to abort handlers, . . .); cache lines gain R and W bits next to the dirty (D) and valid (V) bits. For the transaction `Xbegin; Load A; Load B; Store C <= 5; Xcommit` (Figure 36):

- *begin*: initialize CPU and cache state and take the register checkpoint;
- *loads*: serve cache misses as needed and set the R bits of `A` and `B`;
- *store*: serve the miss if needed and set the W bit of `C`. The line is not loaded in an exclusive state, because in a lazy-optimistic design the store stays private until commit, and requesting exclusivity now would announce the write to other processors early;
- *commit* (fast two-phase commit): validate by requesting exclusive (RdX) access to the write-set lines if needed (here an upgrade of `C`), then gang-reset the R and W bits, turning the write-set data into valid, dirty data;
- *detect and abort*: if a remote processor commits a transaction writing `A` and `D`, its exclusive requests (`upgradeX A`, `upgradeX D`) are looked up in the read and write sets. The remote write of `A` conflicts with the local read of `A`, so the local transaction aborts: invalidate the write set, gang-reset the R and W bits, and restore the register checkpoint.

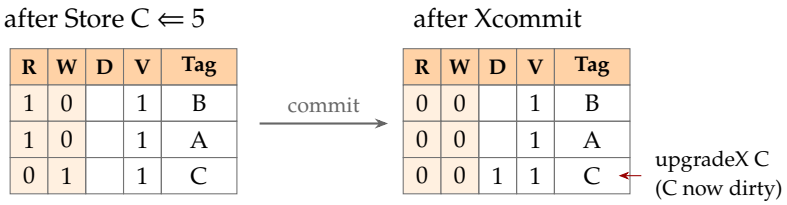


Figure 36: R and W bits of a lazy-optimistic HTM before and after commit of `Load A; Load B; Store C <= 5`.

On Vacation, HTM performs 2–7× better than STM, runs within 10% of sequential code on one thread, and scales efficiently with the number of processors.

16.3 Transactional coherence and consistency

Transactional Coherence and Consistency (TCC) uses TM as the coherence mechanism: all code runs in transactions all the time, and a successful commit updates memory and all caches in the system.

Example: *A TCC execution.*

P1	P2	P3
`Begin T1`	`Begin T2`	`Begin T4`
`Read A`	`Read A`	`Read E`
`Write A, 1`	`Write E, 3`	`Write B, 6`
`Write C, 2`	`Commit T2`	`Write C, 7`
`Read D`	`Begin T3`	`Read F`
`Commit T1`	`Write C, 4`	`Commit T4`
	`Read A`	
	`Write E, 5`	
	`Commit T3`	

Three processors run the transactions at left with lazy, optimistic TM, at most one commit per execution step across all processors, and the commit of one transaction allowed to overlap the begin of a transaction it forces to re-execute; we minimize the number of steps.

Writing each step as action (read set; write set), with all memory initially 0:

Step	P1	P2	P3
1	begin T1	begin T2	begin T4
2	R A (A:0; –)	R A (A:0; –)	R E (E:0; –)
3	W A,1 (A:0; A:1)	W E,3 (A:0; E:3)	W B,6 (E:0; B:6)
4	W C,2 (A:0; A:1, C:2)	commit T2	begin T4 again
5	R D (A:0, D:0; A:1, C:2)	begin T3	R E (E:3; –)
6	commit T1	W C,4 (–; C:4)	W B,6 (E:3; B:6)
7		R A (A:1; C:4)	W C,7 (E:3; B:6, C:7)
8		W E,5 (A:1; C:4, E:5)	R F (E:3, F:0; B:6, C:7)
9			commit T4
10		commit T3	

T2's commit in step 4 writes `E`, which T4 has read, so T4 aborts and re-executes, now reading $E = 3$.

T1's commit writes `A` and `C`; no pending transaction has read either, so nothing aborts, and T3, starting afterward, reads $A = 1$. T4's commit writes `B` and `C`, which T3 writes but does not read, so T3 survives (lazy versioning buffers its write of `C`). T3 commits last, leaving $A = 1$, $B = 6$, $C = 4$, $E = 5$: the serial order is T2, T1, T4, T3.

16.4 HTM in Intel Haswell

Haswell added RESTRICTED TRANSACTIONAL MEMORY (RTM) instructions: `xbegin`, which takes a pointer to a fallback address used on abort (for example a code path with a spin lock), `xend`, and `xabort`. The implementation tracks read and write sets in the L1 cache. The processor ensures all memory operations of a transaction commit atomically, but it may abort a transaction for many reasons (for example, eviction of a line in the read or write set), so it does not guarantee progress, hence the fallback path; Intel's optimization guide gives guidelines for increasing the probability that transactions do not abort.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 16 (Fall 2023).

Lecture 17: Hardware Specialization

A well-optimized parallel implementation of a compute-bound application runs about 40× faster on a quad-core laptop than single-threaded C compiled with `gcc -O3`, so a lot of software uses modern CPUs inefficiently. But the CPU itself is inefficient, even when used as well as possible.

17.1 Heterogeneous processing

Most real applications have complex workload characteristics: components that parallelize widely and components that are hard to parallelize; components amenable to wide SIMD execution and components with divergent control flow; components with predictable data access and components with unpredictable access that may still cache well. Hence the idea that the most efficient processor is a heterogeneous mixture of resources: use the most efficient tool for each job.

- The Intel Skylake (2015) Core i7 has four CPU cores, an integrated Gen9 GPU (graphics and media), a shared last-level cache, and a system agent (display, memory, and I/O controllers). The CPU and graphics cores share the memory system and the L3 cache, enabling low-latency, high-bandwidth communication between them, and the graphics cores are cache coherent with the CPU cores. A laptop may add a power-hungry discrete GPU on a PCIe bus, used only for graphics-intensive applications while the integrated low-power graphics handles the UI.
- Mobile processors such as the NVIDIA Tegra X1 (four ARM Cortex A57 cores for applications, four low-power A53 cores, one Maxwell SMM with 256 CUDA cores) and the Apple A11 Bionic (two high-performance and four low-performance ARM cores, a three-core Apple GPU, an image processor, a neural engine for DNNs, and a motion processor).
- GPU-accelerated supercomputers: Frontier (9,472 64-core 2 GHz AMD CPUs, 37,888 Radeon Instinct MI250X GPUs, 10 PB of DRAM, 21 MW, $600M).

17.2 Energy-constrained computing

Supercomputers are energy constrained because of their sheer scale and the cost of power for the machine and its cooling; datacenters to reduce cooling cost and physical space; mobile devices by battery life and heat dissipation. Since

$$\text{power} = \underbrace{\frac{\text{ops}}{\text{second}}}_{\text{performance}} \times \underbrace{\frac{\text{joules}}{\text{op}}}_{\text{energy efficiency}},$$

under a fixed power budget, higher performance requires better energy efficiency, and specialization (fixed-function hardware) gives better energy efficiency. Rules of thumb, relative to high-quality C code on a CPU: throughput-maximized architectures such as GPU cores give about 10× better performance per watt, assuming code maps well to wide data-parallel execution and is compute bound; fixed-function ASICs (application-specific integrated circuits) can approach 100–1000× or more, assuming code is compute bound and not floating-point math.

Why is a general-purpose processor so inefficient? Executing one instruction involves reading it (address translation, instruction cache access), decoding it (translation to micro-ops, micro-op cache access), checking dependencies and pipeline hazards, identifying an available execution resource, using the decoded operands to read the register file, moving data to the execution resource, performing the arithmetic, moving the result back, and writing the register file. The circuit for the operation itself, say an 8-bit logical OR, is tiny by comparison. SIMD reduces this overhead by amortizing one instruction's control over many data elements, for computations that apply the same operation to many elements. Even so, in H.264 video encoding implemented with SIMD instructions, the functional units consume only a small fraction of the energy; most goes to instruction fetch, register fetch, pipeline control and registers, and the data cache. For the fast Fourier transform, an ASIC matches the performance of one CPU core with about 1/1000 of the chip area and about 1/100 of the power, and GPU cores are about 5–7× more area efficient than CPU cores.

On mobile devices, higher efficiency lets a device run faster for a fixed time (higher clock, more cores, lower latency for critical tasks, more at once) or run at fixed performance for longer, enabling "always-on" functionality that was previously impossible (Google Glass recorded about 40 minutes per charge; early Siri needed a button press, while smart speakers listen continuously).

17.3 Specialized processors

- *GPUs* are themselves heterogeneous: besides the programmable SIMD cores, they contain graphics-specific fixed-function units for texture mapping (warping and filtering images to apply detail to surfaces), tessellation (computing fine-scale geometry from coarse geometry), clipping and culling, rasterization (determining which pixels a triangle overlaps), and Z-buffering and blending, plus a work distributor.
- *Digital signal processors* (DSPs) are programmable but have simpler instruction-stream control paths and complex instructions that perform many operations each (SIMD or VLIW, "very long instruction word", where one instruction specifies several different operations), amortizing the cost of control. The Qualcomm Hexagon DSP, used for modem, audio, and image processing in Snapdragon SoCs, performs 29 RISC operations per cycle in the innermost loop of an FFT.
- The *Anton* supercomputer (D. E. Shaw Research) simulates the time evolution of proteins with 512 ASICs for particle-particle interactions, a throughput-oriented subsystem for FFTs, and a custom low-latency network designed for the communication patterns of N-body simulation.

- DNN accelerators such as Google's *TPU* and Intel's Lake Crest (formerly Nervana), the subject of many architecture papers. Google's Pixel Visual Core is a programmable image processing unit whose cores are 16×16 grids of 16-bit multiply-add ALUs, claimed to be 10–20× more efficient than a GPU at image processing.
- A modern smartphone (Google Pixel 2: Qualcomm Snapdragon 835 SoC plus the Pixel Visual Core) combines a multi-core ARM CPU (4 big and 4 little cores), a multi-core GPU (3D graphics and OpenCL compute), the Hexagon DSP, a video encode/decode ASIC, an image signal processor ASIC for camera sensor pixels, a display engine that compresses pixels for the screen, and the programmable image processor and DNN accelerator.

Definition: An FPGA (field-programmable gate array) is a middle ground between an ASIC and a processor: an array of logic blocks, each with a programmable LOOKUP TABLE (LUT) and a flip-flop (register), connected by a programmable interconnect, so programmer-defined logic is implemented directly. A LUT specifies combinational logic as a table; a 6-input, 1-output LUT6 in Xilinx Virtex-7 FPGAs is a 64-entry table (a 6-input AND outputs 1 only for entry 63), and a 40-input AND can be built by chaining the outputs of eight LUT6s (delay 3). Modern FPGAs also devote much area to hard gates: memory blocks (SRAM) and DSP blocks (multipliers).

Microsoft's Project Catapult offloaded part of Bing's document-ranking logic to FPGAs attached by PCIe to dual-socket servers, and FPGAs are now available in Amazon's cloud (EC2 F1). The options form a spectrum, from most programmable to most efficient: a domain-specific accelerator such as the TPU is limited to a domain programmed through DSLs (such as DNN frameworks); FPGAs are difficult to program, and making them easier is an active research area; ASICs are not programmable and cost tens to hundreds of millions of dollars to design, verify, and create, so they are used for tasks such as video encode/decode, audio playback, camera RAW processing, and perhaps neural networks.

Option	Efficiency vs. CPU	Programmability
Energy-optimized CPU	1×	easiest to program
Throughput-oriented processor (GPU)	~10×	
Programmable DSP		
Domain-specific accelerator (TPU)	~20×	limited domain, programmed via DSLs (e.g. DNNs)
FPGA / reconfigurable logic	~50× (jury still out)	difficult to program
ASIC	~100–1000×	not programmable; tens to hundreds of millions of dollars to design, verify, and create

Heterogeneity has challenges. The software developer must map an application onto a heterogeneous collection of resources, designing algorithms that decompose into components that each map well to a different processing element, and the scheduling problem becomes more complex. The

hardware designer must choose the right mixture: too few throughput-oriented resources lower peak throughput on parallel workloads, too few sequential resources leave performance limited by the sequential part, and it is unclear how much area to dedicate to a specific function such as video.

17.4 Programming accelerators: Spatial

A general-purpose processor runs instruction streams on cores with caches; a special-purpose processor (accelerator) instead has custom memories (SRAMs), custom registers, custom control, and wide ALU arrays fed from DRAM. Designing one traditionally requires years of expertise in a hardware description language (VHDL, Verilog, Chisel). High-level synthesis tools (Vivado HLS, Intel OpenCL, Xilinx SDAccel) accept restricted C with pragmas, but sacrifice performance and are difficult to use. Spatial is a DSL for accelerator design that lets performance-oriented programmers specify parallelism (specialized compute) and locality (specialized memories and data movement). It expresses parallel patterns as parallel and pipelined datapaths, independent and dependent parallelism, hierarchical control, explicit memory hierarchies, and explicit parameters exposed to the compiler, with simple APIs for CPU-accelerator communication; its dataflow model is more intuitive than CUDA's threads.

- *Memory templates*: an explicit memory hierarchy of typed storage, such as `DRAM[UInt8](H,W)`, `SRAM[UInt8](C)`, registers `Reg[Double]`, `FIFO[Float](D)`, `LineBuffer[Int](R,C)`, and `ShiftReg[UInt8](R,C)`; explicit dense and sparse transfers such as `buffer load image(i, j::j+C)` and `buffer gather image(a, 10)`; and streams such as `StreamIn` and `StreamOut`.
- *Control templates*: `Accel {...}` (blocking) or `Accel(*) {...}` (non-blocking) interaction with the CPU, and arbitrary state machines and loop nests (`FSM`, `Foreach`, `Reduce`) with implicit control signals.
- *Design parameters*: explicit parallelization factors (`Reduce(0)(N by 1 par P)`), implicit or explicit control schemes (`Stream.Foreach`), explicit sizes for strides and buffers, and implicit memory banking and buffering for parallel accesses.

Example: *Inner product.* In C: `for (int i = 0; i < N; i++) output = output + vec1[i] * vec2[i];`. In Spatial, the accelerator tiles the vectors into on-chip SRAMs and reduces:

```
// Set up host and memory pointers
val output = ArgOut[Int]
val vec1 = DRAM[Int](N)
val vec2 = DRAM[Int](N)

// Create accelerator
Accel {
  // Allocate on-chip memories
  val tile1 = SRAM[Int](tileSize)
  val tile2 = SRAM[Int](tileSize)

  // Specify outer loop
  Reduce(output)(N by tileSize){ t =>
    // Prefetch data
    tile1 load vec1(t :: t + tileSize)
    tile2 load vec2(t :: t + tileSize)

    // Multiply-accumulate data
    val accum = Reg[Int](0)
    Reduce(accum)(tileSize by 1 par 2){ i =>
      tile1(i) * tile2(i)
    }{ _ + _ }
  }{ _ + _ }
}
```

Spatial generates a three-step controller: load, intra-tile accumulate, and full accumulate (the outer `Reduce`'s final step), and it manages communication with DRAM. The `par 2` annotation makes the inner multiply-accumulate datapath two wide. Tiling is easy (doubling `tileSize` enlarges the SRAMs), and annotating the outer loop as `Pipeline.Reduce` makes its stages run in pipelined fashion, with buffering of memories (double buffering) inferred.

The Spatial programmer specifies the algorithm as a hierarchy of controllers, the memory hierarchy, explicit data movement, and tiling factors, parallelism, and scheduling. The compiler banks and buffers memories to maximize performance and minimize resources, generates hardware for the target (FPGA, CGRA, ASIC), and gives performance-debugging feedback. A TensorFlow-to-FPGA flow translates a dataflow graph of domain-specific operators (convolution, pooling, normalization) into a hierarchical dataflow graph of parallel patterns (map, reduce), compiles that into Spatial's hierarchical dataflow of tiled pipelines with explicit memory and compute units (line buffers, shift registers, SRAMs, register files), and then into Chisel/Verilog for the FPGA tools.

17.5 Streaming execution and attention

FlashAttention (Lecture 10) avoids materializing the $N \times N$ attention matrix, but requires modifying the algorithm and extra computation. Softmax is a three-step operation on each row: an exponential,

a row-wise reduction (row sum), and a division, $P_{ij} = e^{S_{ij}} / \sum_j e^{S_{ij}}$. So attention is the chain $S = QK^T$, $S' = \text{Exp}(S)$, $r = \text{RowSum}(S')$, $P = S'/r$, $O = PV$.

In the KERNEL-BASED EXECUTION MODEL, each step is a separate kernel that reads its inputs from memory and writes its outputs back, so the $N \times N$ intermediates S, S', and P are materialized (large memory footprint) and read and written through memory (high bandwidth).

Definition: In the STREAMING EXECUTION MODEL, the computations are mapped spatially onto hardware stages that pass data piecewise through on-chip FIFOs: each stage loads input pieces, computes, and enqueues output pieces that the next stage dequeues. Fusion comes for free: the $N \times N$ matrices are never materialized and the intermediates never travel to and from memory, without algorithmic changes or extra computation.

For attention (Figure 37), tiles Q_1 and $K_1^T, K_2^T, \ldots$ stream through QK^T to produce blocks $S_1, S_2, \ldots$ of a row, which flow through Exp; the row sum needs the whole row, so the exponentials $S'_1, S'_2, \ldots$ wait in a FIFO, then are divided by r to give $P_1, P_2, \ldots$, which are multiplied with $V_1, V_2, \ldots$ and accumulated into O_1, while the pipeline already works on the next output tile. FlashAttention can improve this further: the FIFO of size N (the sequence length) buffers S' only because the division must wait for the row sum, and FlashAttention breaks that dependency by reordering operations and using a running sum with rescaling, at the cost of a little more computation, eliminating the FIFO.

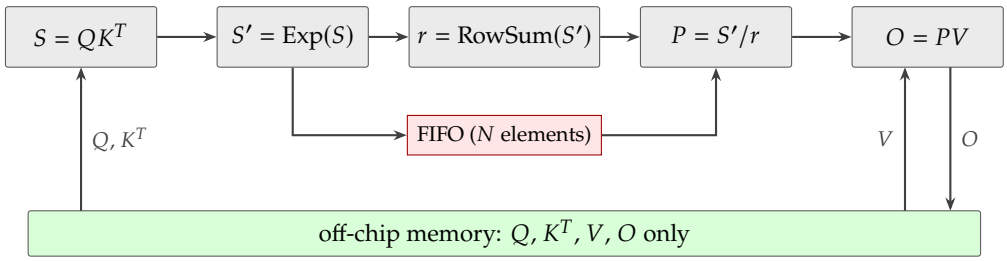


Figure 37: Streaming execution of attention. Only Q, K^T, V, and O live in off-chip memory; the FIFO holds a row of S' until its row sum is known.

Compared with FlashAttention in the kernel model, streaming execution offers more parallelism, since it pipelines the computation for different output tiles whereas fused kernels cannot overlap them, and it does not require hand-written fused CUDA kernels, which are hard to fuse deeply because of the limit on registers per SM; writing the program with FIFOs fuses operations automatically, and the compiler can generate the fused execution.

Specialized accelerators thus offer 100–1000× energy efficiency improvements, but designing one trades performance against resource utilization (parallelism, locality) and requires insight into the application: where the bottleneck is and whether the implementation is compute or memory bound. Spatial helps explore these trade-offs rapidly and design accelerators at a high level.

17.6 Moving less data

The second way to reduce energy, after specialized processing, is to move less data. In mobile design, always seek to reduce the data transferred from memory: earlier communication was minimized to avoid stalls, now also to save energy. Ballpark numbers (Dally, Olson): an integer operation costs about 1 pJ and a floating-point operation about 20 pJ (just the operation, without instruction overhead); reading 64 bits from a small local SRAM 1 mm away costs about 26 pJ, and from low-power mobile DRAM (LPDDR) about 1200 pJ. Reading 10 GB/s from memory therefore costs about 1.6 W, while the entire power budget of a mobile GPU is about 1 W (and an iPhone 6 battery holds about 7 Wh, against 99 Wh in a laptop). Recomputing values rather than storing and reloading them can be better for energy efficiency. In 45 nm CMOS (Han, 2016), a 32-bit floating-point operation costs about 0.9 pJ, a local SRAM access about 5 pJ, and a 32-bit LPDDR load about 640 pJ. Data movement thus limits performance (many processing elements generate many memory requests and need more bandwidth, leading to bandwidth-limited execution) and also has a high energy cost.

Three trends in energy-optimized computing follow.

- *Compute less*: computing costs energy, so parallel algorithms that do more work than their sequential counterparts may not be desirable even if they run faster.
- *Specialize compute units*: heterogeneous processors with CPU-like and throughput-optimized (GPU-like) cores; fixed-function units for audio, motion-sensor processing, video encode/decode, and image processing or computer vision; specialized instructions (the growing set of AVX vector instructions, AES-NI for AES encryption); and programmable soft logic (FPGAs).
- *Reduce bandwidth requirements*: exploit locality by restructuring algorithms to reuse on-chip data as much as possible, and compress data aggressively, performing extra computation to compress application data before transferring it to memory (likely with fixed-function hardware for general compression and decompression).

Heterogeneous parallel processing uses a mixture of computing resources, latency-optimized sequential cores, throughput-optimized parallel cores, and domain-specialized fixed-function processors, that fits the mixture of needs of target applications, as in mobile processors, servers, and supercomputers. The traditional rule of good system design, simple general-purpose components, no longer holds for systems optimized for performance per watt, which want a collection of components that meets the performance requirement and minimizes energy. The challenge of using these resources effectively is pushed up to the programmer: how to write efficient, portable programs for heterogeneous architectures is a current research problem.

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 17 (Fall 2023).

Lecture 18: Accessing Memory and Course Wrap Up

18.1 How DRAM works

In the memory system, a core issues load and store instructions; the last-level cache (LLC) issues memory requests to the memory controller; and the controller sends commands to DRAM over a 64-bit memory bus.

Definition: A DRAM array stores each bit in one transistor and one capacitor (which holds charge). In the example array, rows hold 2 Kbits; a row buffer of 2 Kbits connects the array to 8 data pins leading to the memory controller. Loading one byte takes four steps, each about 10 ns (for DDR3-1600):

1. precharge (PRE): ready the bit lines;
2. row activation (RAS): transfer the row into the row buffer;
3. column selection (CAS);
4. transfer the data onto the bus.

Loading the next byte from an already active row skips precharge and row activation.

DRAM access latency is therefore not fixed. The best case reads from the active row and costs only the column access time (CAS). The worst case, with bit lines not ready and a new row to read, costs precharge plus row activation plus column access (PRE + RAS + CAS). Precharge readies the bit lines and writes the row buffer's contents back into the array, since reading a row is destructive.

The data pins are the scarcest resource, yet with one byte per access they are busy only a small fraction of the time, during the CAS part of each access (Figure 38a). Two techniques raise pin utilization:

- Burst mode amortizes latency over larger transfers: each DRAM command describes a bulk transfer, and bits are placed on the output pins in consecutive clocks (Figure 38b).
- Banks: a DRAM chip consists of multiple banks that share the same pins (only one transfer at a time). Banks pipeline memory requests: the chip can precharge, activate rows, and send column addresses to one bank while transferring data from another, achieving high data pin utilization (Figure 38c).

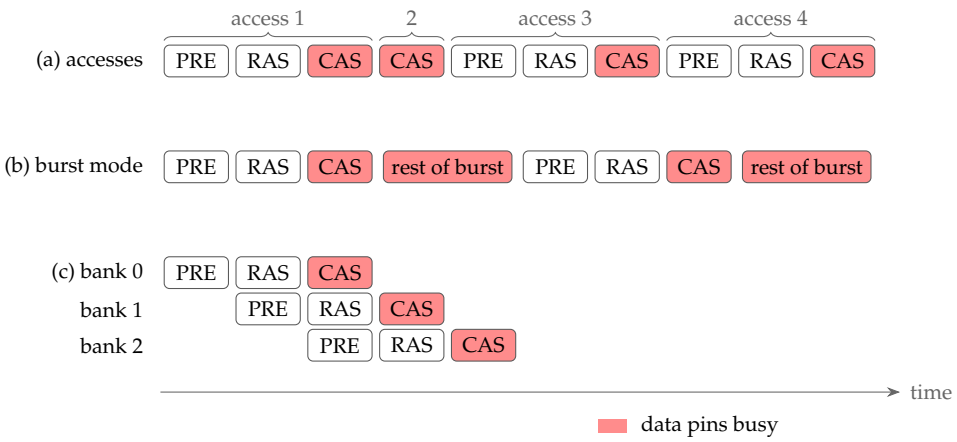


Figure 38: DRAM command timelines: (a) individual accesses leave the data pins mostly idle; (b) burst mode transfers more data per command; (c) banks overlap commands to one bank with transfers from another. Schematic.

DIMMs and cache lines. A DIMM organizes multiple chips, for example eight DRAM chips on a 64-bit memory bus; it appears to the memory controller as a single DRAM module with higher capacity and a wider interface, giving higher aggregate bandwidth, but the minimum transfer granularity becomes 64 bits. The memory controller converts a physical address into a DRAM bank, row, and column. If consecutive physical addresses mapped to the same row of the same chip, one 64-byte (512-bit) cache line would be served by a single chip, its bytes sent consecutively over the same 8 pins, taking 64 clocks. Instead, physical addresses are interleaved across the chips at byte granularity: in response to "read bank B, row R, column 0", the eight chips transmit bits 0–7, 8–15, . . . , 56–63 in parallel; the next request (column 8) returns bits 64–127, and so on, so the line takes 8 clocks. Modern DRAMs would use burst mode here to transfer multiple consecutive columns.

The memory controller is a scheduler of memory requests. It receives load and store requests from the LLC, keeps a request queue per bank, and must balance conflicting goals: maximize throughput, minimize latency, and minimize energy. A common policy is FR-FCFS (first-ready, first-come-first-serve): service requests to the currently open row first, maximizing row locality, and other rows in FIFO order. The controller may also coalesce multiple small requests into large contiguous ones to exploit burst modes.

Channels. Adding memory channels effectively widens the memory bus and increases throughput. In a dual-channel system each channel can issue independent commands, reading a different row and column; a simpler setup drives the same command to multiple channels from a single controller.

Example: *DDR4 bandwidth.* The Intel Core i7-7700K uses DDR4-2400 (DDR stands for "double data rate"): a 64-bit bus $\times 1.2\,\text{GHz} \times 2$ transfers per clock = 19.2 GB/s per channel, and two channels give 38.4 GB/s, with a CAS latency of about 13 ns.

DRAM access latency depends on many low-level factors, such as the state of the chip (row hit or miss, whether precharge is needed) and buffering and reordering of requests in the memory controller. Much of the complexity of a modern multi-core processor has moved into the memory controller, which schedules tens to hundreds of outstanding requests and maps physical addresses onto the geometry of the DRAMs, an area of active research.

18.2 Bringing memory closer

A modern architectural challenge is improving memory performance by decreasing the distance data must move, locating memory closer to the processors, which allows shorter but wider interfaces.

Definition: In 3D STACKING, DRAM dies are stacked and connected by THROUGH-SILICON VIAS (TSVs) that run through the chips and provide highly parallel connections between the DRAMs and a base LOGIC LAYER, which acts as the memory controller and manages requests from the processor. A silicon INTERPOSER serves as a high-bandwidth interconnect between the DRAM stack and the processor die. Technologies include the Micron/Intel Hybrid Memory Cube and HIGH-BANDWIDTH MEMORY (HBM), with a 1024-bit interface per stack, offering more bandwidth, high power efficiency, and a small form factor.

GPUs have adopted HBM. The AMD Radeon Fury (2015) has a 4096-bit interface (4 HBM stacks of 1024 bits) with 512 GB/s; the NVIDIA P100 (2016) has 4 HBM2 stacks (4096 bits), 720 GB/s peak, and $4 \times 4 = 16$ GB; the NVIDIA H100 (2022) has 6 HBM3 stacks (6144 bits), 3.2 TB/s peak, and 80 GB. The Xeon Phi (Knights Landing) has 16 GB of in-package stacked DRAM (MCDRAM) that can be used as a 16 GB last-level cache or as a separate address space ("flat mode"), allocated with `hbw_malloc`; Intel claims about the same latency as DDR4 with about 5× the bandwidth and 5× less energy per bit transferred.

The memory bottleneck is being addressed in many ways: by application programmers, who schedule computation to maximize locality and arithmetic intensity (minimizing data movement); and by new hardware, through intelligent DRAM request scheduling, bringing data closer to the processor (deep cache hierarchies, 3D stacking), wider memory systems, and research on placing limited forms of computation in or near memory and on hardware-accelerated compression. The general design principles are to locate data storage near the processor, move computation to the data, and compress data, trading extra computation for less data transfer.

18.3 Course wrap up

Higher performance now comes from a combination of increased parallelism and hardware specialization: Intel Core i7 CPUs with integrated GPUs and media units, the Xeon Phi (72 cores, 16-wide SIMD, 4-way multithreading), NVIDIA Maxwell GPUs (each SMM a 32-wide SIMD core running up to 2048 CUDA threads), heterogeneous SoCs such as the Apple A9 (multi-core CPU, multi-core GPU, and media ASICs), and FPGAs. Today's software is surprisingly inefficient

compared with the capability of modern machines, and increasingly so as machines grow more complex and more parallel. Extracting this performance can have a notable impact on many compute-intensive fields and enable new applications. With current programming systems and tools, understanding how a parallel machine works is important to achieving high performance, and a major challenge is making it simpler for programmers to extract performance from these complex machines.

The course addressed these key issues:

- identifying parallelism (or, conversely, dependencies);
- efficiently scheduling parallelism: achieving good workload balance; overcoming communication constraints (bandwidth limits, latency, synchronization) by exploiting data and computation locality, that is, managing state efficiently; and scheduling under heterogeneity (using the right processor for the job);

at many scales: heterogeneous mobile SoCs, single-chip multi-core CPUs, multi-core GPUs, CPUs and GPUs connected by a bus, clusters, and large multi-node supercomputers. It also covered abstractions for thinking about efficient code (data-parallel thinking, functional parallelism, transactions, tasks) and how throughput-oriented hardware works (multiple cores, hardware threads, SIMD, and specialization to key domains).

Source. Kayvon Fatahalian and Kunle Olukotun, *CS149: Parallel Computing*, Lecture 18 (Fall 2023).